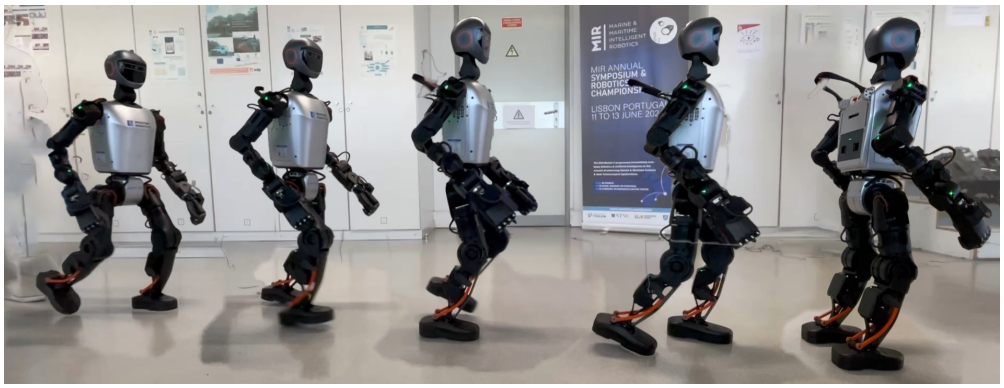
TÉCNICO
UNIVERSIDADE
DE LISBOA



Natural and Robust Humanoid Locomotion using Reinforcement Learning

Duarte Faria dos Santos

Thesis to obtain the Master of Science Degree in

Electrical and Computer Engineering

Supervisor(s): Prof. Pedro Manuel Urbano de Almeida Lima
Prof. Rita Maria Mendes de Almeida Correia da Cunha

Examination Committee

Chairperson: Prof. João Pedro Castilho Pereira Santos Gomes
Supervisor: Prof. Rita Maria Mendes de Almeida Correia da Cunha
Member of the Committee: Prof. Alexandre José Malheiro Bernardino

November 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Declaration

I declare that generative Artificial Intelligence (AI) and AI-assisted tools were used in this dissertation. Nevertheless, I take full responsibility for the content and integrity of this work. In compliance with the Guide for the Responsible Use of Artificial Intelligence at Técnico, I declare that the following tools were used:

1) To Compress Some of the Text to Meet the Maximum Set Page Count: OpenAI. 2026. ChatGPT [Large language model]. https://chat.openai.com

Acknowledgments

First of all, I would like to thank my supervisors, Professors Pedro Lima and Rita Cunha. From the moment I first contacted you both about your research interests to the opportunity to work under your joint supervision following the acquisition of the humanoid robot at ISR, thank you for everything you made possible. I am especially grateful for the countless times you made yourselves available outside our scheduled meetings, whether to help me overcome difficulties in my work or to offer personal advice when I needed it. This chapter of my life has helped me grow immensely as an individual, and your support and trust in me have been invaluable, particularly when faced with adversity.

I would also like to thank the Institute for Systems and Robotics (ISR), and in particular the IRSg and SocRob research groups, which welcomed me and taught me so much about this field. Working alongside you gave me practical knowledge I could not have gained otherwise. Being exposed to others' work and academic trajectories, whether through competitions, seminars, visits, or direct conversations, broadened my understanding of the field far beyond my own research and helped me gain confidence in my abilities along the way.

To Rodrigo Serra, thank you for your leadership and advice. To Rodrigo Coimbra, Afonso Certo, António Morais, and Inês Sousa, thank you for your continuous support, but also for being such great friends to me during this. To every member of SocRob, thank you for the hours we shared in the lab, your willingness to help whenever unexpected difficulties arose, and the environment that made this work enjoyable. I could not have asked for a better environment in which to conduct my thesis.

In particular, thank you to Rui Bettencourt and Gabriel Nunes for their constant help with the technical demonstrations. I always joked that you were the good-luck charm that kept my most important experiments from going wrong, Rui. I am also grateful to Nuno Duarte from VisLab for his availability and help with the recordings that brought human motion into this work.

To my family and close friends, thank you for bringing levity to difficult times.

And finally, to my grandmother, who believed in me every step of the way, no matter how difficult things became. Thank you for always being there for me.

Resumo

Esta tese desenvolve locomoção baseada em aprendizagem por reforço para o Booster T1, um robô humanoide cujos braços longos e pesados e mãos destras lhe conferem uma morfologia assimétrica que o controlador oficial não considera, resultando numa marcha curta e pesada, com os braços recolhidos. O trabalho centra-se num movimento natural e robusto. Um controlador para as pernas é treinado e implementado no robô real, e depois estendido ao corpo inteiro, com os braços em oposição de fase às pernas. Face ao controlador oficial, a política de corpo inteiro reduz a potência mecânica média em 41% e o RMS da taxa de variação do binário em 29,5%. Uma segunda formulação, minimizando o custo de transporte, mostra que uma oscilação contida dos braços é a solução económica para esta morfologia. A marcha é depois enriquecida com movimentos de referência redirecionados de uma biblioteca consolidada e gravações próprias de locomoção humana, organizados por velocidade, para que a marcha seja mais pronunciada a alta velocidade e contida ao abrandar. Esta política tem postura menos agachada e converge com cerca de 65% menos passos de treino. Por fim, estende-se a abordagem à transposição de alturas com perceção: uma política que observa um mapa de alturas do terreno à frente aprende subida e descida como submovimentos estruturados e, em simulação, sobe degraus até 25 cm. Esta é combinada com a política de marcha sem degradar nenhum comportamento, com transições suaves. Todas as etapas seguem o mesmo processo de treino e validação, e são replicadas em seis outros robôs humanoides atuais.

Palavras-chave: Locomoção Humanoide, Aprendizagem por Reforço, Movimento de Referência, Transposição Percetiva de Escadas, Combinação de Políticas, Transferência Simulação-Realidade

Abstract

This thesis develops reinforcement learning-based locomotion for the Booster T1, a humanoid robot whose long, heavy arms and dexterous hands give it an asymmetric morphology that the official controller does not account for, resulting in a short, stompy gait with the arms held retracted. The work centers on natural and robust movement. A legged controller is first trained and deployed on the real robot, and then extended to the whole body, with the arms swinging in anti-phase with the legs. Compared with the official controller, the deployed whole-body policy reduces mean mechanical power by 41% and torque-rate RMS by 29.5%. A second formulation that minimizes the cost of transport shows that a restrained arm swing is the economical solution for this morphology. Walking is then enhanced with reference motions retargeted from an established motion library and our own recordings of human locomotion, organized by speed so that the gait is more pronounced at high velocities and restrained as it slows down. This policy walks with a less crouched posture and converges in roughly 65% fewer training steps. Finally, the framework is extended to perceptive height traversal: a policy that observes a heightmap of the terrain ahead learns ascent and descent as structured sub-movements and, in simulation, climbs steps of up to 25 cm. It is blended with the walking policy so that neither behavior degrades and transitions remain smooth. Every stage follows the same training and validation pipeline, and is replicated on six further state-of-the-art humanoid robots.

Keywords: Humanoid Locomotion, Reinforcement Learning, Reference Motion, Perceptive Stair Traversal, Policy Blending, Sim-to-Real Transfer

Contents

Acknowledgments . . . iv
Resumo . . . v
Abstract . . . vi
List of Tables . . . x
List of Figures . . . xi
Glossary . . . xiv

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Robot Description . . . 2
1.3 Objectives . . . 3

2 Background **5**
2.1 Core Principles of Reinforcement Learning . . . 5
2.1.1 Reward . . . 6
2.1.2 Value Function and Q-Function . . . 6
2.2 Deep Reinforcement Learning . . . 7
2.2.1 TRPO and PPO . . . 8
2.2.2 DDPG and TD3 . . . 9
2.3 Energetics of Humanoid Locomotion . . . 12
2.3.1 Cost of Transport . . . 12
2.3.2 Energy Penalties in Reinforcement Learning . . . 12
2.3.3 Froude Number and Dynamic Similarity . . . 13
2.3.4 Arm Swing in Human Walking . . . 13
2.4 Motion Retargeting . . . 14

3 Literature Review **15**
3.1 Model-Based Controllers . . . 15
3.1.1 Evolution of Classical Control Approaches . . . 15
3.1.2 Model Predictive Control . . . 16
3.2 Learning-Based Controllers . . . 17
3.2.1 Key Innovations in the Core Learning Structure . . . 17

3.2.2 Applications of Reinforcement Learning Algorithms in Robotics . . . 18
3.2.3 Energy Efficiency and Emergent Gaits . . . 19
3.2.4 Research Work Towards Natural Movements . . . 19
3.2.5 Terrain Perception for Stair Traversal . . . 20

4 Locomotion Learning Framework **22**
4.1 Research Strategy . . . 22
4.1.1 Simulator Pipeline . . . 24
4.1.2 Robot Model Implementation . . . 25
4.2 Legged Locomotion Pipeline . . . 28
4.3 Whole-Body Locomotion Extension . . . 30
4.3.1 Unrestrained Upper Body Objective . . . 30
4.3.2 Energy Efficiency Objective . . . 31
4.4 Reference Motion Fine Tuning . . . 32
4.4.1 Reference Motion Library . . . 34
4.4.2 Velocity-Conditioned Reference Tracking . . . 36
4.5 Height Traversal Pipeline . . . 38
4.5.1 Stair Terrain and Curriculum . . . 39
4.5.2 Terrain Perception . . . 40
4.5.3 Step Engagement and Adaptation of the Walking Objective . . . 40
4.5.4 Ascent . . . 42
4.5.5 Descent . . . 44
4.5.6 Blending Walking and Climbing . . . 47
4.5.7 Alternating-Stride Ascent . . . 47
4.6 Real Robot Deployment . . . 48
4.7 Development and Validation Tooling . . . 49
4.8 Camera-Based Perception and Validation Robustness . . . 50

5 Experimental Evaluation and Results **52**
5.1 Legged Locomotion . . . 52
5.1.1 Training Results . . . 52
5.1.2 Validation Results . . . 54
5.1.3 Real Robot Deployment Results . . . 54
5.2 Whole-Body Locomotion . . . 59
5.3 Reference Motion Based Locomotion . . . 65
5.4 Height Traversal . . . 71
5.4.1 Ascent Results . . . 71
5.4.2 Descent Results . . . 73
5.4.3 Alternating Stride Results . . . 75
5.4.4 Live Camera to Heightmap Robustness Trials . . . 75

6 Conclusions **77**

6.1 Achievements 77

6.2 Limitations 78

6.3 Future Work 78

Bibliography **80**

A Appendix **A.1**

A.1 Training Configuration of the Policies A.2

A.2 Training Curves of the Retracted Arms Configuration A.3

A.3 Training Curves of the Rested Arms Configuration A.4

A.4 Joint Limits of the Booster T1 . . . . . A.5

A.5 Joint Velocities of the Walking Policies in Deployment A.6

A.6 Energy Efficiency Policies Across Robot Morphologies A.7

A.7 Real Deployment *vs* Simulation Run for the Other Two Walking Objectives A.9

A.8 Joint Torques of the Walking Policies in Deployment A.10

A.9 Supplementary Results of the Height Traversal Policies A.11

List of Tables

4.1 Reward Terms Modified or Introduced for the Unrestrained Objective. 30
4.2 Modified, Removed and Added Terms in the Reference-Motion Environment. 38
4.3 Walking Objective Terms Changed on the Height Traversal Objective. 42
4.4 Ascent Reward Terms. 45
4.5 Descent Reward Terms. 47

5.1 Aggregated Metrics across All Joints: Booster Policy vs Whole-Body. 62
5.2 Arm Morphology and Swing per Robot. 65
5.3 Leg Load per Stride: Whole-Body vs Reference Motion Policy. 70
5.4 High-Step Completion Rates and Measured Behavior of the Trained Ascent Policies. . . . 73

A.1 Network, Training and Domain Randomization Parameters of the Legged Locomotion and Whole-Body Policies. A.2
A.2 Joint Angle Limits in Radians for the Upper and Lower Body, as Detailed in [4]. A.5

List of Figures

1.1 Showcase of Booster T1 Line of Models. 3

2.1 PPO Architecture. 9
2.2 TD3 Architecture. 11

4.1 Structure of the Thesis Work. 23
4.2 Diagram of the Developed Simulator Pipeline. 24
4.3 Restructuring and Modifying of the XML Model for the Booster T1. 25
4.4 Robot Models Used in This Work (Part 1). 26
4.5 Robot Models Used in This Work (Part 2). 26
4.6 Side-By-Side Comparison for the Studied Robots, Both when Standing at Rest and Showing their Total Wingspan. 27
4.7 Base Underlining of Both the Policy and Network Architecture. 28
4.8 Base Underlining of Both the Learning Networks and Reward Function. 29
4.9 Retargeting with the Base and the Modified Formulation. 33
4.10 Geometry of a Single Ascent Step. 44
4.11 Geometry of a Single Descent Step. 46
4.12 Real Robot Deployment Architecture. 49
4.13 Development and Validation Tooling Across Retargeting and Simulation. 50

5.1 Main Training Results for the Retracted Arms Configuration. 53
5.2 Main Training Results for the Rested Arms Configuration. 53
5.3 Validation Results for the Retracted Arms Configuration in MuJoCo. 54
5.4 Validation Results for the Rested Arms Configuration in MuJoCo. 55
5.5 Deployment Results of the Retracted Arms Configuration Policy 55
5.6 Recorded Joint Angular Positions during Deployment. 56
5.7 Developed Locomotion Policy *vs* Original Booster Model Angular Positions. 57
5.8 Sim2Real Gap Positions. 58
5.9 Deployment Results of the Rested Arms Configuration Policy 59
5.10 Recorded Joint Position Differences during Deployment for Both Arm Configurations. . . . 59
5.11 Validation Results for the Whole-Body Policy. 60

5.12 Deployment Results of the Whole-Body Policy 61
5.13 Recorded Joint Angular Positions during Deployment of Objectives 1 (Rested Arms) and 2 (Whole-Body). 61
5.14 Robustness Trial Run Along Three Full Laps with the Whole-Body Policy. 62
5.15 Trial Run Torque Values and Distribution Analysis Compared with the Booster Policy. . . . 62
5.16 Torque Reduction Analysis. 63
5.17 Whole-Body and Energy Efficiency Policies Compared 64
5.18 Arm Swing of the Cost-Focused Policies over One Gait Cycle from Walking Straight at a Froude-matched Speed. 65
5.19 Excerpt of the Recorded MOCAP Motions and Their Respective Retargeting 66
5.20 Validation Results for the Reference Motion Enhanced Policy in Isaac Sim. 66
5.21 Validation Results for the Learned Velocity-Conditioned Behavior in MuJoCo. 67
5.22 Normalized Training Reward of the Walking Policy With and Without the Reference Motion, for the Booster T1, Unitree G1 and Booster K1. 67
5.23 Validation Results for the Reference Motion Enhanced Policy for the Remaining Robots. . 68
5.24 Deployment Results of the Reference Motion Enhanced Policy 69
5.25 Right Shoulder, Knee Pitch and Elbow Pitch Joints of the Three T1 Policies in Real Deployment. 70
5.26 Stair Ascent of the Booster T1 in Isaac Sim of 18 cm Steps. 71
5.27 Leg Pitch Joints of the Six Humanoid Robots Climbing the Same Flight of Twelve $15\,\mathrm{cm}$ Steps with $0.4\,\mathrm{m}$ Treads at a Commanded $0.5\,\mathrm{m\,s^{-1}}$. 72
5.28 T1 Stair Ascent Success in MuJoCo and Isaac Sim across Step Heights and Tread Depths. 72
5.29 Ascent Validation on Isaac Sim across all the Studied Robots. 73
5.30 Stair Descent of the Booster T1 Guided by its Head Camera. 74
5.31 T1 Hip and Knee Pitch across the Hand-Off from Stair Descent to Walking. 74
5.32 Striding Ascent of the DR02 in MuJoCo. 75
5.33 Validation of the Camera-Based Stair Pipeline 76

A.1 Average Reward and Return During Training of the Retracted Arms Configuration. A.3
A.2 Actor and Critic Losses During Training of the Retracted Arms Configuration. A.3
A.3 Average Reward and Return During Training of the Rested Arms Configuration. A.4
A.4 Actor and Critic Losses During Training of the Rested Arms Configuration. A.4
A.5 Joint Angular Velocities of the Legged Policy Deployed on the Real Robot. A.6
A.6 Joint Angular Velocities of the Reference Motion, Whole-Body and Legged Policies on the Real Robot. A.6
A.7 Energy Efficiency Policy of the Booster T1. . . . . A.7
A.8 Energy Efficiency Policy of the Booster K1. . . . . A.7
A.9 Energy Efficiency Policy of the Fourier GR3. . . . . A.7
A.10 Energy Efficiency Policy of the Unitree G1. . . . . A.8

A.11 Energy Efficiency Policy of the AgiBot A3. . . . . A.8
A.12 Real Deployment *vs* Simulation of the Whole-Body Policy. A.9
A.13 Real Deployment *vs* Simulation of the Reference Motion Policy. A.9
A.14 Joint Torques of the Reference Motion Policy Deployed on the Real Robot. A.10
A.15 Joint Torques of the Reference Motion and Whole-Body Policies on the Real Robot. . . . A.10
A.16 Full-Flight Ascent Success on the Additional Robots. A.11
A.17 Full-Flight Descent Success on the Booster T1. . . . . A.11

Glossary

CoT	Cost of Transport
DDPG	Deep Deterministic Policy Gradient
DoF	Degrees of Freedom
DRL	Deep Reinforcement Learning
GPU	Graphics Processing Unit
IL	Imitation Learning
IMU	Inertial Measurement Unit
IRSG	Intelligent Robots and Systems Group
ISR	Institute for Systems and Robotics
IST	Instituto Superior Técnico
KL	Kullback–Leibler
LQR	Linear Quadratic Regulator
MoCap	Motion Capture
MPC	Model Predictive Control
PID	Proportional Integral Derivative
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
RMS	Root Mean Square
RMSE	Root Mean Square Error
ROS	Robot Operating System
SDK	Software Development Kit
Sim2Real	Simulation-to-Reality
SISO	Single-Input Single-Output
TD	Temporal Difference
TD3	Twin Delayed Deep Deterministic Policy Gradient
TRPO	Trust Region Policy Optimization
URDF	Unified Robot Description Format

1

Introduction

1.1 Motivation

Humanoid robotics has seen an unprecedented surge in development research in the past few years, and the recent wave in availability of bleeding-edge humanoid robots for research and development has allowed for numerous top universities to be at the forefront of humanoid robot research.

Namely, bipedal locomotion has been one of the most actively studied challenges for humanoid robotics research in recent years, due to the inherent difficulty in developing these complex movements for such an intricate robot system, which is very prone to instability when in motion.

This complexity is exacerbated when having to ensure reliability in a plethora of environmental disturbances, measurement noise, and the latency inherent in real-world robotics applications. Modeling these dynamics becomes a very arduous task, so traditional approaches resort to having to perform oversimplifications of the described system and dynamics.

However, recent advances made in the field of reinforcement learning have allowed for a new way to not only create a more stable and robust motion controller, but also for development towards real deployment being made much faster [1], opening opportunities for more thorough experimentation and analysis towards refining generalized stability, establishing coordinated whole-body movement, seamlessly integrating complex tasks such as traversal through elevated slopes, ascending and descending stairs all within one locomotion system, or further increasing movement fluidity for more energy-efficient

and natural locomotion. The latter has only recently emerged as one of the main fronts in humanoid research, as recent developments in imitation learning have led a narrative to depart from rigid robotic movements, which can potentially present dangerous situations in their unpredictability and also display a tendency to systematically consume more energy in abrupt, unstructured movements, to more life-like, and controlled motions [2] [3].

Conversely, a relatively unexplored research area within humanoid robotics comes in the form of asymmetric morphological structures (i.e., a robot where its arms are considerably disproportionate to its legs), which change the system dynamics to a lesser researched structure, thus requiring adaptations of state of the art approaches to account for these differences. This thesis is placed at the intersection of these two fronts: its core is the pursuit of natural and robust movement, on a robot whose morphology makes both harder to obtain.

1.2 Robot Description

The experimental and developmental work reported in this thesis centers on the Booster T1[1], a humanoid robot acquired by the IRSG-ISR group in April 2025 to support the expansion of research into humanoid robotics at IST.

The Booster T1 line of manufactured robots is comprised of three distinct models. A 24-DOF model, as detailed in Figure 1.1a, which features non-manipulative arm links, and serves as the primary and main basis model within the Booster T1 series. Due to its status as the central model, it benefits from the most comprehensive documentation and technical support, forming the foundation for most available resources. The remaining two variants enhance the complexity of the robot by incorporating three additional degrees of freedom per arm, enabling articulated arm motions for manipulation. Additionally, one model integrates a gripper end-effector, while the other employs a dexterous robotic hand. The latter configuration constitutes the specific model utilized in the present work (pictured in Figure 1.1b).

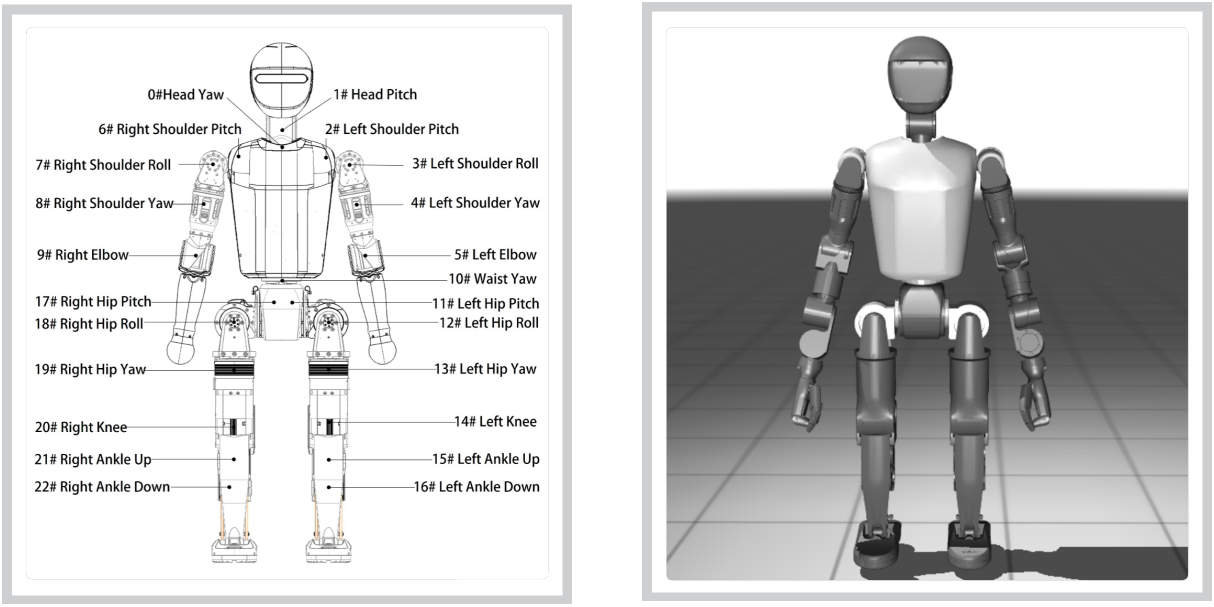
Thus, this extended version of the humanoid robot model provides many additional degrees of freedom for complex grasps and manipulation behaviors, allowing for a wide range of research opportunities relating to humanoid manipulation within our research group at IST. However, the added complexity also introduces substantial additional mass and inertia on the upper body, which for a legged system that is already inherently delicate in ensuring whole-body stability, introduces a much greater challenge in developing dynamic and robust control for locomotion.

Additionally, as can be noted in Figure 1.1b, the arms of the robot take a significant portion of the robot's entire physical structure, reaching down to as low as the robot's knee joints, as well as contributing as one of the heaviest components in the joint space group by a considerable margin. This asymmetric morphology is unlike any streamlined humanoid robot carried out in current research and thus requires a great deal of additional consideration and adaptation to this relatively unknown research environment.

The official motion controller used by the robot was hastily adapted from an alternative variant model originally designed for a gripper configuration. This controller supports only legged locomotion and

[1] Booster T1 Documentation: `https://booster.feishu.cn/wiki/DtFgwVXYxiBT8BksUPjcOwG4n4f`

requires the arms to remain permanently retracted in an unstable position to prevent self-collision. While this configuration avoids interference between limbs, it reduces movement stability and responsiveness.



(a) Booster T1 24-DOF Model with Joint Structure, Reprinted from [4].

(b) Extended Manipulation Model in Simulation.

Figure 1.1: Showcase of Booster T1 Line of Models.

Moreover, the controller is proprietary and distributed only in a pre-compiled form, which prevents any form of modification, optimization, or performance analysis. The resulting gait is characterized by very short steps, likely an adaptation to maintain balance given the retracted arm configuration, but it also produces a noticeably stompy and noisy motion. The controller was also likely trained under the gripper configuration, so a mismatch in weight distribution arises: although the overall morphology is similar, the dexterous hands are heavier than the gripper, leading to further imbalance during locomotion.

1.3 Objectives

The core of the work developed in this thesis is aimed at achieving natural and robust movement for a humanoid robot. To this end, it pursues the following objectives:

- **Legged locomotion.** Develop a reinforcement learning motion controller for the legs, with a fixed upper body, that ensures an adequate weight shift for a morphologically asymmetric robot while remaining robust to the disturbances of real deployment. With the arms retracted, as in the official Booster controller, it is compared directly with that existing baseline in stability and motion fluidity. With the arms lowered, it examines whether the gait holds when the posture and mass distribution of the upper body change.
- **Whole-body locomotion.** Extend the controller to the whole body. Under an unrestrained objective, the arms join the movement through explicit shaping, for more complex and life-like motion. Under an energy efficiency objective, the motion of the arms is left to emerge from minimizing the

cost of transport, to examine how this shapes the locomotion of distinct robot morphologies and whether parallels can be drawn with human walking.

- **Reference motions.** Enhance the walking with reference motions, drawn from an established library and complemented by our own recordings of human locomotion, to induce the nuances and coordination of human walking that a handcrafted reward does not capture. The references are organized by walking speed, so the robot adjusts its movement to the speed it is going: expressive at high velocities, and more reserved as it slows down.
- **Perceptive height traversal.** Extend the controller to instability-prone environments. Rather than stumbling into a step and relying on a recovery reflex, the robot observes the terrain ahead and learns both ascent and descent through structured sub-movements, which make the motion poised, controlled and natural. This climbing behavior is then blended with the reference-motion walking at deployment, so the locomotion covers both flat ground and stairs without either behavior being compromised.
- **Sim-to-real transfer pipeline.** Study how the learned behaviors transfer from simulation to real deployment, through an experimental approach that analyzes the models during training, in validation through sim-to-sim across three different robotics simulators, and lastly on the real robot itself.
- **Cross-robot study.** Train every approach on six further state-of-the-art humanoid robots, to examine how each behavior holds across different stature and morphology.

Part of this work was presented at the Portugal–U.S. Partnerships Summit, in October 2026, as part of a joint research project between ISR-Lisbon and the University of Texas at Austin.

2

Background

The developed work is based on reinforcement learning and its implementation on complex decision problems, so a proper contextualization of the base concepts, as well as the state-of-the-art methods currently used in most robotic learning applications, is required for the subsequent discussion around the adopted architectural decisions. It then covers the energetics of legged locomotion, on which the energy efficiency analysis is built, and the principles of motion retargeting, on which the reference motion library is built.

2.1 Core Principles of Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning that deals with sequential decision-making. It aims to mimic how the human brain learns: through trial and error, given feedback from each action. Thus, for a certain **agent**, that acts as the decision-making entity, placed within an **environment**, that refers to the system that is controlled by the agent, and also everything that exists outside of said system with which it interacts with, actions are performed at each time instant, which may or may not alter the agent's representation of the state space, and this interaction between the agent and environment will yield a numerical **reward** that measures the desirability of the made decision in that state.

The main objective of RL is to develop a policy that will map an optimal action a within the agent's action space $\mathcal{A}$, for any state s in the state space $\mathcal{S}$. This policy, denoted by π, can represent the *probability* of selecting an action a when the agent is in state s. If defined as **stochastic** $\pi(s, a)$ it is

represented as

$$\pi(s, a) = P(a|s), \tag{2.1}$$

and, when defined as **deterministic** $\mu(s)$, it encodes a specific action $a \in \mathcal{A}$ that is *always* taken when in state $s \in \mathcal{S}$ as denoted by

$$\mu(s) = a. \tag{2.2}$$

2.1.1 Reward

Rewards play a central role in RL, as the actions rewarded and their assigned values directly determine the agent's performance. As a result, extensive research has explored methods to modify and structure rewards for the many possible tasks that employ RL. The most prevalent technique is reward shaping, which addresses the convergence of suboptimal policies caused by an exploitation of initial random or biased estimates during the initial stages of training, by constructing a dense reward signal that actively guides the optimization process, rather than providing feedback only upon task completion. This approach has been demonstrated to have a large impact in agent performance [5], especially in RL problems with highly complex goals.

The formulation of the reward function is the way RL-based approaches define how the policy is optimized for its intended objective, and must be conceptualized thoroughly to ensure the desired behavior. However, as the complexity of the state space, action set, environment, or intended goal increases, achieving effective policy optimization becomes more challenging [6] [7]. This added complexity results in greater computational demands and potential convergence issues during learning. Furthermore, when the objective involves highly specific or complex motions, designing an appropriate reward function becomes particularly difficult [8], as it requires an accurate representation of every nuance of the desired movement to produce a smooth and natural behavior.

Policy optimization encompasses seeking to maximize a **cumulative return** that accounts for both the immediate reward received in the current state s, and all future rewards. This is key because often the immediate reward lacks the capacity to characterize how a present action will influence subsequent results. Furthermore, a greater emphasis should be put on the closest rewards, rather than those further in the future, whilst still ensuring a stabilized system, so a **discount factor**, that controls the importance of future rewards, must be included. By definition, the discounted cumulative return is written as

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \tag{2.3}$$

where $r_t \in \mathbb{R}$ is the reward at time t, and $\gamma \in [0, 1)$ is the discount factor. Lower values of γ emphasize immediate gains, while values close to one encourage long-term optimization. And finally, since rewards are given by the environment at the next iteration, the time-step term for the reward in the sum is $t+k+1$.

2.1.2 Value Function and Q-Function

A framework must exist to quantify the desirability of occupying a particular state or executing a specific action within that state, evaluated through the lens of expected cumulative future returns, as

presented in (2.3), achievable under a designated policy. This provides the formal definition of Value and Q-Functions in RL respectively.

As such, given a policy π, the **value function** quantifies the expected discounted cumulative return or reward when starting at **state** s and applying the policy π

$$V_\pi(s) = \mathbb{E}_\pi \left[R_t | s_t = s \right]. \tag{2.4}$$

The **action-value** function, also commonly known as Q-function, differs by representing the value in the sense of expected discounted cumulative return of starting at **state** s, taking **action** a and applying π thereafter

$$Q_\pi(s, a) = \mathbb{E}_\pi \left[R_t | s_t = s, a_t = a \right]. \tag{2.5}$$

By assigning value to all possible actions in a given state, algorithms that utilize the Q-function can greedily optimize a policy whose action yields the highest Q-values at any given state

$$\pi^*(s) = \arg\max_a Q_{\pi^*}(s, a), \tag{2.6}$$

while with the value function, a sequential sum over transition probabilities and rewards over the next time steps is required.

Given that the cumulative return R_t can be written as

$$R_t = r_t + \gamma R_{t+1}, \tag{2.7}$$

both functions can be expressed recursively using the Bellman equations:

$$V_\pi(s_t) = \mathbb{E}_\pi[r_{t+1} + \gamma V_\pi(s_{t+1})] \tag{2.8}$$

$$Q_\pi(s_t, a_t) = \mathbb{E}_\pi[r_{t+1} + \gamma Q_\pi(s_{t+1}, a_{t+1})]. \tag{2.9}$$

These equations are fundamental to many RL algorithms, as they form the mathematical foundation for policy optimization.

2.2 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) solves fundamental limitations with traditional RL methods regarding high-dimensional state spaces by leveraging deep neural networks as function approximators. Rather than storing separate values for each state-action pair, DRL uses neural networks to approximate value functions and policies, enabling agents to effectively process high-dimensional observations in continuous spaces, such as joint positions and velocities from a high DoF robot.

This development has unlocked new possibilities for solving complex robotic control problems through reinforcement learning, and made way for numerous state-of-the-art algorithms to have been researched in recent years. Selecting the most appropriate DRL algorithm to implement and adapt is a pivotal deci-

sion for this work, requiring a careful match between the algorithm's strengths and the specific demands of the intended task.

In this context, the discussion will focus exclusively on DRL algorithms that employ **Policy Gradient** methods, that directly optimize the policy parameters ϕ by computing gradients of the expected return, denoted as the objective function, with respect to those parameters and updating them via gradient ascent.

2.2.1 TRPO and PPO

Trust Region Policy Optimization (TRPO) [9] is an **on-policy** DRL algorithm, which is characterized by a learning structure where policy updates rely solely on data or trajectories sampled by the current policy being updated. Thus, possible instances where the data generated by a bad policy could *contaminate* the policy optimization step can be mitigated, however it comes at the cost of either utilizing less data than what would be possible when improving the policy, or requiring a very long time to generate a sufficient amount of data for every changing policy.

In terms of architecture, this algorithm upholds an **actor-critic** framework, a hybrid structure comprised of an actor network, that controls the agent's behavior and aims to refine a **stochastic** parametrized policy $\pi_\theta(a|s)$ by maximizing a policy gradient, alongside a critic network that actively learns a value function as a baseline to evaluate the performance of the actor, which will in turn reduce the variance of gradient estimates.

TRPO executes policy gradient optimization by maximizing a surrogate objective function $L(\phi)$, a method that diverges from standard policy gradient algorithms, which inherently suffer from high variance, in the sense that additional measures are employed to ensure that the updated policy remains within a specific proximity to the previous policy. This is achieved via the following policy optimization step

$$\phi_{k+1} = \arg\max_{\phi} L_k(\phi) \tag{2.10}$$

$$L_k(\phi) = \mathbb{E}_{s,a\sim\pi_{\phi_k}} \left[\frac{\pi_\phi(a \mid s)}{\pi_{\phi_k}(a \mid s)} A_{\phi_k}(s,a) \right], \tag{2.11}$$

which averages over all states and actions sampled from the old policy ϕ_k, the probability ratio $\frac{\pi_\phi(a|s)}{\pi_{\phi_k}(a|s)}$, which accounts for the differences between the new policy ϕ and the one sampled from ϕ_k, along with the advantage function $A_{\phi_k}(s,a)$, which quantifies the difference between the estimated return and the old value baseline for that state. Additionally, the following constraint must always be kept:

$$D_{KL}^{max}(\pi_{\phi_k}||\pi_\phi) \leq \delta, \tag{2.12}$$

which accounts that the Kullback–Leibler divergence, specifically due to both policies being probabilistic distributions, between the previous and updated policy must remain close within a certain trust region of size δ.

Thus, policy improvement in TRPO is formulated as an approximate second-order optimization problem, because it must maximize the surrogate objective in (2.11) subject to the KL-divergence constraint

in (2.12) at each policy update. This is implemented by using a Fisher approximation to the Hessian of the KL divergence and solving the resulting linear system for the search direction with a conjugate gradient algorithm, which introduces a substantial computational load at each optimization step.

The critic network updates its parameters by minimizing the squared value function error objective $\mathbb{E}[(V_\theta(s_t) - \hat{V}_t(s_t))^2]$ with standard gradient descent, where $\hat{V}_t$ is a target value that is computed as

$$\hat{V}_t(s_t) = \sum_{k=0}^{T-1} \gamma^k r_{t+k+1} + \gamma^T V_{\theta_k}(s_{t+T}), \tag{2.13}$$

where for a T step return, the first term $\sum_{k=0}^{T-1} \gamma^k r_{t+k+1}$ corresponds to the discounted sum of observed rewards and the second term $\gamma^T V_{\theta_k}(s_{t+T})$ is a bootstrapped estimate of the remaining future value using the previous critic parameters from the sampling gathering phase.

Proximal Policy Optimization (PPO) [10] improves on TRPO and looks to mitigate these inherent computational demands, whilst maintaining the same algorithm structure and value function estimation.

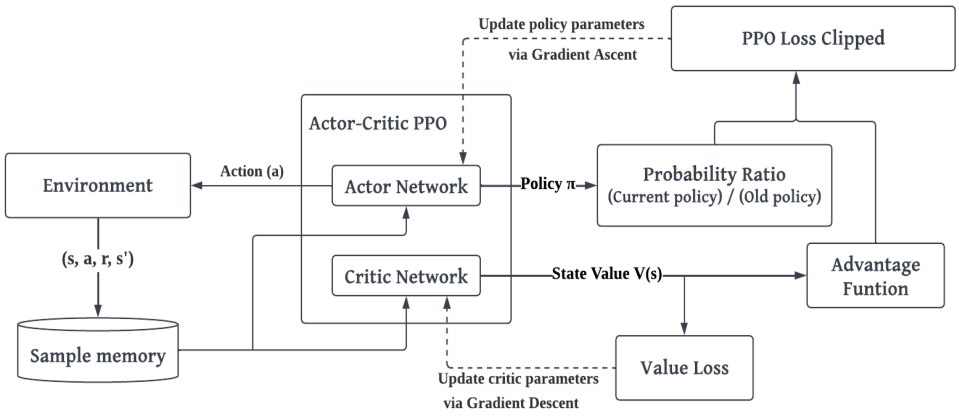


Figure 2.1: PPO Architecture. Adapted from [11].

The algorithm replaces the hard KL-divergence constraint in the policy gradient optimization step with a modified first-order **clipped surrogate objective**, defined as

$$L_k^{\text{CLIP}}(\phi) = \mathbb{E}_{s,a\sim\pi_{\phi_k}}\left[\min\left(\frac{\pi_\phi(a \mid s)}{\pi_{\phi_k}(a \mid s)} A_{\phi_k}(s,a),\ \text{clip}\left(\frac{\pi_\phi(a \mid s)}{\pi_{\phi_k}(a \mid s)},\ 1-\epsilon,\ 1+\epsilon\right) A_{\phi_k}(s,a)\right)\right], \tag{2.14}$$

where a clipped ratio $\text{clip}\left(\frac{\pi_\phi(a|s)}{\pi_{\phi_k}(a|s)},\ 1-\epsilon,\ 1+\epsilon\right)$ will restrict large deviations in the probability ratio, to reduce possibly unstable updates, but most notably removes the constraint in the optimization problem, and so heavily reduces computing time. This has made PPO a state-of-the-art, DRL algorithm widely used in most RL robotics applications.

2.2.2 DDPG and TD3

Deep Deterministic Policy Gradient (DDPG) [12] is an **off-policy** DRL algorithm, enabling policy updates to leverage data and trajectories $(s_t, a_t, r_{t+1}, s_{t+1})$ sampled by previous policies, which are collected into a replay buffer $\mathcal{D}$. This characteristic makes DDPG substantially more sample-efficient than on-policy algorithms. It also holds an actor-critic architecture, just as the other previously described algorithms. However, the actor now optimizes a **deterministic** parametrized policy network, as it is built

on the deterministic policy gradient theorem, which demonstrates that for a deterministic policy $\mu_\phi(s)$, the gradient of the objective can be expressed as an **expectation** over **states** alone, circumventing integration over actions. This formulation yields more efficient gradient estimation in high-dimensional continuous action spaces compared to stochastic policy gradients, which require integration across the action space. As such, the actor is updated via **gradient ascent** of the following policy gradient over an expectation of states s sampled from the replay buffer $\mathcal{D}$:

$$\nabla_\phi J = \mathbb{E}_{s\sim\mathcal{D}}\left[\nabla_\phi Q_\theta(s_t, \mu_\phi(s_t))\right]. \tag{2.15}$$

Since the algorithm requires gradients of the return with respect to actions to update the deterministic policy, the critic learns an **action-state** value function $Q_\theta(s, a)$ to evaluate performance.

Both actor and critic networks maintain target networks, with parameters $\phi^{'}$ and $\theta^{'}$ respectively, which update at lower frequencies to address the moving target problem. These target networks stabilize the Bellman target, allowing the critic to learn more reliably. The critic is updated through **temporal difference** (TD) error estimation, computing the loss between the predicted Q-value and a target y_t obtained from:

$$y_t = r_{t+1} + \gamma Q_{\theta'}(s_{t+1}, \mu_{\phi'}(s_{t+1})), \tag{2.16}$$

$$L_k(\theta) = \mathbb{E}_{(s_t, a_t, r_{t+1}, s_{t+1})\sim\mathcal{D}}\left[(Q_\theta(s_t, a_t) - y_t)^2\right], \tag{2.17}$$

where the expectation is taken over batches of transitions sampled from the buffer $\mathcal{D}$ of collected data.

Target networks receive periodic soft updates via **Polyak averaging**, which blends the updated network parameters with the old target network parameters:

$$\phi^{'} \leftarrow \rho\phi^{'} + (1-\rho)\phi, \tag{2.18}$$

$$\theta^{'} \leftarrow \rho\theta^{'} + (1-\rho)\theta, \tag{2.19}$$

where ρ controls how slowly the target networks follow the non-target actor and critic, effectively acting as a low-pass filter over their parameter trajectories.

A critical limitation of DDPG emerges from its learning structure: the algorithm exhibits a tendency toward **overestimation bias**. This occurs because the same target network is used to both select and evaluate actions in the target Q-value calculation. In environments with function approximation errors or poor exploration, this can lead to inflated Q-value estimates, which propagate through learning and degrade policy performance.

Twin Delayed Deep Deterministic Policy Gradient (TD3) [13] builds upon DDPG and reduces the impact of these deviations by introducing three main modifications:

- **Clipped Double Q-Learning** on target calculation, which involves restructuring the critic network architecture to employ two independent critic networks, Q_{θ_1} and Q_{θ_2}, each with corresponding target networks. The target Q-value is computed using the minimum of the two Q-estimates,

reducing the impact of overestimation:

$$y_t = r_{t+1} + \gamma \min_{i=1,2} Q_{\theta_i'}(s_{t+1}, \mu_{\phi'}(s_{t+1})), \tag{2.20}$$

which effectively mitigates optimistic bias and produces more conservative value estimates. Both critics are trained on identical targets but may diverge due to different initialization and sampling, providing complementary perspectives on state-action value.

- The actor in TD3 is updated **less frequently** than the critics, whilst maintaining the same equation in (2.15). In the majority of the algorithms' implementations, the actor is updated half as many times as the critic. This delay allows critics to stabilize their Q-value estimates before the policy changes, reducing the magnitude of policy gradient errors.

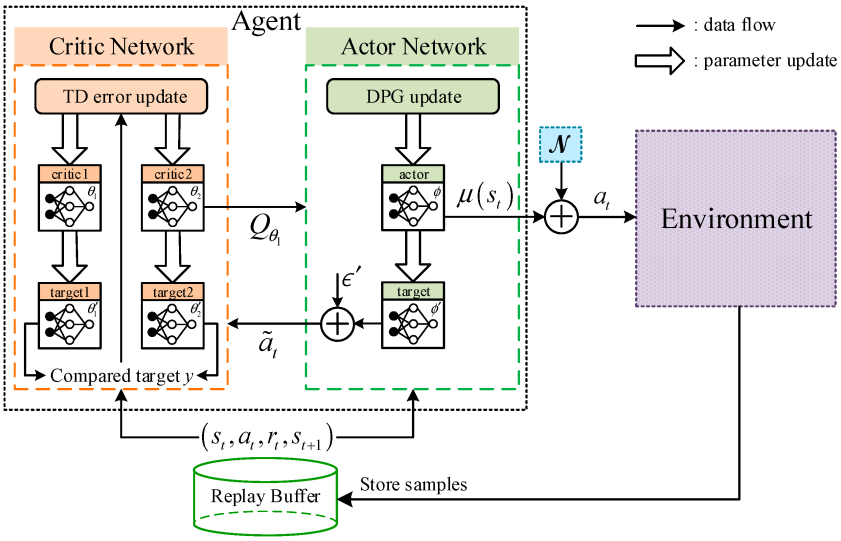


Figure 2.2: TD3 Architecture. Adapted from [14].

- And lastly, to further reduce variance and mitigate the effects of deterministic policies in environments with function approximation errors, TD3 adds clipped noise to the target policy action:

$$\tilde{a}(s_{t+1}) = \text{clip}(\mu_{\phi'}(s_{t+1}) + \epsilon^{'}, a_{\text{min}}, a_{\text{max}}) \tag{2.21}$$

where $\epsilon^{'} \sim \mathcal{N}(0, \sigma^2)$ is sampled noise, and a_{min} and a_{max} corresponding to set bounds in the action space. This introduces regularization that reduces Q-value overestimation at the extremes of the action space and encourages smoother value landscapes.

These modifications make TD3 one of the most popular algorithms in recent reinforcement learning developmental work in continuous control domains, and as it is primarily situated as one of the more stable and sample-efficient approaches to policy optimization, it has provoked a recent resurgence in interest in the last couple of years [15], as the rising complexity in agent state and action spaces for more sophisticated DRL problems have been trending to require and heavily rely on the learning algorithm being as sample-efficient as possible, for an adequate and feasible learning procedure to be established. This characteristic comes at a price, as TD3's reliance on multiple critics, target networks, and sensitive

hyperparameters can make it more difficult to tune in large state and action spaces [13]. Starting from a solid basis for the tuning of elementary hyperparameters and then progressively increasing state complexity can help mitigate this difficulty.

2.3 Energetics of Humanoid Locomotion

2.3.1 Cost of Transport

Comparing the energy efficiency of locomotion across systems of different size, mass and speed requires a normalized measure. The standard choice is the **cost of transport** (CoT), introduced by Gabrielli and von Kármán as the **specific resistance** of vehicles [16] and later adopted in animal biomechanics [17]. It is the energy spent per unit weight per unit distance traveled,

$$\mathrm{CoT} = \frac{E}{m\,g\,d} = \frac{P}{m\,g\,v}, \tag{2.22}$$

where E is the energy consumed over a distance d, P the average power, m the total mass, g the gravitational acceleration and v the average forward speed. The CoT is dimensionless: it can be read as the fraction of the body's weight that would have to be pushed horizontally, over the same distance, to spend the same energy. This makes it directly comparable across robots and animals of very different scales.

Two variants are common, depending on which energy is counted [18]. The **electrical** (or total) cost of transport c_{et} uses the power drawn from the battery and includes actuator, electronics and transmission losses. The **mechanical** cost of transport c_{mt} uses only the positive mechanical work done by the actuators on the joints, $P = \sum_j \max(\tau_j \dot{q}_j, 0)$, and therefore isolates the efficiency of the motion itself from that of the hardware. Negative (braking) work is not subtracted, because most actuators cannot regenerate energy and never recover it [18]. Its cost is not ignored, however, as the current drawn to brake, like other losses that grow with the square of the torque, such as Joule heating, appears in c_{et} rather than in c_{mt}. Human walking costs $c_{mt} \approx 0.05$ and a metabolic $c_{et} \approx 0.2$, whereas the Honda ASIMO was estimated at $c_{et} \approx 3.2$ [18]; the passive-dynamics-based Cornell Ranger reached $c_{et} \approx 0.28$ over a $65\,\mathrm{km}$ walk [19]. This gap of more than an order of magnitude between conventional humanoids and humans motivates treating energy efficiency as an explicit objective rather than a by-product.

2.3.2 Energy Penalties in Reinforcement Learning

In reinforcement learning, energy is often included as a penalty on instantaneous power, $-\sum_j |\tau_j \dot{q}_j|$, or on squared torques. Penalizing power prices energy per unit **time**, and the motion that consumes least power per unit time is not moving at all. If this term carries enough weight to influence the gait, it competes directly with velocity tracking, and the policy can lower its cost by walking slower than commanded or not walking at all.

The cost of transport instead prices energy per unit **distance**. Standing still offers no saving, so the term is minimized by moving efficiently at the commanded speed rather than by not moving. It

also provides the right accounting for whole-body motion: an arm swing is charged for the power it consumes, but if it reduces the power the legs need, through angular momentum cancellation, the total CoT falls and the swing is credited for it. A penalty on the arms alone, or on each joint separately, charges the swing for its power without crediting the saving in the legs, and so always favors keeping the arms still. Because the CoT is dimensionless, a single weight also expresses the same relative price on every robot, regardless of mass or actuator strength, which is a precondition for comparing the emergent gaits of different morphologies. Prior work has shown that, while tracking a commanded speed, minimizing energy is enough for natural gaits to emerge in legged robots, without terms that shape the gait directly [20]. As a reward, the CoT requires care near standstill, where the division by the forward speed diverges.

2.3.3 Froude Number and Dynamic Similarity

Robots of different leg lengths cannot be meaningfully compared at the same absolute speed: $0.5\,\mathrm{m\,s^{-1}}$ is a brisk walk for a small humanoid but a slow amble for a full-size one. Walking is commonly modeled as an inverted pendulum vaulting over the stance leg, in which the gravitational and centripetal forces set the gait dynamics. Their ratio is the **Froude number**,

$$\mathrm{Fr} = \frac{v^2}{g\,L}, \tag{2.23}$$

where L is the leg length (hip to ground). The **dynamic similarity hypothesis** states that geometrically similar animals moving at the same Froude number use dynamically similar gaits, with equal relative stride lengths and duty factors [21] [22]. The inverted pendulum model also bounds walking at $\mathrm{Fr} < 1$, beyond which gravity can no longer keep the body on its arc. In practice, humans switch from walking to running near $\mathrm{Fr} \approx 0.5$ [23]. Comparing legged systems of different sizes at equal Froude number, rather than at equal speed, therefore isolates the differences that come from their morphology.

2.3.4 Arm Swing in Human Walking

A human arm swing is not merely passive. During walking, the legs generate an angular momentum about the vertical axis that the arms, swinging in anti-phase, largely cancel [24] [25]. This lowers the vertical ground reaction moment the feet must resist and, consequently, the effort required from the lower body. Collins et al. showed that holding the arms still while moving increases the metabolic cost of human walking by about $12\,\%$, and swinging them in phase with the legs by about $26\,\%$ [26]. Whether a comparable benefit arises in humanoid robots depends on the arms' share of body mass and on their reach relative to the legs, which can differ markedly from human proportions, particularly in humanoids whose arms are specialized for manipulation rather than for locomotion.

2.4 Motion Retargeting

Human motion is recorded as the trajectories of the performer's joints or body markers, while a humanoid robot is described by its own joint angles, limb lengths and joint limits. **Motion retargeting** is the process of transferring a recorded motion from one body onto another [27]. For a humanoid robot, this means finding, for every frame of the human motion, a robot configuration q_t that reproduces it as faithfully as possible while remaining physically feasible.

The simplest approach copies the joint angles of the human skeleton directly onto the corresponding robot joints. It is fast, but it ignores the differences in limb lengths and joint axes, so the feet slide, penetrate the floor or float above it. Most retargeting methods used in humanoid robotics therefore formulate it as an optimization over keypoints: a set of human keypoints (hands, feet, elbows, knees, pelvis) is mapped to corresponding robot links, and q_t is found by minimizing the distance between them, subject to the robot's joint limits [28] [29].

Keypoint matching treats each keypoint independently, and so it does not preserve how the body parts are positioned relative to one another or to the ground, which is what makes a motion recognizable. **Interaction-mesh** formulations address this [30]. A volumetric mesh is built over the keypoints, and each vertex i is described by its Laplacian coordinate: $L(p_i) = p_i - \sum_{j \in \mathcal{N}(i)} w_{ij}\, p_j$, the offset from the weighted average of its neighbors $\mathcal{N}(i)$.

Retargeting then seeks the robot configuration whose keypoints reproduce the same Laplacian coordinates as the human, that is, the same relative spatial structure, rather than the same absolute positions:

$$q_t^\star = \arg\min_{q_t} \sum_i \left\| L\big(p_{t,i}^{\mathrm{src}}\big) - L\big(p_{t,i}^{\mathrm{tgt}}(q_t)\big) \right\|^2 + \|q_t - q_{t-1}\|_Q^2, \tag{2.24}$$

where the second term keeps consecutive frames smooth. The neighborhoods $\mathcal{N}(i)$ and weights w_{ij} are typically obtained from a Delaunay tetrahedralization of the keypoints, so that every keypoint is tied to those spatially closest to it, regardless of whether they belong to the same limb [30]. This preserves the distance between body parts that come close, which helps avoid collisions, and since the mesh is rebuilt every frame, these connections change as the body moves. Points sampled on the ground or on objects can be included in the same mesh, so that the distances between the feet and the floor, or between the hands and a held object, are preserved in the same way as those between body parts. Because the Laplacian term alone does not guarantee a feasible pose, the optimization is usually subjected to additional constraints, such as the robot's joint limits, the absence of self-penetration and the preservation of foot contacts. Each frame is solved in sequence, warm-started from the solution of the previous one.

The result of any of these methods is a purely kinematic trajectory: it reproduces the shape and timing of the human motion within the robot's joint space, but it accounts for neither the robot's mass distribution nor the torques needed to execute it. It therefore cannot be replayed directly on the robot, and serves instead as a reference that a controller or a learned policy tracks while keeping its balance.

3

Literature Review

Following the core concepts of reinforcement learning and its policy gradient methods, this chapter reviews how humanoid locomotion control has developed. It first discusses how classical and model-based controllers have addressed the challenges of this problem, and the limitations that steered research towards learning-based methods. It then covers the key innovations in learning architectures and training practices for deploying humanoid policies, followed by the most relevant work for each aspect of this thesis: energy-efficient gaits, natural movement through reference motions, and perceptive stair traversal. For each, it identifies what can be drawn upon and what existing work leaves open, and how this thesis addresses it.

3.1 Model-Based Controllers

3.1.1 Evolution of Classical Control Approaches

Many motion controller approaches for robotic systems involve the use of Proportional Integral Derivative (PID) controllers at the low-level joint layer [31], regardless of the overall higher architectural complexity, because it is regarded as a standard, computationally lightweight and robust solution to provide accurate joint tracking. For simple control problems that are single-input single-output (SISO) or only weakly coupled, where the global dynamics can be approximated as nearly independent subsystems, assigning a single layer of PID controllers to individual joints can be effective. In more strongly

multivariable settings, however, independent SISO PID loops treat inter-joint couplings as unmodeled disturbances rather than coordinated dynamics, and their performance degrades as interactions increase [32]. Over the years, cascaded PID schemes [33] have been proposed that introduce inner and outer loops to improve handling of multivariable behavior, but they still lack an explicit multivariable dynamic model and therefore only partially address coupling effects.

Model-based solutions involve using a mathematical representation of the system as a model of its dynamics to predict its future behavior and optimize control actions accordingly.

A classical example is the Linear Quadratic Regulator (LQR), which establishes a linearized state-space model of the robot to design an optimal full-state feedback law that explicitly accounts for coupling between states and penalizes both tracking error and control effort. However, its gain is computed offline from a model linearized around a single operating point, and it has been noted to show difficulty in adapting to real-time constraints or changing operating conditions [34], which are critical necessities in a bipedal locomotion scheme. Furthermore, for this type of control problem, there needs to be an extensive long-term planning of future states, as bipedal walking inherently involves hybrid dynamics with discrete contact transitions and continuous phases that must be coordinated over multiple footsteps to maintain balance, achieve desired locomotion velocities, and avoid terrain obstacles, which are capabilities that infinite-horizon LQR, which operates as a single fixed feedback law, does not provide.

3.1.2 Model Predictive Control

For this, Model Predictive Control (MPC), which has been widely regarded as a quasi-optimal model-based solution for controller development in robotics, with extensive work in robotic arm manipulation [35] and mobile robotics navigation [36] [37], solves a constrained optimal control problem over a finite horizon at every step, applying only the first input and re-solving from the new state, which turns the optimization into feedback control. MPC is structured to handle multiple inputs and outputs simultaneously, respect physical constraints on actuators and system states, and plan movements several steps ahead, through a receding horizon prediction structure that allows for adjusting its strategy in real time. Since it needs the full current state, it is typically combined with a state estimator. In bipedal locomotion, the cyclical movement has to be meticulously planned over several future steps, establishing a mapping of the predicted footholds of states in the horizon, as the robot's ability to maintain overall stability and recover from disturbances depends on coordinated sequences of steps, not just the immediate next step. Given that these multi-step plans get re-optimized at each instance, means that if meticulously crafted enough, an MPC implementation yields very structured and coordinated movements. Over the years some work has been developed that corroborates this claim, in the scope of quadruped locomotion, and in some simplified bipedal locomotion schemes, to produce methodic, well-balanced movement, when under accurate modeling conditions. This is a main strength when compared to state-of-the-art learning-based applications, as approximations of the model and its dynamics often express difficulties in maintaining coherence within the movement, as it can only be indirectly steered into a specific behavioral direction through its reward function.

Being able to integrate constraints into the optimal control problem means, in the context of humanoid

locomotion, modeling the entire system dynamics and path constraints in a sequence of equality and inequality constraints, the former being used to enforce the robot's whole-body dynamics and contact-consistency relations exactly at each step of the prediction horizon, and the latter capturing limits on joints, torques, center-of-mass transferal within the movement, friction, and feasible foothold regions that must not be violated during motion. The model's accuracy has crucial implications on the overall effectiveness of the employed method. For whole-body bipedal locomotion, the complexity of factors that have to be accounted for to ensure stability across external disturbances, rapidly shifting velocities, and alternating terrain, to name a few, is too large to be expressed in a form an optimizer can solve at every control step, even though simulators can reproduce it closely. Additionally, any slight modeling error for any of these particularities can rapidly accumulate and propagate into a substantial impact on the overall performance of the control scheme.

Furthermore, all required computation is performed online, which in some implementations scales cubically with state dimension [38], and thus further pushes for issues in the complex modeling required for a whole-body locomotion context, as it can result in a prohibitive computational load at each time step for such a high-dimensional state space.

These limitations steer the developed work in this context to make significant compromises on the model's complexity to obtain feasibility. Several works on MPC implementations restrict themselves to bipedal locomotion [39], [40], [41], with no additional modeling of the upper-body. Some employ the overall humanoid frame in their work, but heavily reduce upper-body dynamics [42], or significantly simplify the locomotion movement to either operate at a fixed velocity and gait [43], or just perform general stability without any locomotion [44]. Compromises are also inherent in works that are developed in simulation only, where they make use of a reduced model and simplified physics for the computations to be possible [45] [46].

For the goals of this thesis, MPC based implementations are not the most suitable approach, as the deployment-heavy structure of this work would entail significant difficulties and limitations, especially for the whole-body locomotion objectives with improved natural motion, or for instability-prone tasks such as stair traversal, which would simply require too much modeling complexity.

3.2 Learning-Based Controllers

3.2.1 Key Innovations in the Core Learning Structure

The limitations of model-based approaches have motivated significant research into learning-based control methods, particularly DRL, which take a fundamentally different approach: rather than embedding a model of the robot's dynamics in the controller, learning-based controllers are trained on data generated by a simulator to discover control policies that maximize long-term cumulative reward, similar to optimizing over horizons in MPCs. Policy optimization is performed offline rather than at each time step, so the simulator's model can be as detailed as needed. This provides groundbreaking advantages in a humanoid robotics context, as their high state complexity requires a wide spectrum of observational

information that can fully describe the state, as well as a lot of data for feasibility in learning. Pinto et al. [47] proposed an actor-critic learning structure where the critic has access to privileged information during training under a robotics simulator that gets implicitly passed down to the actor. This allows for a significant expansion of the observation state, which is crucial for a humanoid robotic framework that is usually not equipped with a lot of sensors, either due to weight or space limitations, but can effectively be represented as such in simulation. Furthermore, the advantage of training being fully performed within simulation allows for multiple parallel environments to run at the same time, of which state-of-the-art robotics simulators also employ GPU acceleration to speed up the robots' interactions within the environment, drastically increasing data collection during learning.

3.2.2 Applications of Reinforcement Learning Algorithms in Robotics

The described state-of-the-art off-policy algorithms in Chapter 2 use experience replay buffers that store the collected transitions of agent interactions for each parallel environment in a given training iteration. This is the data that is then used to perform learning updates on the established networks, and it is organized in batches to stabilize each update, at a tunable size. To improve on the robustness of the learned policies, along with reducing the inherent gap between the robotics simulator used in training and the real ambient conditions during deployment, domain randomization has become common in recent work, as it takes advantage of training under a simulated setting to alternate physics properties during different training environments, such as friction, gravity, noise in sensor readings or joint weights, and their associated gains. Being able to run multiple training environments simultaneously furthers the collected experience under these randomized conditions, which are different for each running environment, leading to faster to achieve robustness of the trained model.

Developments in curriculum learning implementations, which have been used to periodically increase velocity commands [48], ground roughness [49], or the intensity of perturbation forces [50] throughout later stages of training, help establish a progressive learning evolution, avoiding chaotic initial learning stages and instead quickly establishing early improvements.

The computational cost inherent to the establishment of trust regions for complex robotics problems has resulted in the absence of TRPO implementations in recent humanoid locomotion work [51]. Similarly, the overestimation bias in Q-function learning and high variance denoted in DDPG infer unpredictable training and unstable policy updates on high-dimensional problems [52].

Both have, in this context, been largely superseded by their more recent variants, PPO and TD3, respectively, which have exhibited considerable prominence in recent humanoid robotics surveys [53]. PPO has seen extensive application in recent humanoid locomotion implementations [49] [54], mainly because its clipped surrogate objective and reliance on data generated from the current policy yield relatively stable policy updates in high-dimensional continuous control settings. There are, however, key instances where the sample efficiency that is manifested in off-policy algorithms like TD3 is considered a more essential property to be maintained in contrast to the conservative stability of on-policy methods.

The work structure of this thesis, which expands towards coordinated whole-body locomotion and height traversal, raises the need for a learning algorithm that can extract as much information as possible

from limited interaction data due to a more complex learning environment. This makes the choice of an off-policy algorithm like TD3 the most reasonable one as the tasks grow in complexity. The asymmetric weight load extension of the employed robot architecture detailed in Chapter 1, further justifies this decision, as it will cause trajectories to terminate earlier and more frequently than on a morphologically proportionate robot due to higher instability, resulting in fewer collected transition samples and thus increasing the importance of maintaining high sample efficiency. Additionally, the stair traversal objective further increases the sample efficiency requirements, due to the greater diversity of its training terrain.

Hurdles towards the use of off-policy algorithms in a deployable humanoid locomotion setting requires training to be as time efficient as possible, as it can require a multitude of pre-training scenarios to fully adjust to optimal behavior. Off-policy RL research has made significant progress in improving this deficiency, as recent work by Seo et al. [55] explored integrating the TD3 algorithm with parallel simulation environments, significantly speeding up long wall-clock training times, as well as performing a baseline study for legged locomotion on the simplified Booster model previously depicted in Figure 1.1a, which provides a solid ground for the early work developed in this thesis.

3.2.3 Energy Efficiency and Emergent Gaits

Energy terms appear in most learned locomotion rewards, usually as torque or power penalties that regularize the motion rather than shape it. Fu et al. showed that minimizing energy while tracking a commanded speed is enough for natural gaits to emerge in legged robots, with the gait changing with speed as in animals [20]. These studies treat the legs only, and the arms of humanoid policies are typically fixed or driven by an explicit reward or reference. Whether the arm swing that lowers the cost of human walking (Section 2.3.4) also emerges in a humanoid from the energy objective alone, and how it depends on arm mass and length, has not been examined. This work therefore explores policies trained with the cost of transport as their main objective, with the arms rewarded only through their effect on the legs, and compares robots of different proportions at equal Froude number so that differences in arm swing can be attributed to morphology.

3.2.4 Research Work Towards Natural Movements

The previously described algorithms are designed to employ conventional reward functions, that decompose locomotion into task-specific objectives, such as center-of-mass tracking, footstep timing, and contact force constraints, weighted as a composite cost function. However, this approach reaches a plateau in movement fluidity, as it is systematically constrained to the engineered creativity in the modeled reward function, and thus fails to capture intrinsic and subtle nuanced behaviors that characterize human-like motion. Recent work has looked to leverage motion-captured demonstrations which can be retargeted to the robot's morphology and directly encode natural movement patterns through imitation. This has included Behavior Cloning [56], where policies learn direct state-to-action mappings through supervised learning from expert demonstrations, and Adversarial Motion Priors [57], which employ discriminators trained adversarially to distinguish between agent-generated and expert behaviors, learning

a motion prior that captures the distribution of natural movements without requiring explicit action labels or precise trajectory tracking.

However, these imitation learning methods, while producing visually fluid gaits, suffer from poor robustness to disturbances and generalization failures when tested on real hardware or in unseen scenarios. A promising direction emerging in recent humanoid robotics is to combine both handcrafted rewards with reference motions, thus being able to keep the secured stability of a learning structure that utilizes a carefully handcrafted reward function, along with a retargeted reference motion to establish initial guidance, and provide fine-tuning for a more natural and energy efficient movement. In its most direct form, the policy is rewarded for tracking the joint trajectories of the reference [8], while the manually designed terms continue to govern velocity tracking and balance.

The reference motions themselves come from human motion-capture datasets, such as LAFAN1 [58] and AMASS [59], transferred onto each robot through retargeting algorithms (Section 2.4), from keypoint matching [28] [60] to interaction-mesh formulations such as OmniRetarget [61], which solves each frame as a sequential quadratic program warm-started from the previous one, with hard constraints on joint limits, collisions and foot contact. These methods have mostly been tuned for humanoids of near-human proportions. These datasets also exist originally for an entirely different purpose: LAFAN1, for instance, was recorded for game development, in long takes where walking is interleaved with turns, stops and unrelated movements. This suits the imitation of an entire choreography, but not the use of a reference to enhance a reinforcement learning locomotion policy, for which only the needed portion must be extracted, in the case of walking a single gait cycle of two steps, left and right. Such a cycle also fixes one style of walking at one speed, whereas human gait changes character with speed.

This thesis builds on the combination of handcrafted rewards and reference tracking, and departs from existing work in three respects. First, the retargeting is adapted to robots whose proportions differ from the human's, as the T1 and several of the other robots studied here do, since a formulation tuned for near-human proportions produces references that such robots cannot physically follow. Second, rather than imitating whole takes, individual gait cycles are extracted from both an established dataset and dedicated recordings at multiple speeds, and organized into a library indexed by walking speed. The policy can then be guided by a reference matching its commanded velocity, so the gait changes character with speed as human walking does, instead of fixing a single style. Third, the reference covers the whole body, including the arms, whose swing has more significant effects for robots with long and heavy upper limbs, while the handcrafted terms keep the robustness required for real deployment.

3.2.5 Terrain Perception for Stair Traversal

Learning-based stair traversal divides into two approaches according to what the policy observes. Blind, or proprioceptive, policies observe only joint states and the inertial measurement unit, and infer the terrain from how contact disturbs these signals. Combined with terrain curriculum and domain randomization, this has proven effective for humanoids on rough and sloped ground [62] and on stairs [63], as it keeps the observation space small and requires no perception hardware. However, a blind policy only detects a step once a foot strikes it, so it is conditioned to recover after the toe hits the step

rather than to plan the placement, as it cannot place the foot relative to an edge it has not yet touched. This recovery reflex becomes more dangerous as the step grows taller relative to the robot, both for higher steps and for shorter robots. Providing the policy with a representation of the terrain ahead has accordingly been shown to substantially increase the success rate of humanoid stair traversal over proprioception alone [64].

Perceptive policies address this by observing the terrain ahead, which allows each step to be anticipated and the motion adapted to it. The terrain is represented either directly as depth images [65] [66], or as a heightmap sampled around the robot [67] [68], which on hardware is built from onboard depth cameras or LiDAR [69]. Depth images carry more information, but rendering them across thousands of parallel environments greatly increases the cost of training, and they are typically learned through a second distillation stage from a policy trained on privileged terrain information. A heightmap is a cheap query of the terrain geometry in simulation, at the cost of relying on a separate mapping module at deployment.

Accurate foot placement is more critical for humanoids than for quadrupeds, as they are supported by two feet of finite length rather than four near-point contacts, and a foot landing partly over an edge loses its support [70]. Perceptive humanoid controllers have mostly been demonstrated on full-size platforms, for which a standard stair step height is a moderate fraction of leg length, and they optimize primarily for traversal success and agility. A humanoid of shorter stature such as the Booster T1 is more demanding: a regulation step reaches about a third of its leg length, its blocky feet leave little margin on each tread, and a stumble on a step quickly becomes dangerous, so each footstep must be planned rather than recovered from. This work addresses these requirements in three ways that existing work does not combine. First, whereas perceptive controllers leave the shape of the climbing motion to emerge from task rewards, or learn its style implicitly through discriminators [71], stair traversal is decomposed into the sub-movements a person performs, from positioning the foot before the step to transferring weight onto the tread, and each is rewarded explicitly. Organizing the motion in this way gives the most controlled and precise placement at every stage, which robots with tight placement margins require. Second, rather than training one network for both behaviors, or a mixture of experts within one policy [71], a dedicated climbing policy is blended at deployment with the expressive walking policy. Climbing requires joint ranges and timings that conflict with the walking gait, so a single network compromises both, whereas separate experts can each specialize fully, and the walking gait keeps its full quality on flat ground. Third, where perceptive policies are predominantly trained with PPO [67] [72] [70], the climbing policy is trained off-policy on a heightmap whose imperfections approximate those of the real camera. Stair environments, the precise motions they require and the added complexity of some robots' bodies all raise the cost of training. The sample efficiency of off-policy learning discussed earlier and the low cost of a heightmap keep it affordable, which allows each sub-movement to be trained until it is accurate, making the climb a more reliable alternative to a single long and unstructured motion.

4

Locomotion Learning Framework

4.1 Research Strategy

The work in this thesis, represented in Figure 4.1, is based on a **progression** along three locomotion objectives, each oriented towards more natural and robust movement, followed by a final objective that expands the motion controller to stair traversal.

The **standard locomotion** objectives are tackled progressively with each building on the previous one and increasing in complexity. Thus, the first objective relates to locomotion only involving **leg joint movement**, whilst maintaining the upper body at a fixed position. This simplifies the development process for an initial deployment, and not only allows for starting under the same groundwork as of the original Booster motion model, allowing for a direct comparison towards improving it, and establishing a good baseline for the initial core principles and development practices.

As previously described in the Introduction, the position in which the arms are held during locomotion has a significant impact on the robot's stability. More specifically, one of the additional reasons to select this as the first objective was to allow for the opportunity of experimenting with **different arm configurations**, namely to rest the arms along with the torso, to ensure greater stability and a more natural movement, as opposed to the retraction of the arms that the official Booster motion model requires, which adds considerable weight to the front of the robot and makes it more unstable in motion.

Following up on this, extending the locomotion to be performed as **whole-body** is the clearest pro-

gression from the previous objective, as it involves having the upper-body shift weight along with the movement, allowing for more balance and an overall much more natural motion. It does however, introduce greater complexity to the learning process, and presents more significant challenges in terms of collision avoidance and safety when deploying on the real robot.

From there, the fluidity of the movement is further improved by introducing **reference motions**, to capture nuanced characteristics in locomotion that are not induced by the hand-modeled reward function, and to condition them on the commanded speed. This comprises the third objective, which fine-tunes the previously developed whole-body motion controller.

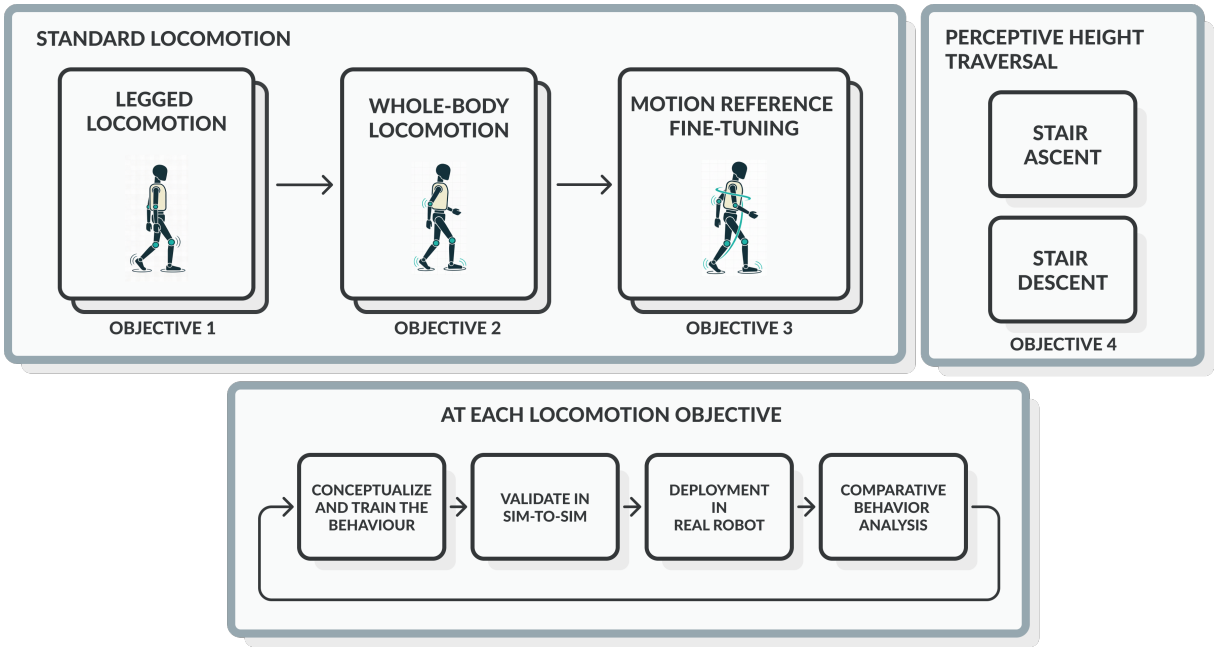


Figure 4.1: Structure of the Thesis Work.

Each objective is addressed following a consistent workflow, to allow for a more direct comparison between goals, and will be composed of:

- A conceptualization of the objective, including the formulation or modification of the learning networks and reward function, followed by training analysis.
- A sim-to-sim validation of the trained policy on multiple robotics simulators, to ensure its generalization and robustness through manually teleoperated stress tests.
- Deployment on the real robot, looking to replicate the same movements performed in simulation and confirm its transferability to ambient conditions in a real environment.
- Perform a comparative study at the joint level between the original Booster motion model baseline and each developed objective, along with a sim-to-real analysis for each deployed policy.

As a natural progression towards building a better motion controller, the fourth objective incorporates **stair traversal**, through a perceptive climbing policy whose rewards decompose ascent and descent into the sub-movements a person performs. This policy is blended with the reference-enhanced walking policy at deployment, so that the resulting locomotion encompasses both flat ground and stairs.

4.1.1 Simulator Pipeline

For the training and validation steps of each proposed objective, the two current state of the art robotics simulators for humanoid research were utilized: Google DeepMind's MuJoCo and NVIDIA Isaac. As stated previously, these environments allow for an unprecedented level of feasibility under the complexity of our reinforcement learning problem, due to their ability to perform large scale parallelized learning rollouts. The physics engine in each platform is built to approximate itself as much as possible to real world physical interactions, but neither can ever represent actual real world physics to an exact level. Thus, the inherent differences between the NVIDIA PhysX engine of Isaac and the accurate, lightweight MuJoCo framework are leveraged not only to utilize each simulator to its relative strengths, but also to serve as an adequate robustness ecosystem in which the policy becomes better suited to real deployment.

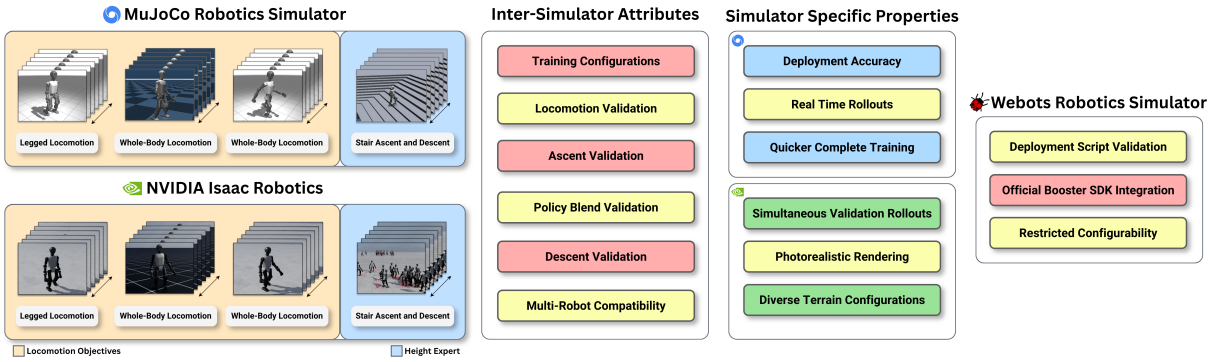


Figure 4.2: Diagram of the Developed Simulator Pipeline.

However, irrespective of the chosen simulator, it is important to enforce operational compatibility between the developed environments, so that the observations, action computing, robot dynamics, and every single shared factor remains consistent throughout both simulators. This poses a certain challenge, as the two use different physics engines, robot model formats and joint control implementations, but it allows for accurate validation across separate physics engines and for a shared ecosystem where every developed training configuration can be adapted to either simulator, so this attribute was maintained as a priority.

Thus, training configurations for each stated objective and their corresponding validation environments were created for MuJoCo and then ported over to Isaac. This work on compatibility required replicating the observation processing and native control action computation of MuJoCo, which were found, through initial experiments, to accurately reproduce the behavior of the physical robot, rather than utilizing Isaac's native frameworks which misrepresented the behavior of a MuJoCo trained policy.

MuJoCo stood out as the better alternative for the initial training development and formulation, due to its lightweight framework allowing for quick iterations of attempted changes. This same property was notably valuable during validation, as it allowed to run a rollout of a trained policy in real time, with the same conditions as real robot deployment would manifest.

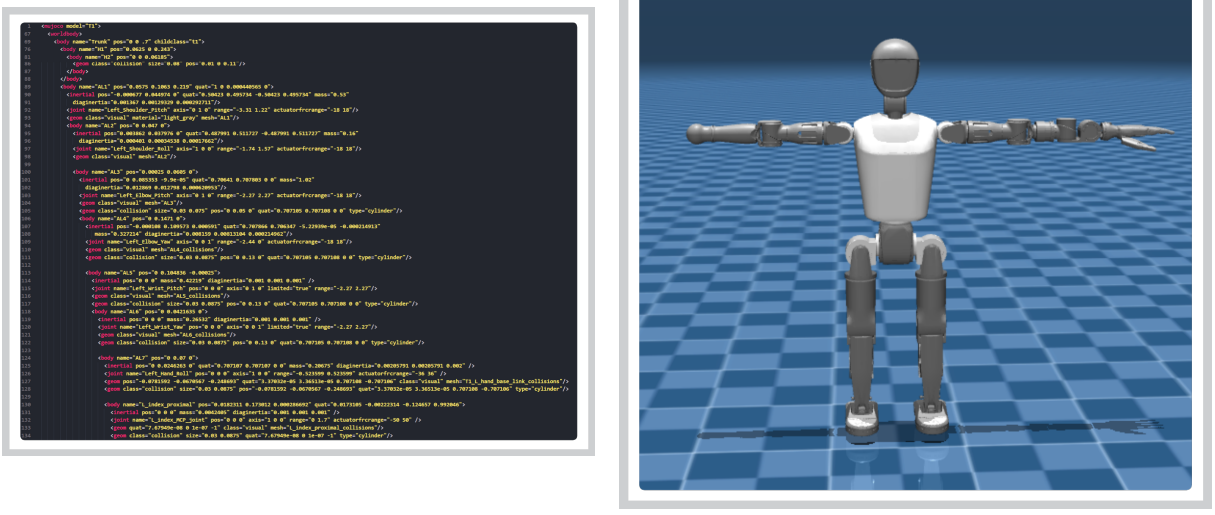
Conversely, the photorealistic rendering of NVIDIA Isaac makes training significantly more resource heavy and time consuming, which is particularly apparent on the developed joystick controlled validation,

which operates at a speed approximately 10 times slower than real time, with a considerable latency in its movements. However, it remains very useful to utilize Isaac in the development pipeline, not only as an additional validation to ensure a robust and generalized policy, but also because its streamlined terrain generation capabilities are unmatched, establishing the necessary conditions for the height expert training and allowing hundreds of robots to roll out at the same time, in different terrain tiles, in validation.

Lastly, a third simulator, Webots, was used to validate the deployment script itself, that is, the code that interfaces with the Booster SDK and runs unchanged on the real robot. The Webots environment is provided by Booster in a pre-compiled format and implements the SDK in simulation. This restricts any modification of the SDK itself, but it allows the deployment script to be checked for bugs or unintended behaviors before it is run on the real robot, where such errors would be dangerous. The scene itself could nonetheless be extended with staircases and with the conversion of the camera point cloud into the heightmap, so that the deployment script of the height traversal policy could be validated in the same way.

4.1.2 Robot Model Implementation

As mentioned in Section 1.2, the majority of the officially provided resources, including the existing XML and URDF files, are made for the simplified model in Figure 1.1a. Representing the real robot therefore required replacing the arm below the elbow with the extended manipulation arm, whose meshes had to be extracted from the official Universal Scene Description (USD) file[1], and rewriting the hierarchical structure of the XML so that the additional DoF are jointed correctly.

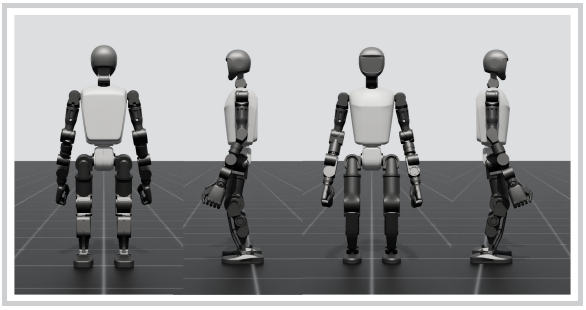


(a) Excerpt of the Modified XML Model. (b) Comparison of Changes Made in Simulation.

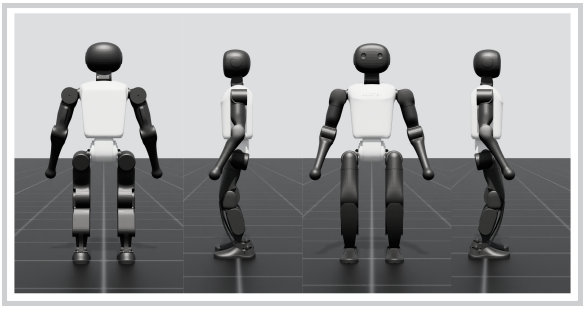
Figure 4.3: Restructuring and Modifying of the XML Model for the Booster T1.

These changes were necessary because the added joints and the mass they carry considerably alter the dynamics of the robot, and with them the learning process. Training on the simplified model would produce policies for a robot that differs from the one on which they are deployed.

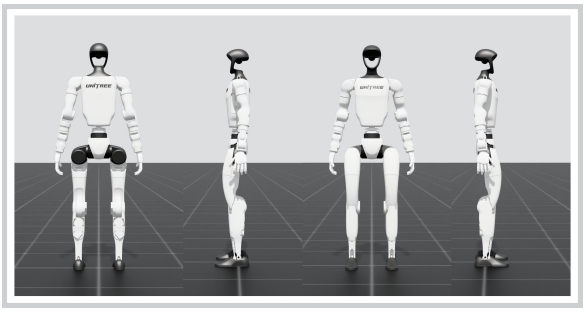
[1]Booster T1 Documentation: `https://booster.feishu.cn/wiki/DtFgwVXYxiBT8BksUPjcOwG4n4f`



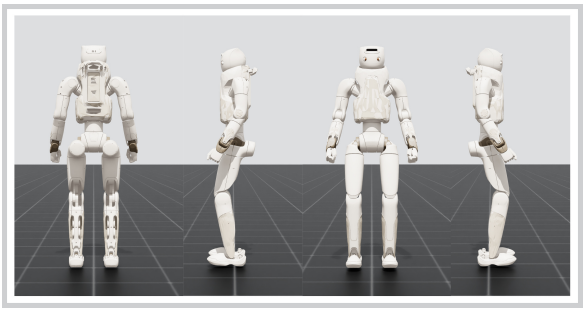
(a) Booster T1 (Announced in January 2025).



(b) Booster K1 (Announced in October 2025).



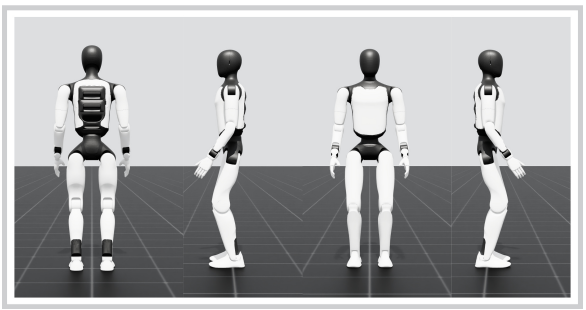
(c) Unitree G1 (Announced in May 2024).



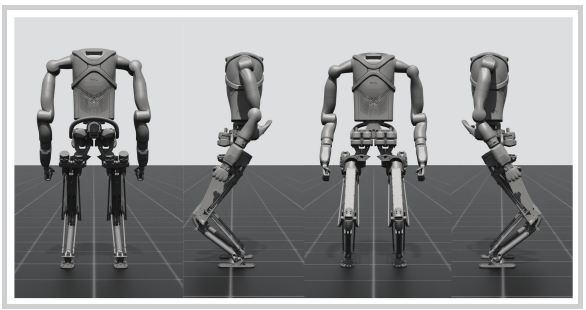
(d) Fourier GR3 (Announced in August 2025).

Figure 4.4: Robot Models Used in This Work (Part 1).

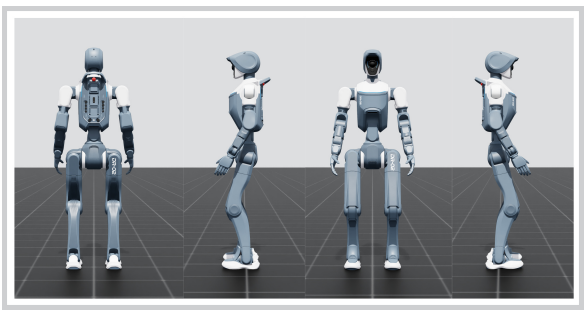
To ensure that the developed approaches generalize across platforms of varying stature and morphology, every training and validation environment also has a counterpart for six additional robots: the Unitree G1, the Booster K1, the Fourier GR3, the Agibot A3, the PAL Robotics Kangaroo and the Deep Robotics DR02. This decision added depth to the studied methods and supplemented the conclusions on ascent and descent behavior, as well as on energy efficiency in whole-body walking.



(a) Agibot A3 (Announced in April 2026).



(b) PAL Kangaroo (Announced in August 2024).



(c) Deep Robotics DR02 (Announced in October 2025).

Figure 4.5: Robot Models Used in This Work (Part 2).

Together, these constitute a group of the current state-of-the-art humanoid robots, released between 2024 and 2026. Although some already had a URDF for them available, each one had to be converted to an XML model and extensively modified to reach the same completeness as the T1 model. Among other changes, their joints were mapped to a common layout and their actuation was normalized relative to mass and leg length, so that differences in the resulting gaits come from each robot's morphology rather than from how its model was set up.

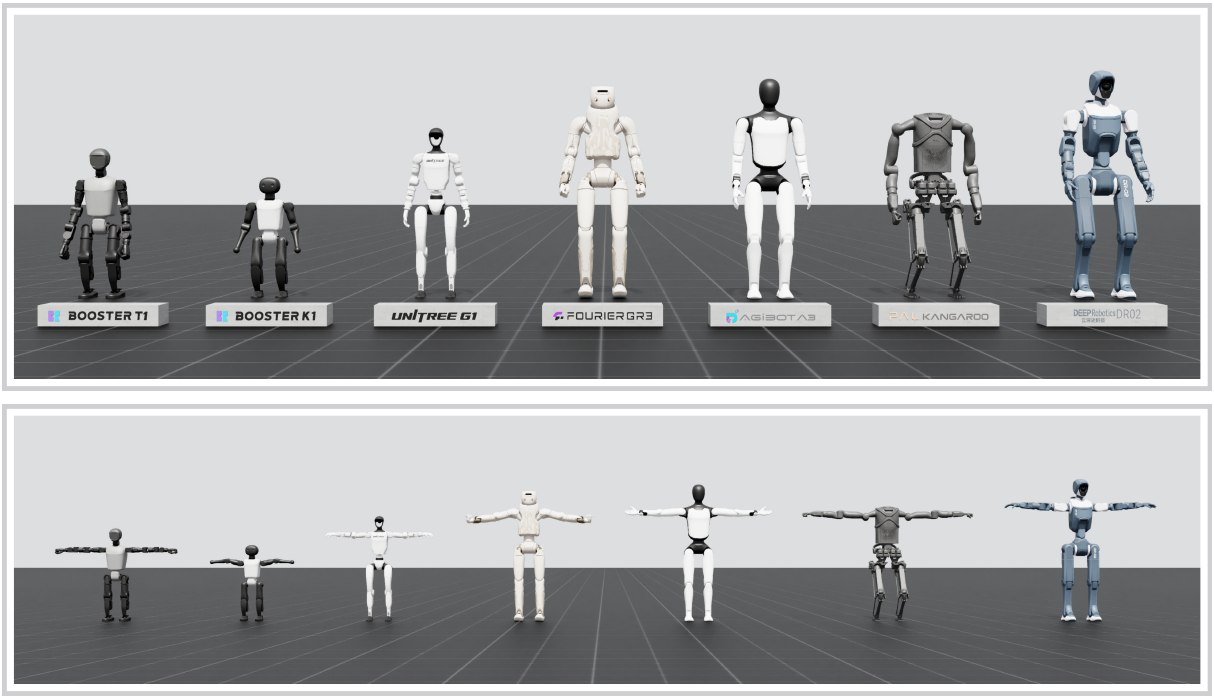


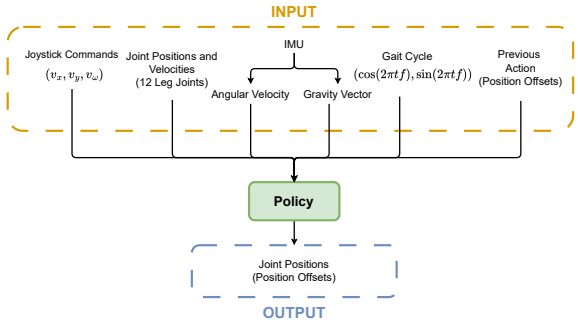
Figure 4.6: Side-By-Side Comparison for the Studied Robots, Both when Standing at Rest and Showing their Total Wingspan.

Lastly, they also introduce a necessary diversity for the robot morphology studies. The upper-body abnormalities of the T1 are now more visually striking when placed alongside more proportional robots. The GR3 exhibits the other side of the extreme: its arms are proportionally shorter and bulkier than the rest. Moreover, the shorter stature of the K1, the human-like proportions of the G1, the greater height and weight of the GR3, A3, and DR02, along with the parallel leg linkage of the Kangaroo, which in itself required added complexity both when porting the model to each simulator and in the developed training and validation configurations, bring interesting points of study for the developed behaviors. Thus, with these models in place, the requirements for training are met.

4.2 Legged Locomotion Pipeline

Leg joint movement locomotion requires a policy that solely controls the leg motors of the robot, whilst keeping every other joint at a fixed position. For this, the policy is simplified to only require observations of the leg joints, and in turn, only outputting joint positions of those same leg motors.

As described in Figure 4.7a, the policy receives as input a velocity vector (v_x, v_y, ω_z) as a command from a teleoperated joystick, along with the previous output action, the actual angular positions and angular velocities of the leg joints measured by their joint (allowing the network to compare intended and realized motion), readings from the IMU at the robots torso indicating the gravity vector along the center of mass, and its angular velocity from the gyroscope of the IMU. Lastly, the policy also receives a gait cycle command, encoded as $(\cos(2\pi ft), \sin(2\pi ft))$, where t is a set internal timer and f the fixed gait frequency assigned during training, indicating the current phase of the periodic gait, which is used for some of the behavior-specific rewards. Within this cycle, the left and right feet are assigned swing windows centered at normalized phases 0.25 and 0.75, respectively, and the reward gives a positive contribution only when the corresponding foot is not in contact with the ground (as determined by a collision check with the floor), thereby encouraging appropriately timed foot lift-off for each leg.



(a) Developed Policy Structure.

Components	Dims	Actor	Critic
Commands	3	✓	✓
Gait Cycle	2	✓	✓
Gravity Vector	3	✓	✓
Angular Velocity	3	✓	✓
Joint Position	12	✓	✓
Joint Velocity	12	✓	✓
Previous Action	12	✓	✓
Body Accelerometer	3		✓
Base Linear Velocity	3		✓
Base Height	1		✓
Push Force	2		✓
Feet Linear Velocity	12		✓

(b) Asymmetric Actor-Critic Structure.

Figure 4.7: Base Underlining of Both the Policy and Network Architecture.

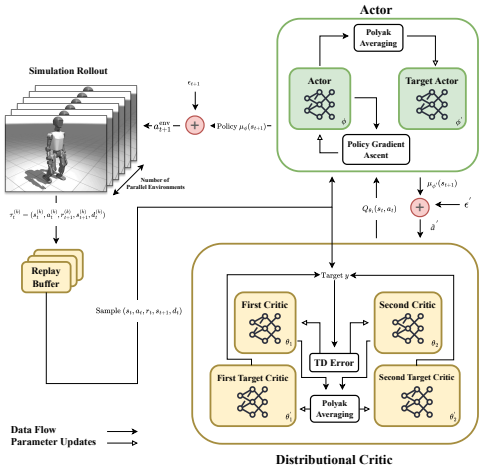
During training, the actor network receives the readings from the IMU and joint encoders with added noise, whereas the critic network, serving as the evaluator, receives all of the actor's observations free of any noise, and is equipped with additional privileged information denoted in Figure 4.7b that is only accessible in simulation and aids in policy optimization. These mostly include some useful state information for some of the employed rewards.

The utilized reward function, detailed in Figure 4.8b, follows the general principles for locomotion on humanoid robots, which include positive reinforcement for maintaining operation and avoiding the termination conditions, that occur when the robot falls or reaches a time limit, and for accurately tracking the specified velocity commands. It is then followed by penalizations when the base height and orientation of the torso does not meet the specified target value, which will encourage the robot's upper body to

remain straight and at a stable position during motion. Excessive movements that would warrant a high torque, joint velocity, or acceleration are slightly penalized, along with when the system nears joint limits or faces collisions.

Lastly, some behavior-specific rewards are designed, with the aid of the gait cycle command, to promote foot swings according to the gait frequency, penalizations for rolling or slipping of the ankles, as well as maintaining each leg at a set distance and with an adequate orientation when idle.

The learning structure itself follows the off-policy TD3 algorithm adapted to speed up training with parallelized environments [13] [55] and adjusted to support the modified robot model.



(a) Utilized Networks for the First Objective.

Components	Equations	Weights
Survival	1	0.25
Velocity Tracking x	$\exp\big(-(v_x^{\text{cmd}} - v_x)^2/\sigma\big)$	1.0
Velocity Tracking y	$\exp\big(-(v_y^{\text{cmd}} - v_y)^2/\sigma\big)$	1.0
Velocity Tracking ω	$\exp\big(-(\omega_z^{\text{cmd}} - \omega_z)^2/\sigma\big)$	2.0
Base Height	$(h - h^{\text{target}})^2$	-20.0
Orientation	$\|g_{xy}\|^2$	-5.0
Torque	$\|\tau\|^2$	-1×10^{-4}
Torque Tiredness	$\|\tau/\tau_{\max}\|^2$	-0.5×10^{-2}
Power	$\max(\tau \cdot \dot{q}, 0)$	-1×10^{-4}
Linear Velocity z	v_z^2	-2.0
Angular Velocity xy	$\|\omega_{xy}\|^2$	-0.2
Joint Velocity	$\|\dot{q}\|^2$	-1×10^{-4}
Joint Acceleration	$\|\ddot{q}\|^2$	-1×10^{-7}
Base Acceleration	$\|\dot{v}\|^2 + \|\dot{\omega}\|^2$	-1×10^{-4}
Action Rate	$\|a_t - a_{t-1}\|^2$	-0.5
Joint Position Limit	$1_{q>q_{\max}} + 1_{q<q_{\min}}$	-1.0
Collision	$n_{\text{collision}}$	-10.0
Feet Swing	$1_{\text{feet swing}} \cdot 1_{\text{swing period}}$	3.0
Feet Slip	$1_{\text{feet stance}} \cdot \|v_{\text{feet}}\|^2$	-0.1
Feet Yaw	$\|\psi_{\text{feet}} - \psi_{\text{base}}\|^2$	-0.1
Feet Roll	$\|\phi_{\text{feet}}\|^2$	-1.0
Feet Distance	$\max(d_{\text{ref}} - d_{\text{feet}}, 0)$	-10.0

(b) Utilized Reward Function for the First Objective.

Figure 4.8: Base Underlining of Both the Learning Networks and Reward Function.

At each global step, in-simulation transitions across the parallel environments are collected and collectively stored into the replay buffer, which is sized *buffer size* $\times$ *num environments* , so that it becomes comprised of an even distribution of samples collected from every environment. The transitions are represented as $\tau_t^{(k)} = (s_t^{(k)}, a_t^{(k)}, r_{t+1}^{(k)}, s_{t+1}^{(k)}, d_t^{(k)})$ for state, action, reward, next state and a termination flag, respectively, for each parallel environment k. During the initial global steps, the algorithm will operate solely on data collection, gathering samples from environment steps without network parameter updates. Due to an employed high count of parallel environments, just after a couple initial steps the buffer is large enough for learning to start, and from then on, at every environment step, batches sampled by the replay buffer are used to update both critic networks Q_{θ_1} and Q_{θ_2} multiple times per global step, with their corresponding target networks, $Q_{\theta_1'}$ and $Q_{\theta_2'}$, and the delayed actor μ_ϕ being updated half as frequently. Using batches instead of single samples greatly reduces the noise associated with such a high dimensional state and action space. By optimizing the distributional critic networks multiple times between each environment step, it greatly increases sample efficiency for learning in such a complex space in order for the tasked goal to be feasible in a reasonable time-frame. At the end of each global step, the updated actor policy samples the next set of exploratory actions a_{t+1}^{env} by applying per-environment Gaussian noise to the deterministic actor outputs, which are used to obtain the next

sets of transitions at the beginning of the next global step.

4.3 Whole-Body Locomotion Extension

This section extends the action space to the upper body and investigates how arm motion can be shaped during locomotion. Two training objectives are considered. Under the **unrestrained objective** (Section 4.3.1), the arms are subject only to physical feasibility and a small set of coordination priors. Under the **energy efficiency objective** (Section 4.3.2), arm motion is not rewarded directly; it is expected to emerge only if it reduces the cost of transport. Both objectives share the observation space, network architecture, domain randomization and optimization procedure of the base environment in the previous section. They differ only in the action space and in the reward terms listed in Table 4.1.

4.3.1 Unrestrained Upper Body Objective

The shoulder and elbow joints were added to the action space, and the baseline reward was retained without modification. This produced policies that moved the arms during the gait, but in an erratic and physically implausible manner. Three deficiencies were identified, each of which motivated a corresponding change to the reward structure: (i) self-collision between the upper and lower limbs, (ii) exploitation of arm posture to compensate for the robot's mass distribution, and (iii) a lack of temporal coordination between arm and leg motion.

Self-collision. The simulated model provides no collision geometry for the upper limbs. As a result, the arms passed freely through the legs during the swing phase. Collision meshes were therefore attached to the hands and arms, and any contact between them and the legs is penalized (Table 4.1). This term carries one of the largest weights in the reward function. The intent is that the optimization treats it as a near-hard constraint rather than as a quantity to be traded off against task performance. The resulting policies avoid limb-to-limb contact even under large external perturbations and on drops of considerable height. Under these conditions they may execute rapid recovery motions that restore balance, but without resorting to self-collision.

Table 4.1: Reward Terms Modified or Introduced for the Unrestrained Objective. 1_c is an indicator equal to 1 when condition c holds and 0 otherwise. ϕ and θ denote the shoulder roll and pitch angles, respectively; subscripts L and R refer to the left and right arms.

Component	**Equation**	**Weight**
Feet Swing	$1_{\text{foot in swing window}} \cdot 1_{\text{no contact}} \cdot 1_{\text{limbs follow}}$	3.0
Collision (Arm–Leg)	$1_{\text{arm collides with leg}}$	-10.0
Arm Roll Window	$1_{\phi_{\text{L}} \notin [\phi_{\text{L,min}}, \phi_{\text{L,max}}]} + 1_{\phi_{\text{R}} \notin [\phi_{\text{R,min}}, \phi_{\text{R,max}}]}$	-0.1
Arm Pitch Window	$1_{\theta_{\text{L}} \notin [\theta_{\text{L,min}}, \theta_{\text{L,max}}]} + 1_{\theta_{\text{R}} \notin [\theta_{\text{R,min}}, \theta_{\text{R,max}}]}$	-0.1

Posture exploitation. Relative to the rest of the robot's morphology, the arms account for a disproportionate share of its mass and volume. Without constraints on their configuration, the policy converged to a local optimum in which the arms were raised and held in induced balance postures. This redistributes

the center of mass and prevents arm motion from destabilizing the gait. Although the adaptation is effective, it requires sustained holding torques against gravity and external disturbances. It also departs markedly from human gait, which is the behavior this work seeks to approximate. To discourage it, admissible intervals were defined for the shoulder roll and pitch angles. Each interval encloses the range of a natural swing, and any excursion outside it is penalized (Arm Roll Window and Arm Pitch Window, Table 4.1).

Inter-limb coordination. With these constraints in place, the arms swing within plausible bounds, but do so without consistent timing relative to the legs. Human walking exhibits contralateral coordination: each arm swings forward together with the opposite leg. This relationship does not emerge naturally, so arm timing was made a precondition for an existing reward: the feet swing reward is granted only when the upper limbs follow the stepping pattern (the $1_{\text{limbs follow}}$ factor in Table 4.1), so a step taken with uncoordinated arms earns nothing. During the swing of leg a, this requires the shoulder of the opposite arm $\bar{a}$ to pitch in the same direction as the swinging hip:

$$1^{(a)}_{\text{limbs follow}} = 1_{\dot{q}^{\,\text{sh}}_{\bar{a}}\,\dot{q}^{\,\text{hip}}_{a}>0}, \qquad a \in \{\text{L}, \text{R}\}, \tag{4.1}$$

where $\dot{q}^{\,\text{sh}}$ and $\dot{q}^{\,\text{hip}}$ are the shoulder and hip pitch velocities. Since the condition compares directions rather than prescribing one, it holds for both forward and backward walking.

Limitations of reward shaping. The coordination condition prescribes the direction in which each arm should move relative to the legs, but not the shape of the swing trajectory. Specifying the trajectory through additional handcrafted terms would require decomposing the desired motion into a sequence of sub-objectives. This would further densify an already extensive reward function and increase the burden of weight tuning. Approaches based on reference motions, either through explicit tracking [8] or adversarial motion priors [57], avoid this decomposition: they specify the desired behavior directly from data instead of abstracting it into individual reward terms. No such prior is used under the unrestrained objective. In the absence of an explicit incentive for human-like arm swing, and given the inertial properties of the robot's upper body, the policy is therefore expected to converge to comparatively low-amplitude arm motion.

4.3.2 Energy Efficiency Objective

The unrestrained objective imposes arm behavior through explicit shaping. The energy efficiency objective instead examines whether arm swing emerges as a consequence of minimizing energy expenditure, as it does in human walking (Section 2.3.4). A second training environment was therefore constructed in which the cost of transport is the primary optimization target, and the arms are rewarded only implicitly, through their effect on the mechanical work performed by the leg joints. A power penalty was considered first, but, as anticipated in Section 2.3.2, when weighted at roughly a third of the objective the T1 reached only 0.10–0.08 m/s at commands of 0.30–0.50 m/s and stood still at 0.85 m/s.

The per-step cost therefore follows Eq. (2.22) with positive mechanical power,

$$\mathrm{CoT} = \min\left(\frac{\sum_j \max(\tau_j \dot{q}_j, 0)}{m\, g\, \max(|v_x|, v_{\min})},\ c_{\max} \right), \tag{4.2}$$

with $v_{\min} = 0.15\,\mathrm{m\,s^{-1}}$ to avoid the singularity near standstill and $c_{\max} = 3$ (about five times the CoT of a normal gait) to avoid a stumble behavior from dominating the reward. The term is active only when a non-zero velocity command is given. Losses that grow with the square of the torque are partially accounted for by the separate squared-torque penalty.

Actuated upper-body joints. During locomotion, arm motion is generated predominantly at the shoulder. Shoulder pitch drives the front-to-back swing of the arm during forward and backward walking. Shoulder roll dominates during lateral stepping. Elbow pitch provides the flexion required to shorten the effective pendulum length of the arm, which is particularly relevant given the robot's long upper limbs. Elbow yaw, the fourth arm joint, contributes little to the swing itself and serves mainly a stabilizing role.

Cross-robot evaluation. The same environment was trained on the additional robots, and their arm swing is compared at equal Froude number (Eq. (2.23)). Each robot is evaluated at the speed matching that of the T1 at its reference speed, $v_i = v_{\mathrm{T1}}\sqrt{L_i / L_{\mathrm{T1}}}$, with $v_{\mathrm{T1}} = 0.46\,\mathrm{m\,s^{-1}}$ and $L_{\mathrm{T1}} = 0.52\,\mathrm{m}$ ($\mathrm{Fr} \approx 0.04$), so that the remaining differences can be attributed to morphology and learned behavior rather than to one robot walking relatively faster. The swing is quantified by the mass-weighted measure $(m_{\mathrm{arm}}/m_{\mathrm{body}})\,(\Delta x_{\mathrm{arm}}/L_{\mathrm{leg}})$, which combines the arm's share of the body mass with how far its center of mass travels relative to leg length.

4.4 Reference Motion Fine Tuning

The whole-body policies of Section 4.3 are designed to swing their arms within plausible bounds and with the correct timing, but the shape of the movement is limited to what the hand-modeled reward function specifies. This section extends them with reference motions, retargeted from an established motion library and from our own recordings, to induce the behavioral nuances that humans perform while walking. The references are organized by walking speed, so that the robot also adjusts its movements to its changing reference speed. This involves three stages: retargeting the human motion onto each robot, building a velocity-indexed reference library from it, and tracking that library during training.

The utilized retargeting in this work builds on OmniRetarget [61], whose interaction-mesh formulation is described in Section 2.4. Its implementation however, is not morphology-agnostic. Although mapping entries for the Booster T1 exist, the tuning behind them does not: the human targets are scaled by a fixed constant, fitted so that the Unitree G1, a humanoid of near-human segment proportions, meets the floor. The T1 stands at 1.20 m, roughly 29% shorter than the average human, and its segment ratios depart further still. When applied without prior changes, the algorithm over-scales the targets by approximately 10%.

Over-scaled targets lie beyond the robot's reach and drive its joints to saturation, where no gradient

remains to escape. In the legs, the ankles remain extended, so the robot walks on its toes and the feet never establish contact with the floor. In the arms, large motions lose tracking, with the joints pinned at their limits near, but not at, the intended keypoint. A second, independent defect compounds the first. Foot contact is only constrained laterally, which keeps the foot flat only when the ankle frame sits close to the sole. The T1's ankle sits well above it, so the toe is held in place while the heel is free to lift. None of these failures are reported by the solver, which converges at an unremarkable cost even when the feet float above the floor or remain permanently on tiptoe. Solver cost is therefore not a measure of physical validity.

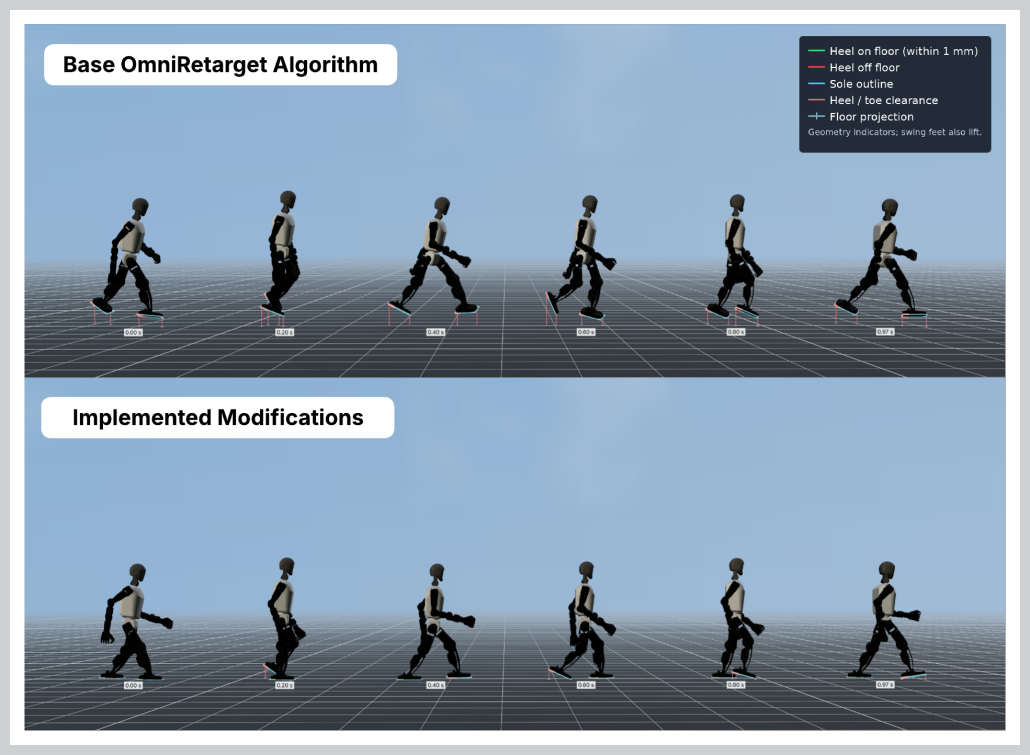


Figure 4.9: The Same Walking Clip Retargeted onto the T1 with the Base OmniRetarget Formulation (Top) and with the Implemented Modifications (Bottom) (Video).

These issues were addressed at three levels of the pipeline, with the result shown in Figure 4.9:

- **Preprocessing.** The fixed scale is replaced by one relative to the robot's height. It is calibrated so that the soles touch the floor, rather than by minimizing solver cost.

- **Keypoint selection.** The mesh nodes used for each robot are selected by their kinematic manipulability, given by the robot's Jacobian, rather than by their anatomical name. A same-named link may sit elsewhere, such as the T1's ankle frame well above the sole, and a node the joints barely move gives the solver a target it can only chase by saturating them.

- **Optimization.** A soft term draws the contacting soles flat onto the ground, and a hard constraint keeps a minimum lateral separation between the feet. The contact term is soft rather than a hard height constraint, since the latter becomes infeasible when the foot starts far from the floor.

The resulting problem, solved at every frame t, is

$$
\begin{aligned}
q_t^\star = \underset{q_t}{\operatorname{argmin}} \quad & \underbrace{\sum_i \left\| L\big(p_{t,i}^{\text{source}}\big) - L\big(p_{t,i}^{\text{target}}(q_t)\big) \right\|^2 + \|q_t - q_{t-1}\|_Q^2}_{\text{Reference}} + \underbrace{w_g \sum_{i \in \mathcal{C}_t} s_{t,i} \left(e_z^\top f_i(q_t) - \bar{z}_{t,i} \right)^2}_{\text{Sole Contact}} \\
\text{subject to} \quad & \textbf{the original OmniRetarget constraints}, \\
& \underbrace{n(q_t)^\top \left(f_l(q_t) - f_r(q_t) \right) \geq d_{\min}, \qquad \forall (l,r) \in \mathcal{F}_L \times \mathcal{F}_R,}_{\text{Lateral Foot Separation}}
\end{aligned}
\tag{4.3}
$$

where $f_i(q_t)$ is the position of a sole keypoint i, $\mathcal{C}_t$ the set of sole keypoints in contact, $\mathcal{F}_L$ and $\mathcal{F}_R$ the sole keypoints of the left and right foot, and $n(q_t)$ the trunk lateral axis projected onto the ground plane. The height target $\bar{z}_{t,i}$ moves each sole keypoint towards the floor by at most δ per frame, and $s_{t,i}$ fades the term out over the last r frames before lift-off, k frames from it, so the heel rolls off naturally:

$$
\bar{z}_{t,i} = z_{t-1,i} + \operatorname{clip}\left(z_{\text{floor}} - z_{t-1,i}, -\delta, \delta \right), \qquad s_{t,i} = \min\left(1, \left(\frac{k+1}{r+1} \right)^2 \right). \tag{4.4}
$$

The same modifications were applied to the six further robots, each of which reproduced one of the T1's failure modes before the algorithm modifications in a different form, and the added parameters were tuned for each robot. All robots thus attempt to mimic the same human motion, and how closely each one reproduces it reflects its own physical properties, which is the opposite perspective to the energy efficiency objective, where each robot was left to find its own movement.

4.4.1 Reference Motion Library

The retargeted motions are the raw material for the reference-motion training environments, but on their own they do not suit a locomotion policy. As discussed in Section 3.2.4, motion-capture takes are recorded for other purposes, with walking interleaved with unrelated movements at speeds the performer chose freely. A locomotion policy, by contrast, is conditioned on a commanded velocity and runs on a periodic gait clock. Bridging the two requires three steps: isolating individual gait cycles from the takes, normalizing them onto a common gait clock, and organizing them into a library indexed by walking speed. The policy can then imitate motion appropriate to whatever velocity it is commanded. The goal is whole-body locomotion that is expressive at high speed, particularly in its upper-body swing, and more restrained as the commanded speed decreases.

Gait Cycle Extraction. Walking is periodic, so a single stride (one right step followed by one left step) is a complete reference that can be repeated indefinitely. Strides are extracted from the retargeted robot motion rather than from the human data, so that the detected contact events are those the robot itself will reproduce.

For each frame t, forward kinematics gives the height of the lowest contact sphere of each foot, z_t^L and z_t^R. A foot is considered in contact when it lies within a margin $\epsilon = 3$ cm of the lowest height reached over the whole take:

$$
c_t^s = 1_{z_t^s < z_{\text{floor}} + \epsilon}, \qquad s \in \{L, R\}. \tag{4.5}
$$

Touchdowns are the rising edges of c_t^s. A candidate cycle $t \in [a, b]$ is defined between two consecutive right-foot touchdowns, at frames a and b. Each candidate is scored by how suitable it is as a looping reference:

$$S_{ab} = \underbrace{\left\|\mathbf{x}_a - \mathbf{x}_b\right\|}_{\text{Loop Closure}} + 2\,\underbrace{\frac{|T_{ab} - \tilde{T}|}{\tilde{T}}}_{\text{Regularity}} + 0.02\,\underbrace{|\Delta\psi_{ab}|}_{\text{Heading Change}} + 5 \cdot \underbrace{1_{n_L \neq 1}}_{\text{Alternation}}. \tag{4.6}$$

The terms are as follows:

- $\mathbf{x}_t$ stacks the joint angles and the root height at frame t, so the first term measures how well the cycle closes on itself.
- T_{ab} is the cycle's duration and $\tilde{T}$ the median cycle duration in the take. The second term therefore favors strides typical of steady walking over accelerations and stops.
- $\Delta\psi_{ab}$ is the change in root heading (in degrees), which excludes turning strides.
- The last term requires exactly one left touchdown inside the cycle (n_L counts the rising edges of c_t^L for $t \in [a, b]$), which enforces a clean alternating stride.

Low-scoring cycles are selected. Each selected frame range is then solved again in isolation by the retargeter, with a one-second lead-in so that the warm-started solution matches the one obtained over the full take.

The selected cycles are replayed in simulation at the 50 Hz control rate. The replay records joint positions and velocities and the root trajectory. The captured walking speed of each cycle is taken as the mean planar speed of the root over the stride:

$$\bar{v}_i = \frac{1}{T_i} \sum_{t=1}^{T_i} \left\|\dot{\mathbf{p}}_{xy,t}^{\text{root}}\right\|. \tag{4.7}$$

Temporal Normalization. The extracted cycles naturally differ in duration: slow walking produces longer strides in time than fast walking. The training environment, however, drives the gait with a single phase clock, and letting the reference frequency vary between cycles would require the clock to change rate whenever the reference changes. All cycles are therefore normalized to a common length of $N = 49$ frames by linear interpolation over their full span:

$$\hat{Q}_i[n] = Q_i\left(\frac{n}{N-1}\,(T_i - 1)\right), \qquad n = 0, \ldots, N-1, \tag{4.8}$$

where n indexes the frames of the normalized cycle, as opposed to the time step t. This fixes a common gait frequency

$$f_{\text{ref}} = \frac{50\ \text{Hz}}{N} \approx 1.02\ \text{Hz}. \tag{4.9}$$

The normalization changes only each cycle's tempo. The spatial shape of the stride and of the arm swing is preserved, and each cycle keeps its captured speed $\bar{v}_i$ as a label of its style, while the robot's actual speed is set by velocity tracking. As a result, every cycle is aligned frame by frame with every other: frame n corresponds to the same instant of the gait in all of them.

Phase Alignment. The environment's gait clock has a fixed convention: the left foot is expected to swing around normalized gait phase $g = 0.25$ and the right foot around $g = 0.75$. Each cycle begins at a right touchdown, but where its swing phases fall depends on the proportion of stance to swing in that particular stride. Each cycle i therefore receives a phase offset o_i, measured from its foot-height trajectory, that moves the left-foot swing apex, at $g_i^{L\text{-apex}}$, to $g = 0.25$:

$$o_i = \left(0.25 - g_i^{L\text{-apex}}\right) \bmod 1. \tag{4.10}$$

Since the right swing follows the left by about half a stride, the right-foot swing apex of every cycle then also falls within the right-swing window. The reference and the environment's contact-timing rewards therefore agree on the timing of both feet.

Library Composition. The library combines cycles from two sources. LAFAN1 [58] supplies high-quality fast walking. Our own captures, recorded with an OptiTrack optical motion-capture system[2], supply slower walking with more restrained upper-body motion. Together they give 26 cycles spanning 0.23–1.28 m/s: 18 from our recordings covering the lower range and 8 from LAFAN1 covering the upper range. Each cycle is stored with its speed label and phase offset. At load time the cycles are sorted by speed and stacked into a single tensor $\mathcal{Q} \in \mathbb{R}^{R \times N \times n_u}$, where $R = 26$ and n_u is the number of actuated joints.

Before stacking, the arm trajectories are lightly conditioned for the T1's morphology. Where the retargeted right arm of a cycle does not form a valid swing, it is reconstructed by mirroring the left arm, shifted by half a cycle (24 frames, wrapping around its end) to restore the anti-phase swing:

$$\mathbf{q}_R[n] = S\,\mathbf{q}_L\left[n - \tfrac{N}{2}\right], \qquad S = \mathrm{diag}(1, -1, 1, -1), \tag{4.11}$$

where S flips the sign of the roll and yaw joints between the left and right sides, acting on shoulder pitch, shoulder roll, elbow pitch and elbow yaw. Elbow flexion is then scaled by 0.7, and both the backward shoulder swing and the shoulder roll by 0.9. These adjustments increase clearance between the hands and the legs, which is limited on the T1 by its wide, proportionally large arm links.

4.4.2 Velocity-Conditioned Reference Tracking

Phase Clock and Clip Selection. The per-episode gait frequency is fixed to the library frequency, $f = f_{\text{ref}}$. At a control period of $\Delta t = 0.02\,\text{s}$, the phase clock then advances exactly one reference frame per control step:

$$\phi_{t+1} = \left((\phi_t + 2\pi f_{\text{ref}}\,\Delta t + \pi) \bmod 2\pi\right) - \pi. \tag{4.12}$$

At every step, the tracked cycle is the one whose captured speed is closest to the planar speed of the command $\mathbf{c} = (v_x, v_y, \omega_z)$:

$$i^* = \arg\min_i \left|\, \bar{v}_i - \|\mathbf{c}_{xy}\| \,\right|. \tag{4.13}$$

[2]OptiTrack: `https://optitrack.com`

Because all cycles share a single phase clock, a change in the commanded velocity changes the style and stride of the reference while its step frequency stays the same, so switching cycles never breaks the rhythm of the gait. The reference pose is read from the selected cycle at the current gait phase $g = \big((\phi_t + \pi) \bmod 2\pi\big)/2\pi$ by linear interpolation, wrapping around the end of the cycle:

$$u = \big((g - o_{i^*}) \bmod 1\big)\, N, \qquad \hat{\mathbf{q}}_t = (1-\alpha)\, \hat{Q}_{i^*}[\lfloor u \rfloor] + \alpha\, \hat{Q}_{i^*}[(\lfloor u \rfloor + 1) \bmod N]\,, \tag{4.14}$$

with $u \in [0, N)$ the sample index and $\alpha = u - \lfloor u \rfloor$ the interpolation weight.

The reference specifies joint angles only. It is independent of the robot's position and heading, so the velocity-tracking rewards alone decide where the robot goes, while the reference shapes how it moves. The policy's observations are unchanged: it receives the command $\mathbf{c}$ and the phase $(\cos\phi_t, \sin\phi_t)$, and it learns to associate each commanded speed with the corresponding style of motion.

Imitation Reward. Joint imitation is rewarded separately for the arms and the legs:

$$r_G = 1_{\|\mathbf{c}\|>0.1} \cdot \exp\left(-\frac{1}{\sigma_G} \sum_{j \in G} w_j\, (q_j - \hat{q}_j)^2\right), \qquad G \in \{\text{Arms}, \text{Legs}\}, \tag{4.15}$$

with reward weights of 2.0 for the arms and 1.5 for the legs. Splitting the reward keeps the error of one group from saturating the shared exponential and suppressing the learning signal of the other.

The per-joint weights w_j emphasize the joints that carry the character of the gait:

- $w_j = 1.0$ for hip pitch, knee pitch, shoulder pitch and roll, and elbow pitch and yaw;
- $w_j = 0.5$ for ankle pitch;
- $w_j = 0.3$ for the remaining leg joints (hip roll and yaw, ankle roll).

The reference therefore guides the style of the lateral balance joints without overriding the balance strategy. The widths $\sigma_G = 1.0$ were set close to the error of a policy that does not imitate the reference, which keeps the reward in its most informative region early in training.

The indicator disables imitation when the robot is commanded to stand still. In that case a position-only posture term holds the robot near its home pose:

$$c_{\text{stand}} = 1_{\|\mathbf{c}\|<0.1} \sum_j \omega_j\, (q_j - q_j^{\text{home}})^2, \qquad \omega_j = \begin{cases} 1.0 & \text{Arms} \\ 0.2 & \text{Legs} \end{cases} \tag{4.16}$$

with weight -1.0. The term penalizes deviations in position only, not velocity, so balance corrections remain free. The legs are weighted lightly so that corrective steps are not discouraged.

Adaptations of the Whole-Body Environment. The reference-motion environment is derived from the whole-body locomotion environment described in Section 4.3.1. Some of the terms previously included in the reward, encoded behaviors that the reference now supplies, and they were adapted accordingly (Table 4.2). The arm roll, pitch and yaw windows held the arms within fixed joint bands, which

would oppose the reference swing, so they were removed. The base-height penalty was relaxed to allow the natural vertical oscillation of the center of mass and the knee flexion of the reference. The randomized gait frequency was replaced by the library frequency, which keeps the phase clock synchronized with the reference. Finally, the imitation and stand-still terms described above were added. All other terms carry over unchanged: velocity tracking, orientation, energy and torque regularization, action smoothness, collision avoidance, and the foot-contact rewards. The foot-swing reward in particular is retained, and the phase alignment of the library is what makes it consistent with the reference.

Table 4.2: Modified, Removed and Added Terms in the Reference-Motion Environment.

Component	**Equation**	**Whole-Body**	**Reference**
Arm Roll Window	$1_{\phi_{\mathrm{L}} \notin [\phi_{\mathrm{L,min}}, \phi_{\mathrm{L,max}}]} + 1_{\phi_{\mathrm{R}} \notin [\phi_{\mathrm{R,min}}, \phi_{\mathrm{R,max}}]}$	-0.1	–
Arm Pitch Window	$1_{\theta_{\mathrm{L}} \notin [\theta_{\mathrm{L,min}}, \theta_{\mathrm{L,max}}]} + 1_{\theta_{\mathrm{R}} \notin [\theta_{\mathrm{R,min}}, \theta_{\mathrm{R,max}}]}$	-0.1	–
Arm Yaw Window	$1_{\psi_{\mathrm{L}} \notin [\psi_{\mathrm{min}}, \psi_{\mathrm{max}}]} + 1_{\psi_{\mathrm{R}} \notin [\psi_{\mathrm{min}}, \psi_{\mathrm{max}}]}$	-0.1	–
Base Height	$(h - h_{\mathrm{target}})^2$	-20.0	-4.0
Gait Frequency	f	$\mathcal{U}(0.75,\, 1.25)\,\mathrm{Hz}$	$f_{\mathrm{ref}} \approx 1.02\,\mathrm{Hz}$
Leg Imitation	$1_{\lVert \mathbf{c} \rVert > 0.1} \cdot \exp\big(-\sum_{j \in \mathrm{legs}} w_j (q_j - \hat{q}_j)^2 / \sigma \big)$	–	1.5
Arm Imitation	$1_{\lVert \mathbf{c} \rVert > 0.1} \cdot \exp\big(-\sum_{j \in \mathrm{arms}} w_j (q_j - \hat{q}_j)^2 / \sigma \big)$	–	2.0
Stand Still	$1_{\lVert \mathbf{c} \rVert < 0.1} \cdot \sum_j \omega_j (q_j - q_j^{\mathrm{home}})^2$	–	-1.0

4.5 Height Traversal Pipeline

The walking policy becomes unstable on steps higher than approximately 4 cm. The smaller heights it does clear are a result of the stage of the curriculum in which it trained on mildly irregular terrain: any irregularity larger than the tallest bump of that terrain lies outside everything the policy has experienced.

The naive approach is to place stairs in the environment and expect the velocity-tracking rewards to be sufficient for the policy to learn the necessary motions to climb them. In practice, however strongly these rewards are weighted, the policy settles on refusing to move once the robot is pressed against the first set of steps. This avoids falling, but the desired movement is never learned.

What is required instead is a set of rewards that actively induce the climbing movement, such as rewarding the robot for reaching a higher altitude. With a proprioceptive-only observation, such rewards reproduce the blind strategy discussed in Section 3.2.5: the robot clears taller steps than the walking policy, but it does so by stumbling into the step and relying on a learned recovery reflex to climb it (as seen in this video). For the reasons given there, the climbing policy developed here perceives the terrain ahead, and its rewards follow, in summary, the sequence in which a person climbs: seeing the step, placing the stance foot at a suitable distance, lifting the swing foot over the height, and then transferring weight onto the tread.

The primary target was for the Booster T1 to traverse the maximum step height permitted by building regulations, of approximately 18 cm [3]. This is no small feat. Given the T1's blocky feet and short stature, an 18 cm step corresponds, in proportion to its size, to a person stepping onto 36 cm steps in a controlled manner, repeatedly, for an entire flight. Training is designed around robustness over this regulated range.

[3] In Portugal, Decreto-Lei n.º 163/2006 limits the step height of stairs in public buildings to 0.18 m. Other European codes set similar limits for public and accessible buildings.

Taller steps are still included as an exploration ceiling, so that the maximum height each robot in this study can reach is found rather than imposed.

To achieve this, a dedicated climbing policy was built on top of the walking policy. It observes a heightmap of the terrain ahead, which on hardware is reconstructed from the head camera, decomposes stair traversal into the same sub-movements a person performs, and hands control back to the expressive walking policy on flat ground.

4.5.1 Stair Terrain and Curriculum

Terrain. The stairs are built as an inverted pyramid: a central platform with a half-width of 1.5 m, surrounded by 12 concentric square rings, each one step higher than the last, and finally a 6 m flat area. Walking radially outward ascends the flight, and walking inward descends it, so a single structure serves both tasks. The central area ensures that successful ascents and descents end on flat ground instead of at a ledge that the terrain observation would not anticipate.

Every parallel environment samples its own staircase once, at the start of training. The first step height is $h_{\min} \sim \mathcal{U}(0.02, 0.04)$ m and the final step height is $h_{\max} \sim \mathcal{U}(0.08, 0.18)$ m, covering the regulated range. The step heights increase linearly over the first eight rings and remain at $h_{\max}$ for the last four:

$$h_k = \begin{cases} h_{\min} + \dfrac{k-1}{7}\,(h_{\max} - h_{\min}), & k = 1, \ldots, 8, \\ h_{\max}, & k = 9, \ldots, 12. \end{cases} \tag{4.17}$$

The step depth is sampled as $\ell_s \sim \mathcal{U}(0.4, 0.8)$ m. Each flight therefore acts as its own curriculum: every ascent starts gently and ends at the flight's hardest step. In addition, 20% of the environments are anchored at $h_{\max} = 18$ cm, spread evenly over step depths of 0.4 to 0.8 m. This guarantees steady exposure to the target height at every step depth, rather than leaving it to the tail of a continuous distribution. To probe the limits of each robot beyond the regulated range, a later ascent-only stage, warm-started from the stair policy, draws $h_{\max} \sim \mathcal{U}(0.18, 0.30)$ m in 30% of flights, while the remaining 70% keep the regulated distribution so that the behavior within it is preserved. The lift and tempo targets extend the same linear laws above 18 cm, with the lift ceiling raised to 35 cm, and all other settings are unchanged.

Spawn curriculum. A policy that always starts on flat ground encounters its first step rarely, and only after it has already learned to walk up to it, which reinforces the "walk to the step and stop" behavior. Instead, 76% of episodes start on the stairs. The robot is placed mid-tread on a random ring and face, with a small offset along the tread, and receives a forward command of 0.5 m/s:

- 68% of stair episodes face outward and ascend, starting on rings 1–8;
- 32% are rotated by 180° and descend, starting on rings 1–12, with the 18 cm anchored flights always starting at the top so that the full plateau is descended.

The remaining 24% of episodes start on flat ground with the usual randomly sampled commands. Overall, training time is split into approximately 52% ascent, 24% descent and 24% flat walking. The flat

share attempts to preserve the walking behavior and the approach from flat ground to the first step.

4.5.2 Terrain Perception

The climbing policy observes a 5×5 heightmap in its own heading frame. The samples span 0.1–0.7 m ahead of the base and ± 0.3 m to either side. Each sample $\mathbf{p}_{ij}$ is expressed relative to the terrain directly under the base:

$$\mathrm{hm}_{ij} = \mathrm{clip}\Big(\min\big(z_{\text{terrain}}(\mathbf{p}_{ij}) - z_{\text{terrain}}(\mathbf{p}_{\text{base}}),\ h_{\text{next}}\big),\ -1,\ 1\Big). \tag{4.18}$$

Two choices here are deliberate:

- **Referencing the terrain under the base, rather than the feet.** On flat ground the heightmap is then identically zero. Referencing the feet would make it oscillate with the gait and feed that oscillation back into the policy.
- **Capping positive values at the next step height h_{next}.** On narrow treads the grid may be able to see two steps ahead. The cap keeps the signal equal to the step that is actually about to be climbed. Negative values, which indicate drops, are not capped, because the second edge ahead is the only cue for how much landing room a narrow tread leaves on descent.

Gaussian noise of 1 cm per cell is added to the observed heights during training. On the robot, the same heights are built from the depth camera's point cloud (Section 4.8).

The heightmap is appended to the walking policy's 71-dimensional observation, giving 96 dimensions. The climbing policy is **warm-started** from the walking policy: the weights of the first layer are copied, the 25 new input columns are initialized to zero, and the observation normalizer is extended with unit statistics for the new inputs. At the start of training, the climber therefore behaves exactly like the walker and only has to learn how to use the new input. The critic is extended in the same way.

4.5.3 Step Engagement and Adaptation of the Walking Objective

Several terms of the walking objective oppose stair traversal. The reference gait, a periodic swing clock and the smoothness penalties all encourage a flat-ground stride, whereas tall steps require hip and knee flexion well outside the walking reference, both up and down. These terms are not removed. Instead, they are released gradually through a continuous **step-engagement** signal $w \in [0, 1]$, so that the flat-ground behavior is untouched away from the stairs.

Engagement. Ascent engagement is computed from the highest point h_{near} of the two nearest heightmap rows, and it ramps over a small height band:

$$w_{\text{asc}} = \mathrm{clip}\left(\frac{h_{\text{near}} - 0.005}{0.018 - 0.005},\ 0,\ 1 \right). \tag{4.19}$$

The ramp avoids a discontinuous step in the reward as a function of a continuous observation, which a value function fits poorly. Ascent engagement is also held at 1 in two situations:

- while a foot is already positioned at the step, since a small yaw or a backward shift of the center of mass can hide a tall lip from the first heightmap rows;
- while a foot is still in the air on a tall step, until it lands.

Descent engagement w_{desc} is computed in the same way from the depth of the drop ahead, and it is zero on every ascending episode. The walking terms are released by the combined signal

$$w = \max(w_{\text{asc}},\, w_{\text{desc}}), \tag{4.20}$$

since a flat-ground stride opposes stepping down as much as stepping up.

Modified walking terms. With h the height of the step or the depth of the drop ahead, the flat walking terms are modified as follows (Table 4.3):

- **Reference imitation and phase-based foot swing** are scaled by $1 - \rho(h)\, w$. The release depth $\rho(h)$ grows linearly from 0.5 for steps of 5 cm or less to 0.95 for steps of 15 cm or more. On low steps the reference gait remains a useful prior. On tall steps it is almost entirely released, because it would directly oppose the required flexion. **Arm** imitation follows the same principle, but its overall weight is reduced, in order to allow for upper-body balance maneuvers at the highest steps, whilst still maintaining proper coordination.
- **Smoothness and energy penalties** (action rate, joint velocity and acceleration, base acceleration, torque, tiredness and power) are scaled by $1 - 0.6\, w$, allowing the brief, larger effort that a step requires. Foot slip is not relaxed, so the stance foot remains firmly planted.
- **Vertical base velocity and base height** are scaled by $1 - w_{\text{asc}}$, since rising is precisely the goal of ascent. On descent they are kept at full weight: they are the natural penalty against falling, and releasing them while descent progress pays for losing height would make tumbling down the stairs the most rewarding behavior. Base height is measured relative to the terrain under the robot instead of the world floor.
- **Forward velocity tracking** keeps its peak value of 1, but its error tolerance is widened by up to $0.7\, w$. Slowing down in front of a step then costs little, and the robot is not rewarded for turning away until the step leaves its field of view.

Slower gait clock. Traversing a step is slower than walking. The phase clock is therefore slowed near a step:

$$\dot{\phi} = 2\pi f_{\text{ref}} \big(1 - s(h)\, w\big), \tag{4.21}$$

where $s(h)$ grows from 0.25 (75% cadence on small steps) to 0.5 (50% cadence on tall steps).

Expanded flexion authority. The action mapping of the walking policy cannot command the deep, sustained hip and knee flexion that tall steps require: an 18 cm step needs approximately 1.3 times the

walker's action range. Actions that bend the hip and knee are therefore amplified by up to 1.35, blended in only for steps between 10 and 16 cm and only while the robot is engaged with a step:

$$q^{\text{target}} = q^{\text{home}} + a\,\Big(1 + \lambda\,(k_{\text{flex}} - 1)\Big), \qquad \lambda = w \cdot \text{clip}\left(\frac{h - 0.10}{0.06},\, 0,\, 1\right). \tag{4.22}$$

This extends the policy's reach without extending what the robot is allowed to do. At full expansion the deepest targets are -1.55 rad at the hip and 1.75 rad at the knee, against joint limits of -2.06 and 2.15 rad, and the actuator torque limits of 45 and 65 Nm are enforced exactly as before. Only actions that bend the hip and knee are amplified, so actions that straighten them, which support the robot's weight, are mapped as before. On flat ground the mapping is identical to the walker's, so the learned walking gait is not reinterpreted.

Table 4.3: Walking Objective Terms Changed on the Height Traversal Objective.

Term	**Modification**
Reference Imitation and Foot Swing	$\times\,\big(1 - \rho(h)\,w\big),\ \ \rho \in [0.5,\, 0.95]$
Smoothness and Energy Penalties	$\times\,(1 - 0.6\,w)$
Vertical Velocity and Base Height	$\times\,(1 - w_{\text{asc}})$
Forward Velocity Tracking	Tolerance Widened by $0.7\,w$
Gait Clock	$\dot{\phi} \times \big(1 - s(h)\,w\big),\ \ s \in [0.25,\, 0.5]$
Hip/Knee Flexion Range	$\times\,\big(1 + \lambda\,(1.35 - 1)\big)$
Foot Slip	unchanged

4.5.4 Ascent

A person climbing stairs in a step-to-step pattern follows a repeatable sequence of sub-movements: (i) approaching the step; (ii) placing the stance foot at a distance that leaves sufficient clearance for the swing; (iii) lifting the swing foot above the height; (iv) placing it fully on the tread; (v) transferring weight and bringing the trailing foot up beside it; and (vi) pausing briefly before the next step. The ascent rewards reproduce this sequence one sub-movement at a time (Table 4.4). Most of them are **event rewards**, granted once at touchdown rather than at every control step. Their weights are correspondingly large: an event reward is scaled by the control period Δt only once per occurrence, whereas a continuous term accumulates over every control step.

Approach. While a step is visible but the feet are not yet in position, forward velocity towards the flight is rewarded. The weight of this reward fades as the nearest foot enters the engagement band, so the robot keeps closing the gap until the **foot**, not only the base, is at the step.

Positioning. Each touchdown is rewarded by its distance d from the center of the sole to the next step:

$$r_{\text{foothold}} = \exp\left(-\frac{(d - d^*)^2}{\sigma^2}\right), \qquad d^* = \frac{\ell_f}{2} + \delta + k\,\max(0,\, h - h_0), \tag{4.23}$$

where ℓ_f is the foot length, δ the desired toe-to-step gap and h the step height. Above h_0, the target moves back with height to leave room for the swing arc, since a foot placed close to a tall step scrapes

up its face. The reward is paid only if the foot was airborne for at least T_{air} and moved at least Δd closer to the step, so tapping in place earns nothing. For the T1, $\ell_f/2 + \delta = 13\,\text{cm}$, $k = 0.8$, $h_0 = 10\,\text{cm}$, $\sigma = 5\,\text{cm}$, $T_{\text{air}} = 0.1\,\text{s}$ and $\Delta d = 5\,\text{cm}$.

Lift. The required foot clearance is set once, at lift-off, by probing the terrain 0.30 m ahead of the lifting foot in the direction of travel:

$$h^* = \text{clip}\left(\Delta z_{\text{ahead}} + 0.05,\ 0.12,\ 0.30\right), \tag{4.24}$$

that is, the step height plus a 5 cm margin, never lower than the flat-ground swing height and capped at 0.30 m, so that an outlier in the heightmap cannot demand a lift beyond the robot's reach. Freezing the target at lift-off shapes the entire swing towards the real obstacle from its first frame. Rather than demanding the full apex immediately, which would reward an explosive kick, the required height follows a smooth schedule of the air time t_{air}. The rise time T_{rise} increases from 0.35 s for 5 cm steps to 0.58 s for 18 cm steps, and the amount by which the foot's highest lift so far falls below this schedule is penalized:

$$c_{\text{clear}} = \sum_{\text{airborne } f} \max\Big(0,\ h_f^*\,\mathcal{S}\big(\tfrac{t_{\text{air},f}}{T_{\text{rise}}}\big) - \big(z_f^{\text{peak}} - z_f^{\text{liftoff}}\big)\Big), \qquad \mathcal{S}(x) = x^2(3-2x). \tag{4.25}$$

Forward progress of the swing foot is rewarded only once it has reached at least half of the required clearance, and only up to a speed cap of 0.7 m/s. At that cap, a slower swing covering the same distance earns more reward, and the slower swing is what gives the foot time to rise higher and land deeper on the tread. The sole is also penalized for tilting excessively during the swing, which moves the lift primarily into the hips and knees instead of the ankles.

Tempo. Each touchdown on a climbing step is rewarded when its air time matches a target that depends on the step height:

$$r_{\text{tempo}} = \exp\left(-\frac{(t_{\text{air}} - T^*(h))^2}{0.04}\right), \qquad T^* \in [0.55,\ 0.78]\,\text{s}. \tag{4.26}$$

The range was derived from inverse-dynamics estimates of the earliest apex times that the hip torque can achieve (0.46 s, 0.55 s and 0.58 s for 11, 16 and 18 cm), plus approximately 0.2 s to move onto the tread and settle.

Landing and gathering. Touching down with the toe within 10 cm of the next step, or with the sole across a height, is penalized, as is continuous contact between a foot and a step face. Rising is rewarded only for new maxima of base height, so the robot cannot earn it by oscillating up and down in place. The trailing foot is rewarded for landing beside the planted lead foot on the same tread, 4 cm ahead of it. Together with the positioning reward, this makes the human step-to pattern the most rewarded route: meet the lead foot, then take a short positioning step. Lifting a foot before both feet have been planted for 0.15 s is penalized, and so is having both feet in the air near a step. Together these enforce a settled weight transfer between steps.

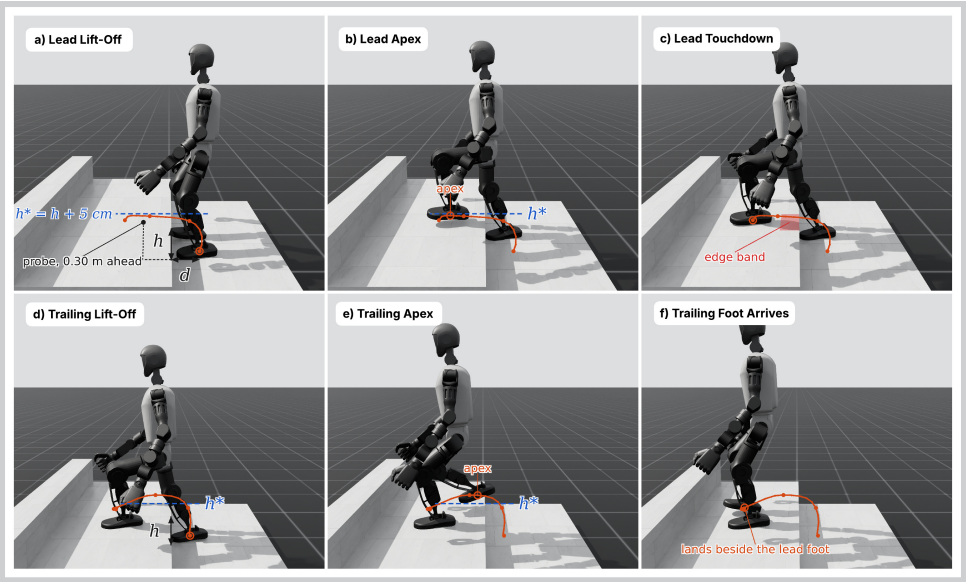


Figure 4.10: A Single Ascent onto a 17.9 cm Step, Taken from a Rollout of the Climbing Policy. The top row follows the lead foot and the bottom row the trailing foot. In each panel, the swing path of the moving foot's sole is drawn with a dot every control step (0.02 s) and a larger dot every 0.1 s, and a ring marks the foot's position in that frame. **(a)** Lead lift-off: the stance foot is at distance d from the step of height h, and the terrain is probed 0.30 m ahead of the lifting foot to set the clearance target $h^* = h + 5$ cm, which is frozen for the whole swing. **(b)** Lead apex: the foot reaches its highest point at the clearance target. **(c)** Lead touchdown on the upper tread, beyond the edge band within 10 cm of the height, where touchdowns are penalized. **(d)** Trailing lift-off, with the target frozen again for this swing. **(e)** Trailing apex, above the clearance target. **(f)** The trailing foot arrives beside the planted lead foot on the upper tread.

Anti-stall and heading. A stall timer starts when the robot has a forward command and is positioned at a step but is not making upward progress. Lifting a foot without climbing does not reset it, so tapping at the step cannot avoid the cost. Only reaching a higher tread or base level does. After a grace period of 0.8 s, a cost ramps in over 1 s. This makes parking in front of a tall step strictly worse than attempting it. The torso is also kept facing the direction of travel while climbing, and this constraint is released when a yaw command is given.

4.5.5 Descent

The policy is not told whether it is ascending or descending. It infers this from the heightmap, where positive values indicate a step ahead and negative values a drop. Descent is not a mirror of ascent, however. Going down, the difficulty lies in controlling the lowering motion and landing safely on a tread of limited depth, rather than in generating lift. Its rewards are therefore designed separately, and every one of them is gated by the descent-engagement signal w_{desc}, which activates solely on descending episodes, leaving the ascent objective unchanged.

Progress. Downward progress is rewarded as a potential over new **minimum** heights reached by a supported foot, counted only once the center of the foot has crossed the step edge. Each level is therefore paid for once. A rushed fall that skips several levels forfeits the reward, and lowering the stance or hanging a foot over the edge earns nothing.

Table 4.4: Ascent Reward Terms.

Component	Equation	Weight
Climbing	$\dot{z}^{+}_{\text{base}}\,(\text{new maxima}) \cdot 1_{v_x^{\text{cmd}}>0.1} \cdot w$	5.0
Approach	$v_{\text{fwd}} \cdot 1_{\text{step ahead}} \cdot (1 - w_f)$	1.0
Positioning Foothold	$\exp\big(-(d-d^*)^2/0.0025\big) \cdot 1_{\text{real step}}$	20.0
Feet Together	$\exp\big(-(\Delta x - 0.04)^2/0.008\big) \cdot 1_{\text{rose, same tread}}$	20.0
Swing Forward	$\min(\dot{x}^{+}_{\text{swing}},\, 0.7) \cdot g_{\text{clear}}$	2.5
Step Tempo	$\exp\big(-(t_{\text{air}} - T^*(h))^2/0.04\big)$	5.0
Step Extend	fraction of tread depth reached at touchdown	1.0
Foot Clearance	$\max\big(0,\, h^* \mathcal{S}(\cdot) - \Delta z^{\text{peak}}\big)$	-8.0
Foot Orientation	$\text{sole tilt}^2 \cdot 1_{\text{swing}}$	-5.0
Step Edge	$1_{\text{sole across height at touchdown}}$	-15.0
Foot on Step	$1_{\text{touchdown within 0.10 m of step}}$	-2.0
Step Scrape	$1_{\text{sole contacts step face}}$	-5.0
Rushed Lift	$1_{\text{lift with double support}<0.15\,\text{s}}$	-10.0
Flight on Step	$1_{\text{both feet airborne}} \cdot w$	-1.0
Step Dwell	$\text{clip}\big((t_{\text{stall}} - 0.8)/1.0,\, 0,\, 1\big)$	-1.0
Stair Heading	$1 - \cos\psi_{\text{err}}$	-2.0

Controlled lowering. Because descending pays for losing height, the cheapest way to earn it would be to step off the edge and fall. This is prevented by a cost on both the base's downward speed and the swing foot's downward speed at touchdown, above a free band of 0.35 m/s:

$$c_{\text{slam}} = \Big(\max(0,\, -\dot{z}_{\text{base}} - 0.35)^2 + \sum_{\text{touchdowns}} \max(0,\, -\dot{z}_{\text{foot}} - 0.35)^2\Big)\, w_{\text{desc}}. \tag{4.27}$$

Controlled lowering within the free band is therefore not penalized.

Landing. Ascent requires a precise stance distance, since the position relative to the step determines whether the next lift is possible. Descent only requires the foot to land safely on the lower tread, clear of both edges. The landing reward is therefore a plateau over the distance d past the edge that was just crossed, instead of a Gaussian around a single point:

$$r_{\text{land}}(d) = \begin{cases} 0, & d < 0.12, \\ \text{Ramp from 0 to 1}, & 0.12 \le d < 0.16, \\ 1, & 0.16 \le d \le 0.24, \\ \text{Decay from 1 to 0}, & 0.24 < d \le 0.35, \end{cases} \tag{4.28}$$

Below 12 cm the heel still overhangs the edge, and beyond 0.35 m the step becomes a lunge. Because the band is defined relative to the crossed edge, it does not depend on tread width, and a 20 cm step-down is not scored the same as a 60 cm lunge on a wide tread. A touchdown that leaves less than 4 cm before the **next** edge is penalized separately.

To keep the landing reward learnable, 25% of it is paid as soon as a supported contact with the lower tread occurs, and the remainder once the full sole is in contact. The feet-together reward is mirrored, with the trailing foot centered beside the lead foot rather than ahead of it, since on a narrow tread any

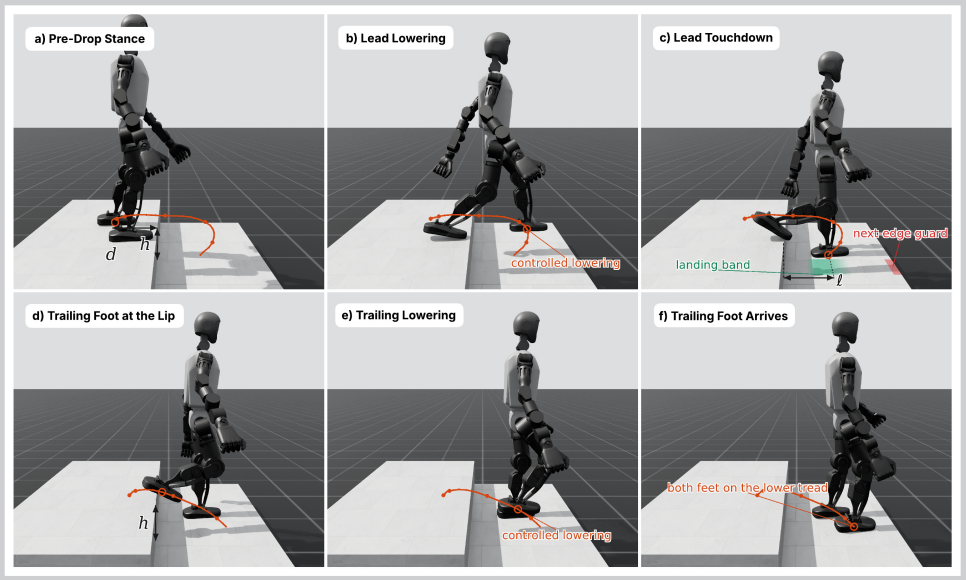


Figure 4.11: A Single Descent Down a 17.9 cm Drop, Taken from a Rollout of the Climbing Policy. The top row follows the lead foot and the bottom row the trailing foot. In each panel, the swing path of the moving foot's sole is drawn with a dot every control step (0.02 s) and a larger dot every 0.1 s, and a ring marks the foot's position in that frame. **(a)** Pre-drop stance: the lead foot is at distance d from the lip of a drop of height h. **(b)** The lead foot is lowered past the lip. **(c)** Lead touchdown at distance ℓ past the crossed lip. The landing reward is full over the band 0.16–0.24 m past the lip and fades out on either side, as indicated by the shading. Touchdowns within 4 cm of the next lip are penalized (next-edge guard). **(d)** The trailing foot at the lip. **(e)** The trailing foot is lowered. **(f)** Both feet on the lower tread.

forward offset would carry it towards the next lip.

Preparation and safety. Before a step-down, the feet are guided towards a pre-descent stance: the foot that will move rests 12–15 cm from the lip, and the supporting foot 16–24 cm from it. On each step, reward is paid only when the feet come closer to this stance than before, so retreating and re-approaching earns nothing. It depends only on the separation of the feet, so carrying both towards the lip together earns nothing either.

A small cost is applied while the feet remain in an unsafe position, such as overhanging the lip. Descent also has its own anti-stall cost, with the same 0.8 s grace period as ascent. This period was chosen from measured rollouts, where a normal step down took well under 0.8 s and stalls lasted longer.

Descent-specific adjustments.

- Reaching down onto the next tread requires ankle pitch. On lateral faces of the pyramid, this pitch appears as roll in the world-frame foot-roll penalty. That penalty is therefore relaxed by a factor of 0.4 on descent. The relaxation is kept moderate because the network is shared with ascent, where the penalty applies at full strength.
- Base height on descent uses a separate cost that accounts for the lower landing.
- For the hardest combination, narrow treads (below 0.5 m) with tall drops (above 14 cm), forward-velocity tracking is released during the drop, letting the robot stabilize instead of being pushed

forward.

Table 4.5: Descent Reward Terms.

Component	Equation	Weight
Descending	$\dot{z}^{-}$ (new supported minima) $\cdot\, 1_{v_x^{\mathrm{cmd}}>0.1}$	5.0
Descent Foothold	$r_{\mathrm{land}}(d)$ at supported lower-tread touchdown	20.0
Descent Feet Together	$\exp\big(-\Delta x^2/0.004\big) \cdot 1_{\text{same tread}}$	20.0
Preparation	$\Delta\Phi^{+}_{\text{stance}}$	5.0
Drop Slam	c_{slam}	-5.0
Next Edge	$1_{\text{landing within 0.04 m of next edge}}$	-10.0
Stance Safety	$1_{\text{unsafe same-tread stance}}$	-2.0
Descent Dwell	$\mathrm{clip}\big((t_{\mathrm{stall}}-0.8)/1.0,\, 0,\, 1\big)$	-1.0

4.5.6 Blending Walking and Climbing

The climbing policy still imitates the reference on flat ground, so that it walks soundly if the blend misfires, but the added climbing objective degrades this gait below the expressive walking of the reference-motion policy (Section 4.4.2). Rather than asking one network to learn both behaviors, the two policies are combined at deployment. The walker receives the first 71 observation dimensions, and the climber receives all 96. Their actions are blended by a gate β computed from the heightmap:

$$\mathbf{a} = \beta\, \mathbf{a}_{\mathsf{walk}} + (1-\beta)\, \mathbf{a}_{\mathsf{climb}}, \qquad \beta^{\mathsf{inst}} = 1 - \mathrm{clip}\left(\frac{\max|\mathrm{hm}| - 0.02}{0.05 - 0.02},\, 0,\, 1\right), \tag{4.29}$$

where $\max|\mathrm{hm}|$ is the largest height difference in the heightmap, so that both steps and drops engage the climber. The gate has hysteresis. It follows β^{inst} downwards immediately, so control passes to the climber as soon as an ascending or descending step comes within view. It returns towards the walker by at most $1/40$ per step, which corresponds to 0.8 s at 50 Hz:

$$\beta_t = \begin{cases} \beta_t^{\mathsf{inst}}, & \beta_t^{\mathsf{inst}} < \beta_{t-1}, \\ \min\big(\beta_t^{\mathsf{inst}},\ \beta_{t-1} + \frac{1}{40}\big), & \text{otherwise.} \end{cases} \tag{4.30}$$

The slow return keeps control from bouncing between the two policies when the heightmap flickers, as when the last step leaves view or tread noise crosses the threshold. This is separate from the engagement w of Section 4.5.3, which acts only on the climber's training reward.

Blending in action space is well posed because both policies share an action space, a home pose and a phase clock, and because the climber was warm-started from the walker (Section 4.5.2). The two policies therefore produce compatible actions near the switch.

4.5.7 Alternating-Stride Ascent

Step-to climbing, where both feet meet on each tread, is the robust default for a robot of the T1's stature. For larger robots on low, shallow stairs, however, a person would climb step-over-step, placing each foot on alternate treads. This was introduced for the DR02 as an **additional** behavior that is used only when it is geometrically safe. On any tread where it is not, the robot falls back to the step-to gait.

Reachability. At each lift-off of the trailing foot, a stride to the tread beyond the planted lead foot is eligible only if all of the following hold:

- the tread depth is at most 0.50 m;
- each of the two steps crossed is at most 12 cm, and together they are at most 24 cm;
- the travel required from the actual launch position to the target is at most 0.90 m;
- the resulting span between the target and the lead foot is at most 0.62 m;

The conditions are evaluated on the current positions and full sole extents of both feet, so a marginal stance reverts to the step-to gait even on a narrow tread.

Rewards. A successful stride replaces two step-to milestones, the gathering touchdown and the following positioning touchdown. It is therefore rewarded at the combined weight of both, 40, for a full-sole landing 20 cm into the target tread. The following additional terms apply:

- **Launch reward (10).** Paid when the stride begins, and larger the longer both feet were planted beforehand, so that strides start from a settled stance.
- **Lead-foot foothold (20).** Rewards the ordinary step that sets up the split stance a stride starts from.
- **Miss penalty (-20).** Applied when an eligible stride touches down anywhere other than the target tread.
- **Wrong-lift penalty (-10).** Applied when the wrong foot is lifted.
- **Premature-lift penalty (-5).** Applied when the stride begins before the stance has settled.
- **Rush (-8) and overshoot (-12) penalties.** Continuous costs that penalize excess outward velocity approaching the landing and crossing the target before touchdown, discouraging lunging strides.

Curriculum. Stride opportunities would be rare within the general stair distribution. 20% of flights are therefore drawn from a dedicated distribution: narrow treads of 0.40–0.50 m, with low steps growing from 2–3.5 cm to 5.5–9 cm. The remaining 80% keep the original curriculum, so the step-to behavior on tall and wide steps is maintained.

4.6 Real Robot Deployment

During deployment, the robot's sensors and actuators are controlled through the official middleware of the robot manufacturer, **Booster SDK**[4], which acts similar to frameworks like ROS, where measurements can be accessed by subscribing to a topic, and motors can be controlled by publishing the desired joint position command.

[4]Booster SDK Repository: `https://github.com/BoosterRobotics/booster_robotics_sdk`

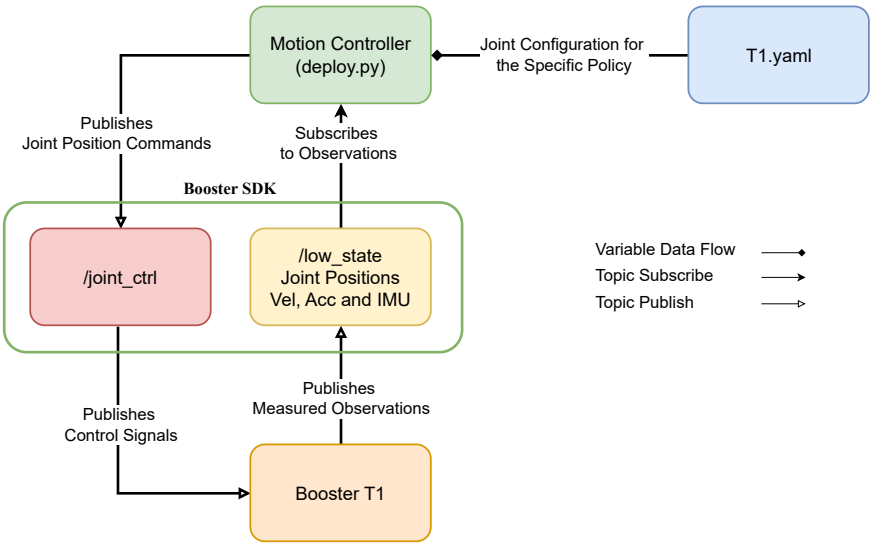


Figure 4.12: Real Robot Deployment Architecture.

Illustrated in Figure 4.12, policy deployment on the real robot involves utilizing the `/joint_ctrl` service to publish the joint positions that serve as the policy output to then be converted internally to a control signal for the robot to execute, and the `/low_state` topic to obtain live measurements of both the joint positions and velocities recorded by each joint encoder and data read from the IMU.

The deployment script runs a simultaneous threading process composed of inference calculation of the output, and publishing through the SDK. Lastly, `T1.yaml` houses a specified stiffness and damping configuration across all joints, which is the same during training, specifies the desired initial joint positions, and references the intended .pt policy model to load.

Deployment involves two separate modes: an initial one where the robot starts from its base retracted arms configuration and moves to a specified initial joint position in the yaml file, before the policy is loaded, so it is crucial that the movement made from the retracted arm configuration to the desired initial joint position is not drastic enough to cause the robot to lose balance, because as the policy is not yet loaded the robot cannot adjust himself to regain stability. This is why the initial configuration tested was one that maintained the retracted arms position, as to serve as the initial safest test.

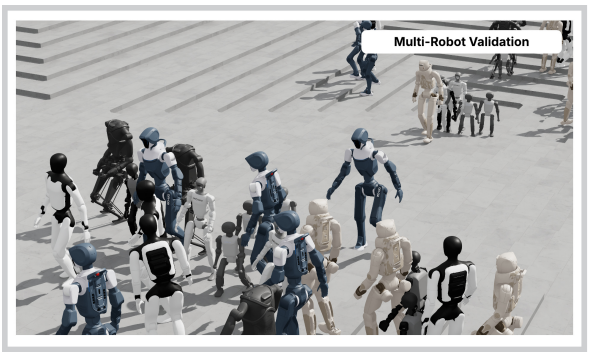
The same procedure was used for every subsequent policy, with the initial joint positions set to the home pose of each one, including the lowered arms of the whole-body and reference motion enhancing.

After entering the specified initial positions, the second mode involves loading the policy and thus being able to control the robot omnidirectionally through teleoperation.

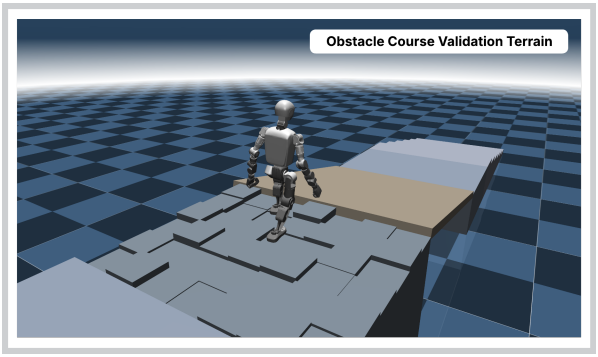
4.7 Development and Validation Tooling

Alongside the training pipelines, a considerable amount of additional tooling was developed to make development faster, more practical and more reliable, covering motion capture input, the review of retargeted motions, interactive and large-scale simulation, and the rehearsal of the deployment itself. Retargeted motions, for instance, could be reviewed many at a time, with the whole robot fleet each

performing a different clip, and manually cut to their useful segment before entering the reference library (Figure 4.13c). Each policy during validation can be driven by joystick through the same remote-control interface as the real robot, so as to be able to perform the same types of motions and more extreme stress tests before robot deployment.



(a) Multi-Copy Validation of Each Robot in Isaac Sim.



(b) Obstacle Course Validation in MuJoCo.

(c) Retargeted Motion Review Across the Multiple Robots.

Figure 4.13: Development and Validation Tooling Across Retargeting and Simulation.

Since training saves checkpoints at fixed intervals, some of these tools are mainly used to help choose among many candidate policies, exposing behaviors a training curve does not reveal, such as an unnatural motion that does not affect the reward. In Isaac Sim, up to 1024 copies of each robot on flat ground and stair pyramids reveal at a glance the rare stumbles and drifts that a few rollouts would miss (Figure 4.13a), while an obstacle course for each robot exercises the blend between walking and climbing in one continuous run, on terrain unlike what is seen in training, such as shallow building stairs, uneven tiles and ramps (Figure 4.13b).

4.8 Camera-Based Perception and Validation Robustness

During training, the heightmap of Section 4.5.2 is read from the terrain geometry, so it is exact and complete wherever it must be, namely in the reward formulation and the termination conditions. The heights the policy observes, however, are made intentionally imperfect as part of the domain randomization implemented, to approximate the conditions the real camera imposes. What training does not include is the perception pipeline itself, in which the head camera's point cloud is reconstructed into the

heightmap: as discussed in Section 3.2.5, rendering a depth image for every parallel environment would greatly increase the cost of training. The perception pipeline is therefore run in validation, and must be tested by the deployment script in Webots and by point clouds recorded on the real robot.

Heightmap randomization. The randomization reproduces how the pipeline corrupts the heightmap. Most rows are observed directly, some only partially or with cells obscured, by 30 Hz frames arriving 60 ms late and with feed drops of 60–300 ms. The rows directly under the robot's feet lie outside the camera's view, so they come from memory once the robot has walked past them. The observed heights in training carry 1 cm noise, per-frame offsets and occasional outliers, unseen cells read as zero, and errors in the robot's position estimate shift the whole map relative to the terrain by up to 5 cm horizontally and 1.5 cm vertically. Overall, the observed heights differ from the exact ones by a median of 0.8 cm, with a 95th percentile of 2.7 cm.

Perception pipeline. In the pipeline, the head is pitched down to its limit of approximately $47.6°$, and each depth frame is registered with the camera pose at its capture instant. Points on the robot's own limbs are removed, and only points on horizontal, continuous surfaces are kept, discarding the vertical faces and edges of the steps. The points are accumulated in a world-frame elevation map of 4 cm cells, in which a cell with too few or too widely spread points is left empty rather than averaged, so that no cell straddling a step edge produces an intermediate height. The rows the camera cannot see are read from this map once walked past, and the 25 heights are sampled from it in the heading frame, relative to the ground under the base, as in training. Since the robot's interface provides only a planar position and heading, the base is localized from the IMU and leg odometry, with its vertical drift corrected by matching successive frames. The same estimate gives each foot's distance to the next step, which drives the climbing context of Section 4.5.3.

Deployment script. As it was done for the previous policies, the deployment script was then validated in Webots, the only environment in which it runs through the Booster SDK exactly as on the real robot (Section 4.1.1). This step is more important here, because much more of the script had to change: the heightmap observations computed online, the climbing context and slower gait clock near a step, and the switching between policies through the blending gate. These had already been exercised in the validation simulators, but had to be confirmed to behave the same way in the actual script, for which the Webots scene was extended with staircases and a head camera.

Recorded point clouds. A simulated camera only reproduces the imperfections that were modeled, while testing against the unmodeled ones directly on a staircase would be dangerous. The point clouds of the real head camera were therefore recorded as ROS bags along a walking trajectory of the real robot and replayed through the same pipeline in simulation, testing the reconstruction and the policy's response on real camera data without exposing the robot to the risk.

5

Experimental Evaluation and Results

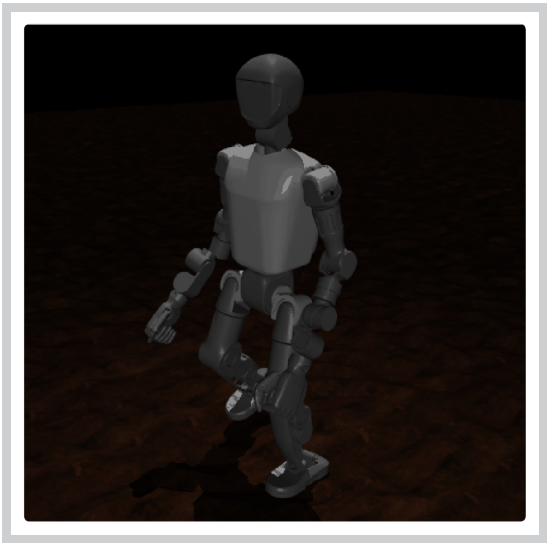
5.1 Legged Locomotion

The first objective establishes the developed gait on the real Booster T1 while retaining a fixed upper body. The retracted-arm configuration provides the direct comparison with the original Booster controller. The lowered-arm configuration then examines whether the main gait can be retained while changing the posture and distribution of the upper-body mass.

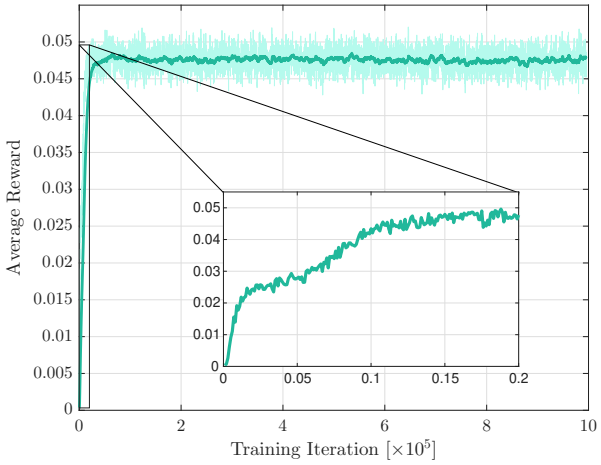
5.1.1 Training Results

For this first objective, the training results performed with specified parameters detailed in Appendix A.1 for a fixed retracted arm configuration, a configuration which was chosen due to being an exact match on the official Booster motion model, are shown in Figure 5.1.

Total training time took approximately 12 hours on a single RTX 5080 GPU, for a total of one million iterations. The averaged reward across all parallel environments per global step stabilized at about 8 $\%$ of the total training sequence (around the 8×10^4th iteration), and from then on oscillated around the established plateau. The employed reward function detailed in Figure 4.8b alludes to the quick rise evidenced in these initial iterations as the byproduct of the robot no longer easily falling, or colliding with itself, as these termination conditions hold the biggest impact on measured rewards due to trajectories being cut early, and thus by being in a state where it can remain standing, already heavily contributes



(a) Training Environment for the Adapted Robot Model.

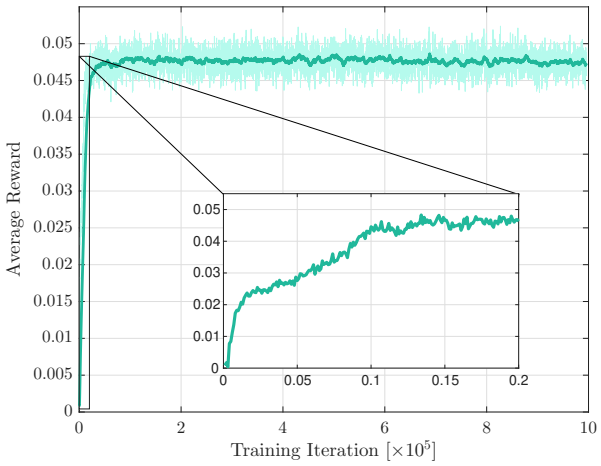


(b) Averaged Reward Obtained During Training.

Figure 5.1: Main Training Results for the Retracted Arms Configuration.

to a high average reward. During the initial subsequent iterations after stabilizing, several undesirable effects, such as endless micro-movements when standing still and slight instabilities when decelerating to a stop, manifested themselves in the extracted model. These get somewhat ironed out, evidenced by the continuous decay of the loss functions of the estimated networks shown in Appendix A.2, however due to most of these nuanced behaviors either not holding enough weight to be prioritized, or not being possible to account for in the reward function, remnants of such inconsistencies still can sometimes randomly appear from one iteration to the other. This is why training was run for an extended period after the supposed stabilization: as described in Section 4.7, it saves a checkpoint at fixed intervals, and the deployed policy was selected among these as the one showing no such motions in validation.

(a) Training Environment for the Adapted Robot Model.



(b) Averaged Reward Obtained During Training.

Figure 5.2: Main Training Results for the Rested Arms Configuration.

The other section of results shown in Figure 5.2 relate to the same exact training environment but with modifications made to the robot model to rest its arms nearer to the center of mass. It was hypothesized that this improved weight distribution would show a faster convergence during the initial steps where initial stability is the first learned characteristic, but the results show to be very similar in behavior between both configurations, despite showing higher reward values at each instance during the ascent, even if slightly. Nevertheless, the training environment shows transferability across multiple arm configurations.

5.1.2 Validation Results

Each periodically saved policy of both configurations is then evaluated in the joystick-controlled MuJoCo validation environment described in Sections 4.1.1 and 4.7, adapted from the official framework of the manufacturers [73] to our robot model.

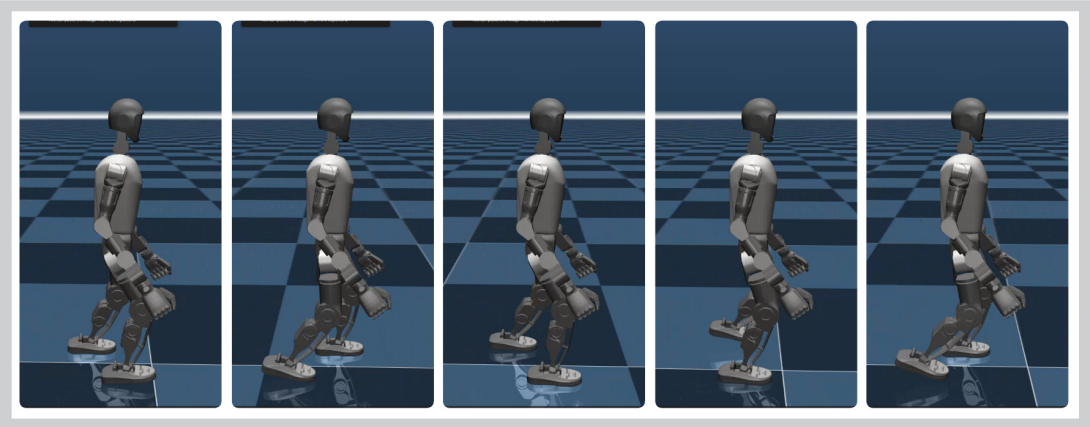


Figure 5.3: Validation Results for the Retracted Arms Configuration in MuJoCo (MuJoCo) (Isaac).

This environment was how the qualitative selection of the best model to deploy was made, as it allowed for performing manual stress tests on the controller to ensure complete stability when transferring to real deployment.

Figures 5.3 and 5.4 respectively showcase the models saved at the 9.5×10^5 th and 8×10^5 th iteration of their training environments, and were specifically selected on the basis of exhibiting no unusual behaviors and never losing their balance across the many employed control experiments.

Both policies were also run in Isaac Sim and Webots, for the purposes described in Section 4.1.1, with the MuJoCo results remaining the closest approximation to the robot's behavior in real deployment.

5.1.3 Real Robot Deployment Results

Figure 5.5 showcases an experimental trial run of the previously validated policy in Figure 5.3 on the real robot, exhibiting the various types of velocity commands it is robust to. An exact same run controlling the official Booster model was also recorded, to provide a side-by-side comparison of improvements in motion fluidity. Overall, the exhibited behavior managed to be extremely similar to the conducted tests in simulation, maintaining adequate stability both when idle and in motion.

Bipedal walking is a periodic behavior that can be analyzed at a joint level to observe characteristic

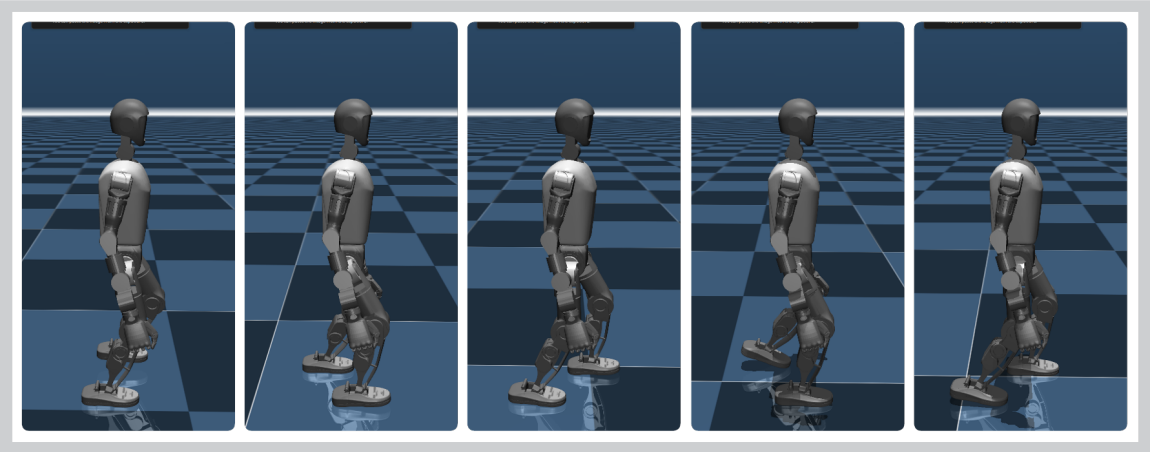
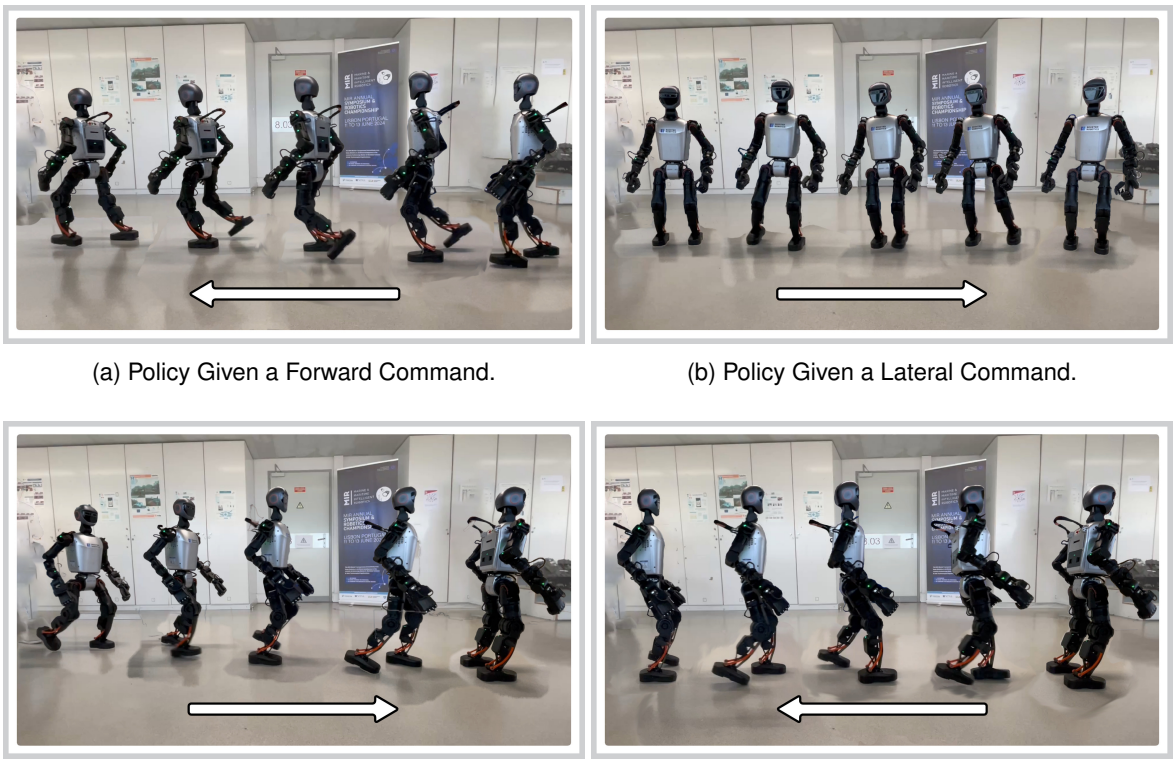


Figure 5.4: Validation Results for the Rested Arms Configuration in MuJoCo (MuJoCo) (Isaac).

patterns that may not be visible in live deployment demonstrations. For strictly walking forward, leg joints that infer pitch are the only actuators that present periodic describable behaviors, as other joints involving roll or yaw act as stabilizers during that motion, trying to remain very close to a set position at all times to avoid instability. In Figure 5.6, angular positions in radians of three joints for each leg are presented, during a forward movement of around eleven total steps. The deployed policy under forward, lateral and backward commands is shown in Figure 5.5.



(a) Policy Given a Forward Command.

(b) Policy Given a Lateral Command.

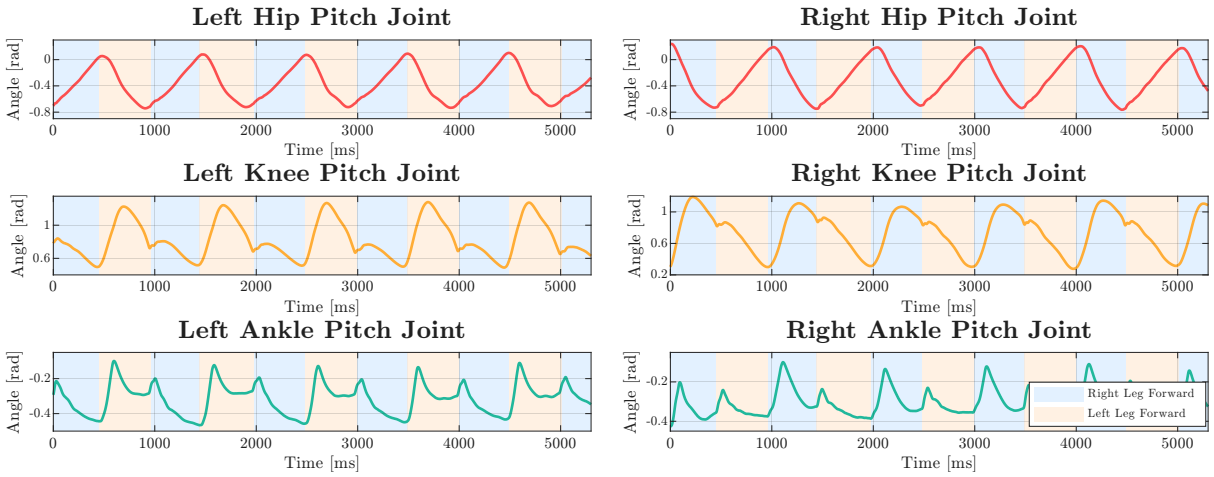
(c) Policy Given a Forward Command.

(d) Policy Given a Backwards Command.

Figure 5.5: Deployment Results of the Retracted Arms Configuration Policy (Video).

The hip pitch joint acts as the leading driver of the movement, from which the knee and ankle pitch joints act upon in the hierarchical leg joint structure. It spans the widest range of the three (47–54°) and,

through the leg's kinematics, produces about two thirds (66–68 %) of the ankle's horizontal displacement relative to the hip, with the knee producing the rest. Its behavior can be separated into two instances. When the corresponding leg is leading, the joint departs from its highest angular value and steadily lowers over about 420 ms (42 % of the stride), accelerating to a peak of 3.6–3.7 rad/s about a third of the way into the swing, detailed in Appendix A.5, and then decelerating towards the end of the step, where a final small oscillation marks the leading leg's contact with the ground. When the other leg is leading, the joint behaves in the opposite way, rising towards higher angular values to maintain the balance of the step, at more controlled velocities owing to the stabilization of ground contact, peaking at only 2.0–2.2 rad/s. The signal's periodicity is very apparent, resembling a sinusoidal wave with a gait frequency of 1 Hz (stride period of 1000 ± 15 ms). The two legs are not perfectly symmetrical, most notably in the actuator range interval, with the right hip spanning $7°$ more than the left ($53.9°$ against $46.9°$), one of the expected outcomes of RL-based methods. The recorded signal nonetheless remains smooth and highly repeatable, its strides deviating from one another by under 2 % of the hip's range, showing both the stability and naturalness of the movement.



(a) Left Leg Joint Angles in a Forward Walk Cycle.

(b) Right Leg Joint Angles in a Forward Walk Cycle.

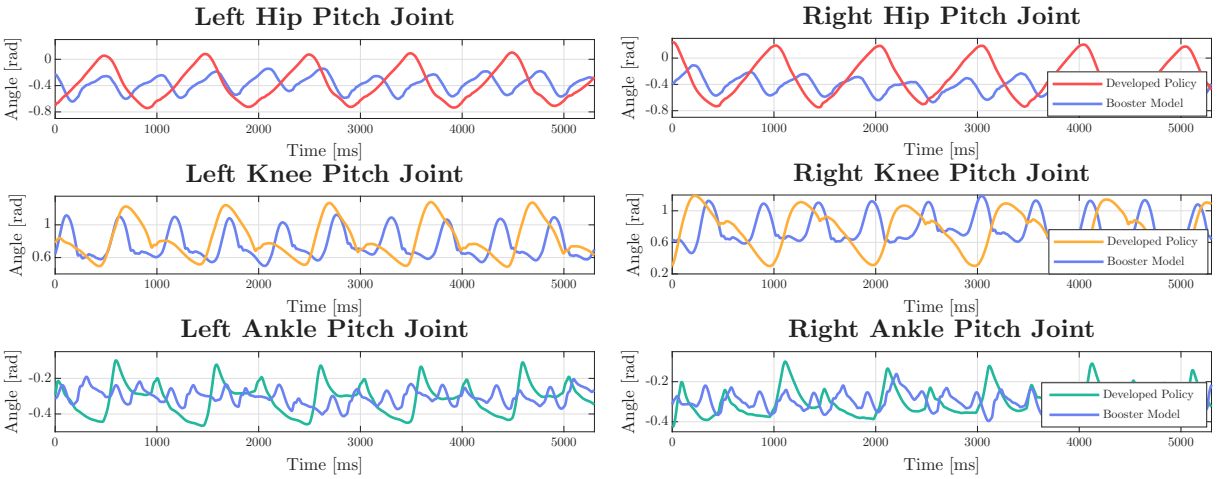
Figure 5.6: Recorded Joint Angular Positions during Deployment.

The knee pitch joint mostly follows the drive started by the hip pitch when that leg is leading, flexing to keep the lower leg from hitting the ground during the swing and reaching its peak of 64–72° at about the half point of the step (48–56 % of the swing, about 220 ms after the hip starts it). It then extends to keep the lower leg perpendicular to the ground, in a stable position for the final contact, where the same slight oscillation appears. When the other leg is leading, the knee mostly aids in generating momentum for the swing, slightly shifting the center of mass towards the forward movement. As opposed to the hip, the lower leg pitch joints show larger differences between legs: the average left and right strides correlate at 0.996 for the hip, but only at 0.92 for the knee and 0.73 for the ankle, whose range differs by 34 % ($19.2°$ against $13.6°$). These joints have a lesser impact on the overall movement, and since their deviations never detracted from the velocity tracking or stability goals, they never warranted correction during training, being essentially imperceptible when the robot is moving live.

Lastly, the ankle pitch joint, similarly to the knee, moves the foot to keep it from hitting the ground

during the initial swing and converges to be parallel to the ground at its end, right before contact. With the smallest range of the three (14–19°), it mostly helps shift the total weight forward when the other leg is leading, like the knee.

Superimposing the same movement on the official Booster motion controller, as seen in Figure 5.7, firstly shows a significantly wider range of movement in the developed policy, 2.1 times larger at the hip ($47.5°$ against $22.5°$ on average), 1.4 times at the knee and 1.8 times at the ankle, whilst every pitch joint stays at least $15°$ away from its actuator limits (listed in Appendix A.4), leading to a more natural looking walking gait. It also shows clear improvements in robustness, as the Booster controller's step cycles differ noticeably from one another. Taking the deviation between time-normalized strides relative to each joint's range, the developed policy reduces it from 12 % to 7.5 % at the hip, from 10 % to 7.7 % at the knee and from 21 % to 12 % at the ankle, while the variation in stride duration drops from 3.6 % to 2.4 %. The policy also walks at 1 Hz against the Booster's 1.9 Hz, taking fewer and longer steps. Graphically, this matches what is seen in a video comparison between the two, where each joint of the Booster motion model moves in a more robotic and wasteful manner, with curves that change from one step to the next. This comparison is quantified over a longer trial in Section 5.2, where the torque, power and smoothness of the whole-body policy are measured against the Booster controller.



(a) Left Leg Joint Angles in a Forward Walk Cycle.

(b) Right Leg Joint Angles in a Forward Walk Cycle.

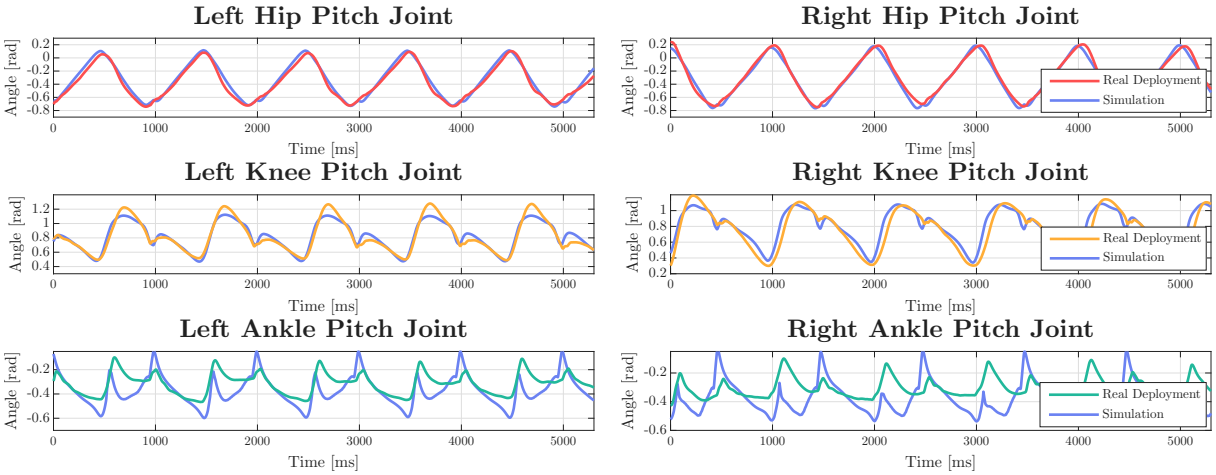
Figure 5.7: Developed Locomotion Policy *vs* Original Booster Model Angular Positions.

Finally, the developed policy shows a lower gait frequency, manifested in the signal period of each showcased joint, leading to a more natural-looking movement and overall a less noisy gait.

In Figure 5.8 the behavioral differences between the policy in simulation and the real robot are addressed, which ideally would correspond to being identical to what was recorded during deployment. Naturally, no robotics simulator can perfectly model real world physics and environment conditions, and despite visually being very difficult to distinguish by watching both in motion, some differences are visible when looking at both behaviors at the joint level.

The curve shape is mostly maintained, with both sharing the same gait frequency, and the hip and knee pitch joints closely following their simulated trajectories, while the ankle pitch, being the joint most affected by the contact with the ground, shows a visibly different pattern. The hip ranges match within

$2°$, while on the robot the knee works about 8–$10°$ more and the ankle about $10°$ less than in simulation.



(a) Left Leg Joint Angular Positions in a Forward Walk Cycle.

(b) Right Leg Joint Angular Positions in a Forward Walk Cycle.

Figure 5.8: Sim2Real Gap Positions.

For the policy trained under the rested arms configuration, deploying it in the real robot required careful planning around performing the lowering of the arms movement from its retracted position, in a way that wouldn't cause the robot to lose its balance or damage any actuator. This configuration, due to the asymmetric morphology of the robot causing the arms to be very near its legs, made it susceptible to minor collisions during heavy movement, most notable from unintended arms swings when moving laterally. To mitigate this, a careful rundown of the best stiffness and damping configuration for each arm joint was made, varying for the initial lowering of the arms movement, where the arm joints that would make the most movement should move slower, not to quickly lose balance, and the joints that could cause it to swing and possibly hit the legs during the motion should have high damping values to remove any oscillation. When loading the policy, ensuring that the arm joints have a sufficiently high damping to not swing enough to hit any other joint when in motion, most notably when moving laterally, was also carefully considered and tested. This was a relatively practical endeavor by being able to validate the configuration in simulation first, and then being transferred near optimally to the real robot.

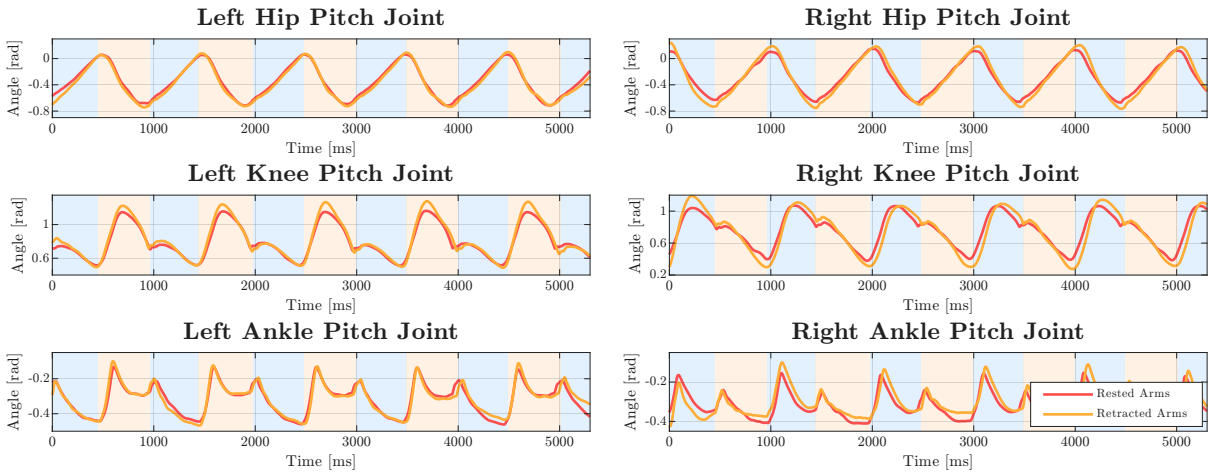
By mitigating all these concerns, a similar experimental run was also employed for this policy, as shown in Figure 5.9, where it produced a similar walking behavior to the test run for the other arm configuration but, as hypothesized, this configuration yielded a better weight distribution in motion and therefore looked more stable and poised during abrupt shifts in velocity commands, such as fully stopping when moving backwards.

And most notably, by the direct comparison of the recorded leg joint positions during a forward movement, an easing, even if slight, can be denoted on the hip, knee and ankle pitch motors, meaning that because the disproportionate arms are no longer in an awkward front position, and rather resting along near the ground, even if still fixed in place, does seem to reduce the load that the legs have to take. Measured on the recorded trajectories (Figure 5.10), the rested configuration reduced the range of motion of these joints by about 13 % on average, with the largest easing at the knee, between 17 and 23 %.

This effect is examined further in Section 5.2, where the arms are free to move with the gait, and

Figure 5.9: Deployment Results of the Rested Arms Configuration Policy (Video).

their swing can offset the momentum of the legs rather than only shift the robot's weight.



(a) Left Leg Joint Angles in a Forward Walk Cycle.

(b) Right Leg Joint Angles in a Forward Walk Cycle.

Figure 5.10: Recorded Joint Position Differences during Deployment for Both Arm Configurations.

5.2 Whole-Body Locomotion

As theorized in the methodology, the adapted training configuration detailed in Section 4.3.1 converges towards a policy (Figure 5.11) that moves its upper body along with the motion, but conservatively. This is noticeable in the validation results across both simulators: the desired anti-phase arm swing relative to each stepping leg is present, and although it shows a clear difference from the fixed upper-body policy in Figure 5.9, it is much fainter than the average human walking.

The trained motion transfers directly onto the real robot. When comparing the real deployment against an average of both Isaac and MuJoCo under the same command, the cycle-averaged joint trajectories of the shared joints differ by an RMSE of only $1.8°$, and the mean posture of every joint agrees within $1.5°$. The leg joints transfer almost exactly, with a correlation of 0.99 and an RMSE of 6% of their range of motion for both hip and knee pitch. The arms show a similarly close match: shoulder

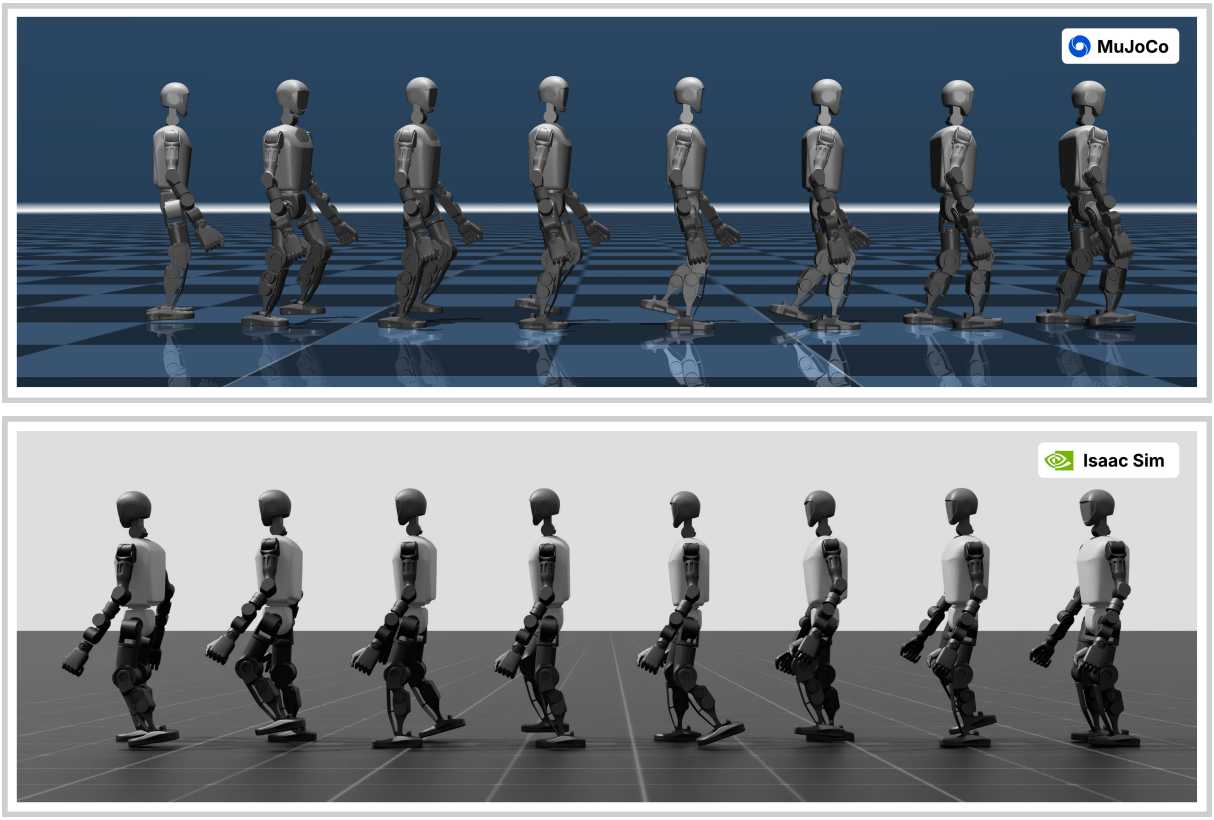


Figure 5.11: Validation Results for the Whole-Body Policy across Both Simulators (MuJoCo) (Isaac).

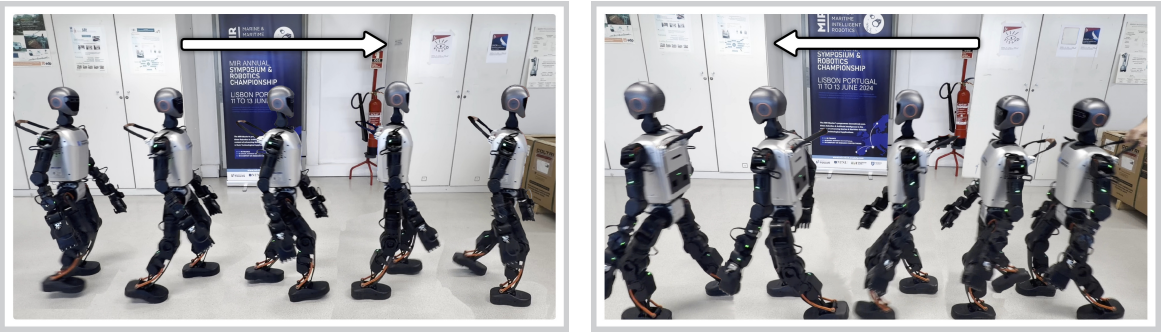
and elbow trajectories correlate at 0.95–0.99, along with other properties that are exhibited in Figure A.12 on the Appendix. The aggregate effort also carries over, as the joint torque RMS and positive mechanical power differ by 9% and 10%, respectively.

Most notably, the whole-body policy exhibits improved stability during rapid deceleration and stopping. Unlike the fixed upper-body configuration, the learned policy allows the arms to move in coordination with the legs, acting as counterbalances to help regulate the robot's center of mass before reaching a standing posture. In contrast, the fixed-arm policy must maintain its prescribed upper-body position throughout deceleration, restricting such compensatory movements. This contributes to smoother and more controlled stops, particularly relevant for tasks requiring precise positioning, such as small adjustments prior to manipulation, where bipedal locomotion inherently complicates accurate final positioning.

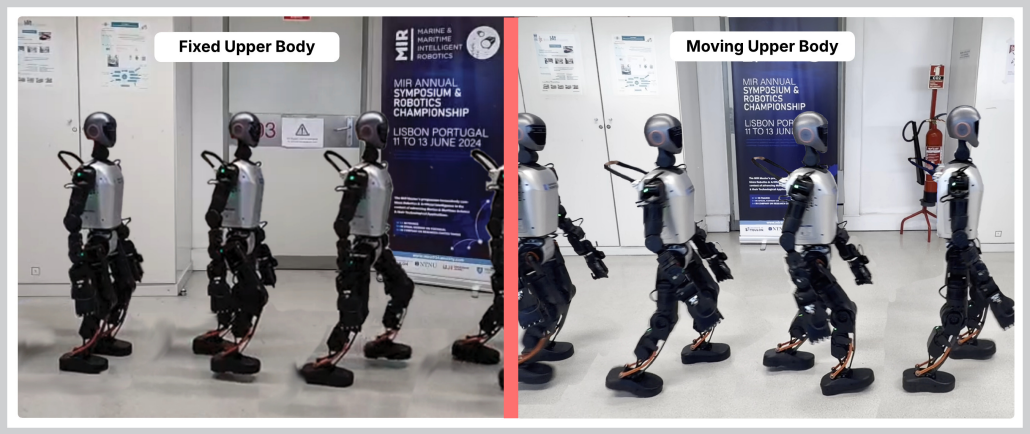
The recorded joint trajectories (Figure 5.13) reveal some left-right asymmetry, also present in the first objective policies but more pronounced in the now-active upper-body joints. Since symmetry is not explicitly enforced during training, the policy converges towards different motion amplitudes on each side. Any resulting asymmetry may propagate between the upper and lower body as their movements become coordinated, persisting even if barely perceptible during deployment.

Both characteristics motivate introducing reference motion guidance to the policy. While the developed policy exhibits desirable upper-body behavior at low speeds, joint symmetry remains to be enforced. This can be achieved through deliberately symmetric walking references or preprocessing techniques that mirror movements between the left and right sides.

The comparison with the rested-arm policy in Figure 5.13 also shows that introducing upper-body



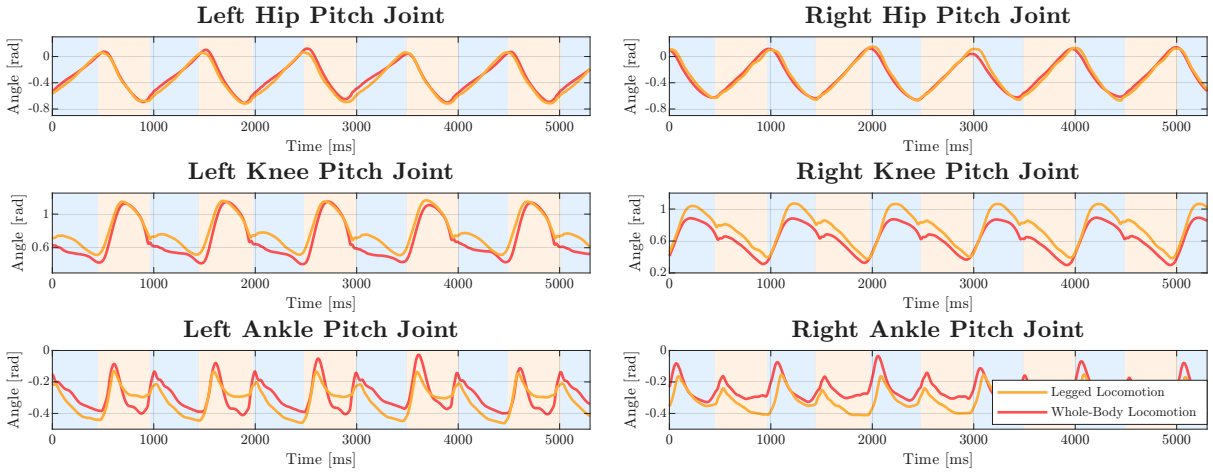
(a) Policy Given Forward Commands.



(b) Direct Comparison of the Policies of Objectives 1 (Rested Arms) and 2 (Whole-Body).

Figure 5.12: Deployment Results of the Whole-Body Policy (Video).

movement preserves the main stepping pattern while changing the lower-body configuration within it. The hip-pitch trajectories remain closely aligned, whereas clearer differences appear at the knees and ankles. Knee flexion is reduced over substantial portions of the cycle, particularly on the right leg, while the ankle-pitch trajectories shift without losing their periodic structure.



(a) Left Leg Joint Angles in a Forward Walk Cycle.

(b) Right Leg Joint Angles in a Forward Walk Cycle.

Figure 5.13: Recorded Joint Angular Positions during Deployment of Objectives 1 (Rested Arms) and 2 (Whole-Body).

Together with the direct deployment comparison in Figure 5.12b, these results show that the progression to whole-body locomotion is not simply the addition of arm motion to an otherwise unchanged controller. Rather, it also manifests itself in adjustments throughout the lower body. The mechanical consequences of these changes are examined in the longer trial below.

To evaluate the whole-body controller beyond the short walking sequences, a longer deployment trial was performed over three complete laps, as illustrated in Figure 5.14. The same route was also completed using the original Booster controller, allowing the joint measurements to be compared over sustained locomotion rather than only a selected gait cycle. This experiment examines whether the additional upper-body movement is accompanied by an increase in overall mechanical demand, or whether its cost is offset by changes in the lower-body effort.

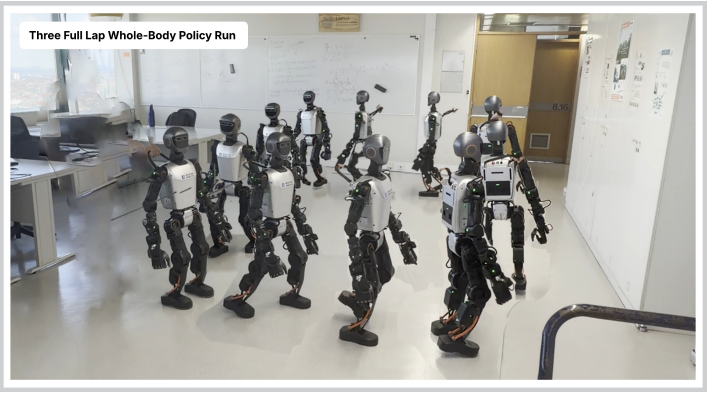
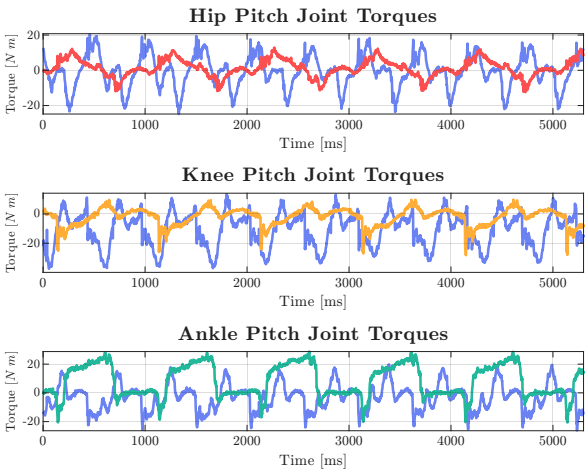


Figure 5.14: Robustness Trial Run Along Three Full Laps with the Whole-Body Policy (Video).

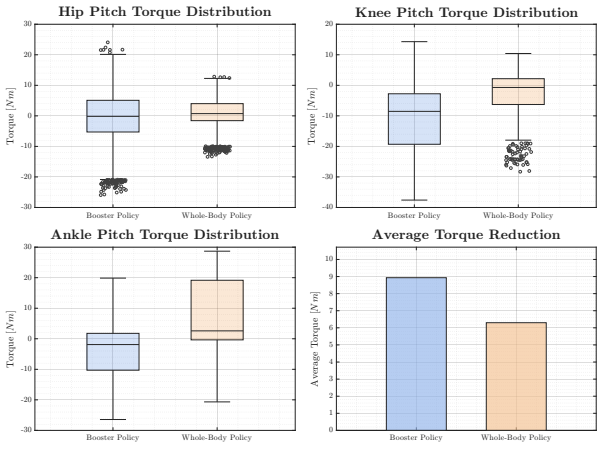
Table 5.1: Aggregated Metrics across All Joints: Booster Policy vs Whole-Body.

Metric	Booster	Whole-Body	Red. [%]
Torque (motor strain across all joints)			
Mean $\lvert\tau\rvert$ $[Nm]$	106.3	82.4	22.5
RMS τ $[Nm]$	132.7	111.0	16.4
Peak $\lvert\tau\rvert$ $[Nm]$	345.0	306.8	11.1
Power (energy expenditure across all joints)			
Signed $\overline{\tau\dot{q}}$ $[W]$	26.2	16.4	37.4
$\overline{\lvert\tau\dot{q}\rvert}$ $[W]$	91.3	53.7	41.2
Positive power $[W]$	58.8	35.1	40.3
RMS(τ)·RMS($\dot{q}$) $[W]$	165.4	112.6	31.9
Smoothness			
RMS $\dot{\tau}$ $[10^3\,Nm/s]$	10.2	7.2	29.5
RMS jerk $\dddot{q}$ $[10^3\,rad/s^3]$	806.3	156.0	80.7

Figure 5.15a shows a short excerpt of the recorded hip, knee and ankle torques, while Figure 5.15b summarizes their distributions over the entire run. The whole-body policy exhibits less pronounced torque fluctuations at the hip and knee, particularly the repeated negative knee-torque peaks present in the Booster controller.



(a) Leg Joint Torques During an Excerpt of the Trial Laps.



(b) Leg Joint Torque Distribution for the Entire Run and Average Torque Reduction Across All Joints.

Figure 5.15: Trial Run Torque Values and Distribution Analysis Compared with the Booster Policy (Blue).

The narrower hip and knee distributions indicate that this difference is retained across the run rather than being restricted to the selected excerpt. The ankle behaves differently, with more pronounced positive-torque intervals in the whole-body policy. The improvement therefore reflects an overall reduction in joint effort, although the benefits are not uniform across all actuators.

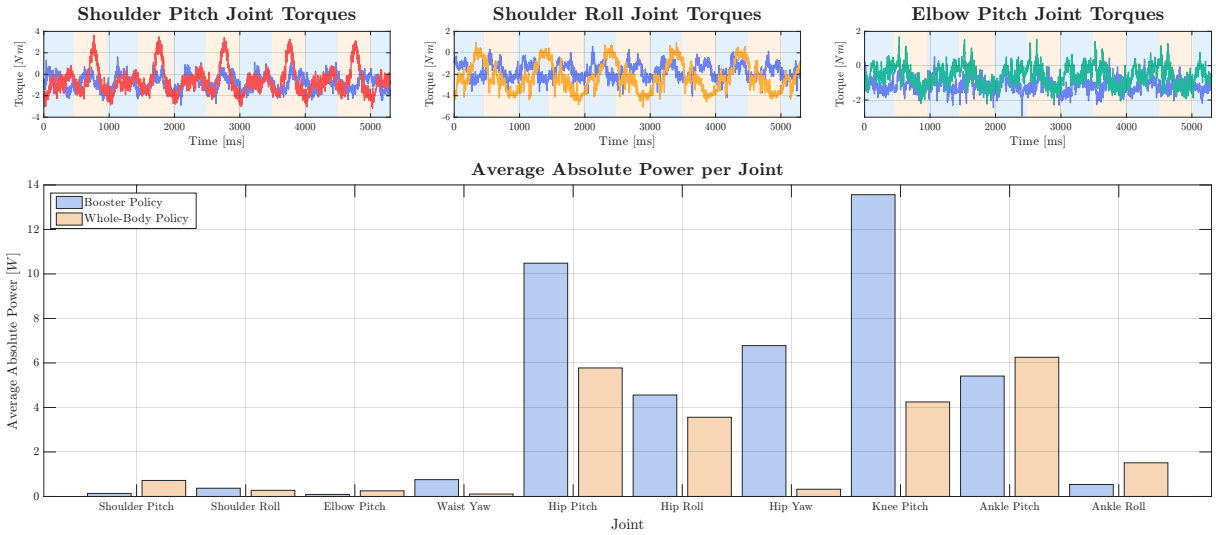


Figure 5.16: Trial Run Whole-Body Torque and Power Reduction Compared to the Booster Policy.

The upper-body traces in Figure 5.16 show the complementary effect: the shoulder-pitch joint develops distinct, recurring torque peaks respective of the added arm motion, even though its angular excursion remains conservative. This still manifests itself in a higher consumption on the upper body joints, but the corresponding power comparison shows that the additional shoulder-pitch and elbow-pitch demand is small relative to the reductions at the hip-pitch, hip-yaw and knee-pitch joints. Although ankle power increases, the lower-body savings outweigh the additional demand elsewhere. Active upper-body motion is therefore compatible with a reduction in the overall mechanical cost of the deployed controller.

This net improvement is quantified in Table 5.1. Mean absolute mechanical power decreases from 91.3 W to 53.7 W, a reduction of 41.2%, while positive mechanical power decreases from 58.8 W to 35.1 W, a reduction of 40.3%. The reported aggregate torque RMS also falls from 132.7 Nm to 111.0 Nm, corresponding to a 16.4% reduction. Thus, the developed controller achieves the additional movement complexity of whole-body locomotion with lower measured mechanical demand than the original Booster policy.

The improvements extend to motion smoothness. Table 5.1 reports a 29.5% reduction in the aggregate torque-rate RMS and an 80.7% reduction in angular-jerk RMS, providing quantitative support for the smoother movement observed during deployment. Lower mechanical demand is therefore accompanied by less abrupt joint motion, rather than obtained through a more irregular gait. A smaller decrease in the reported battery charge was also observed over the three-lap route, providing a practical observation consistent with the mechanical-power comparison.

Under the energy-efficient training configuration, Figure 5.17 compares the conventional whole-body policy with a separate policy trained to explicitly minimize cost of transport. Despite being free to adopt larger arm movements, the previous whole-body policy had already converged towards a conservative

swing, which the energy-efficient policy similarly reproduces. This suggests that the restrained arm motion represents an economical solution for the T1's morphology, whose long and relatively heavy arms can contribute to reducing the effort required from the legs without excessive upper-body movement. However, the stronger emphasis on energy efficiency also produces a more flexed leg posture, lowering the arms closer to the ground and limiting their influence on the robot's balance while preserving their compensatory swing. Thus, while both policies converge towards similar arm behavior, the cost-focused policy achieves it through a distinctly different whole-body posture.

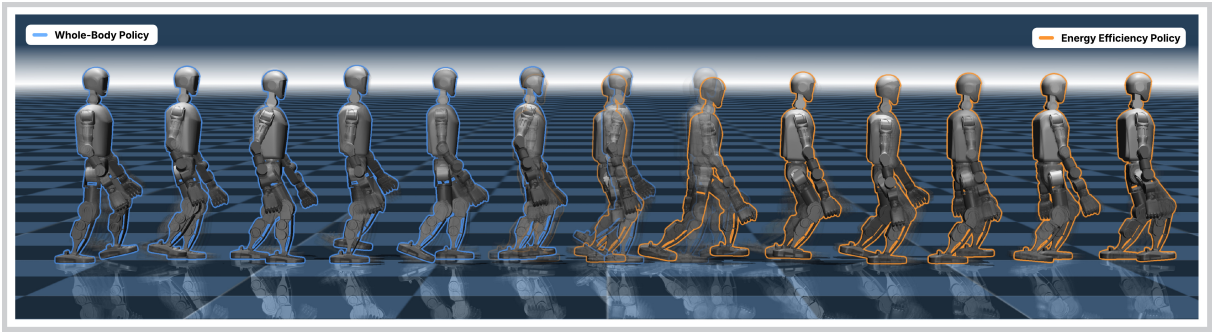


Figure 5.17: Whole-Body (Blue) and Energy Efficiency (Orange) Policies (MuJoCo) (Isaac).

The cross-robot comparison in Figure 5.18 and Table 5.2, also illustrated in Figures A.7 to A.11 of the Appendix, shows that the economical arm swing is dictated by each robot's morphology rather than converging towards a universal, human-like pattern. Among the evaluated platforms, the T1 is the furthest from human proportions. Its arms, extended for manipulation, reach 1.13 times its leg length and hang below its knees, whereas a human arm reaches about 0.83 times the leg length, with the fingertips at mid-thigh [74]. The G1 and A3 match the human ratio almost exactly, and the K1 is close to it. Each T1 arm also carries 10% of the body mass, twice the share of a human arm, which makes it the hardest of all the arms to swing relative to the robot's size.

Under the same cost-focused objective, the robots with more human-like proportions swing their arms considerably more: the shoulder-pitch excursion is 41.0° for the K1, 24.0° for the G1 and 21.3° for the A3, against 16.4° for the T1. The GR3 is the exception, with only 11.0° despite having proportionally the shortest arms. As the tallest and heaviest of the robots, with large mass and bulk in each limb, any large movement is costly for it. The two robots furthest from the human arm-to-leg ratio, deviating in opposite directions, are therefore also the two that depart most from a human-like swing, each in its own way. The T1's long arms act as levers: it rotates its shoulder little, yet still displaces about as much arm mass as the G1 and K1. The GR3's short, bulky arms instead barely move, with both the smallest shoulder excursion and the smallest arm displacement of the group. The robots closest to human proportions fall in between, with clearly visible arm swings.

The T1–K1 comparison illustrates this well. Although the T1's shoulder excursion is only 40% of the K1's, their mass-weighted arm swings are nearly identical (12.9×10^{-3} and 12.6×10^{-3}), as are their hand travels (12.4 cm and 12.1 cm). With arms reaching 1.13 times its leg length against the K1's 0.91, the T1 moves a comparable amount of arm mass with far less shoulder rotation. Its restrained-looking arm motion is therefore an economical use of the upper body for its morphology, rather than an absence

of it.

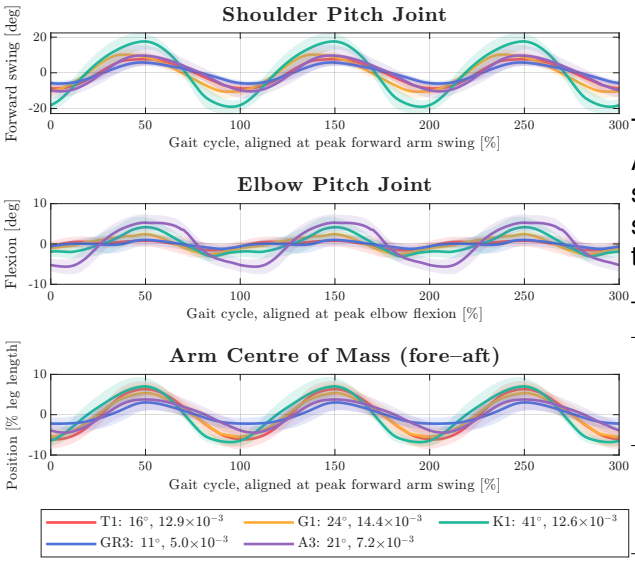


Figure 5.18: Arm Swing of the Cost-Focused Policies over One Gait Cycle from Walking Straight at a Froude-matched Speed ($0.46\,\mathrm{m\,s^{-1}}$; up to six seeds, $11\,\mathrm{s}$ measured after $7\,\mathrm{s}$ of settling). From top to bottom: shoulder pitch, elbow pitch and fore–aft arm center-of-mass position (% of leg length), mean posture removed; mean ± 1 SD.

Table 5.2: Arm Morphology and Swing per Robot. Arm inertia: shoulder moment of inertia of the straight arm over $m_{\mathrm{body}} L_{\mathrm{leg}}^2$. Mass-weighted swing: $(m_{\mathrm{arm}}/m_{\mathrm{body}})(\Delta x_{\mathrm{arm}}/L_{\mathrm{leg}})$, with Δx_{arm} the fore–aft travel of the arm's center of mass.

Metric	T1	G1	K1	GR3	A3
Morphology					
Leg length $[m]$	0.52	0.65	0.45	0.91	0.88
Arm reach / leg length [–]	1.13	0.82	0.91	0.69	0.82
Arm inertia $[10^{-3}]$	21.6	12.5	7.2	10.0	9.5
Joint swing					
Speed $[m/s]$	0.46	0.53	0.42	0.60	0.58
Shoulder pitch range [°]	16.4	24.0	41.0	11.0	21.3
Elbow range [°]	3.7	6.6	7.7	3.2	8.7
Arm displacement					
Hand travel $[cm]$	12.4	15.6	12.1	10.6	15.7
Hand travel / leg [%]	23.9	24.1	27.1	11.7	17.9
Arm CoM travel / leg [%]	12.7	13.7	15.7	5.0	8.8
Mass-weighted swing $[10^{-3}]$	12.9	14.4	12.6	5.0	7.2

Posture separates locomotion economy from human-like movement further. In the same rollouts, the K1, G1 and A3 kept upright configurations, whereas the T1 cost-focused policy adopted the flexed stance described above. Taken together, the results expose a tension at the core of this work. Its aim is more natural, human-like locomotion, but for a robot whose upper body is as far from human proportions as the T1's, the most economical way to walk is neither upright nor expressive in its arm swing. The robots with more human-like arms move closer to a human-like swing under the same objective, which suggests that the T1's restraint comes from its morphology rather than from a shortcoming of the cost-focused objective.

These findings motivate reference-motion guidance as an additional design objective rather than a replacement for the gains already achieved. The whole-body controller has established transferable arm coordination, smoother movement and reduced mechanical demand. Since efficiency alone will not lead the T1 towards a human-like gait, the desired characteristics have to be specified explicitly. Reference motion keeps the useful restrained behavior at low speed while describing bilateral symmetry, upright posture and how arm–leg coordination evolves with commanded velocity. Its purpose is not simply to increase arm excursion, but to specify those characteristics of the gait that task performance and locomotion cost do not determine on their own.

5.3 Reference Motion Based Locomotion

As described in the methodology, a sequence of in-house whole-body walking captures was recorded, providing stride samples at multiple velocities from which to construct the library of reference move-

ments. Combined with the proposed retargeting modifications and their robot-specific tuning, these recordings produced the actual motion sequences to be used as references during training, as illustrated in Figure 5.19.

Although the robots reproduce the same captured movement, differences in stature, limb proportions and degrees of freedom lead to different joint configurations. The effect is strongest for the T1 and K1, which are considerably smaller than the $1.9\,\mathrm{m}$ recorded subject and cannot reproduce some of the original movements exactly, whereas robots closer to the subject in proportions, or with more degrees of freedom, follow the motion more faithfully.

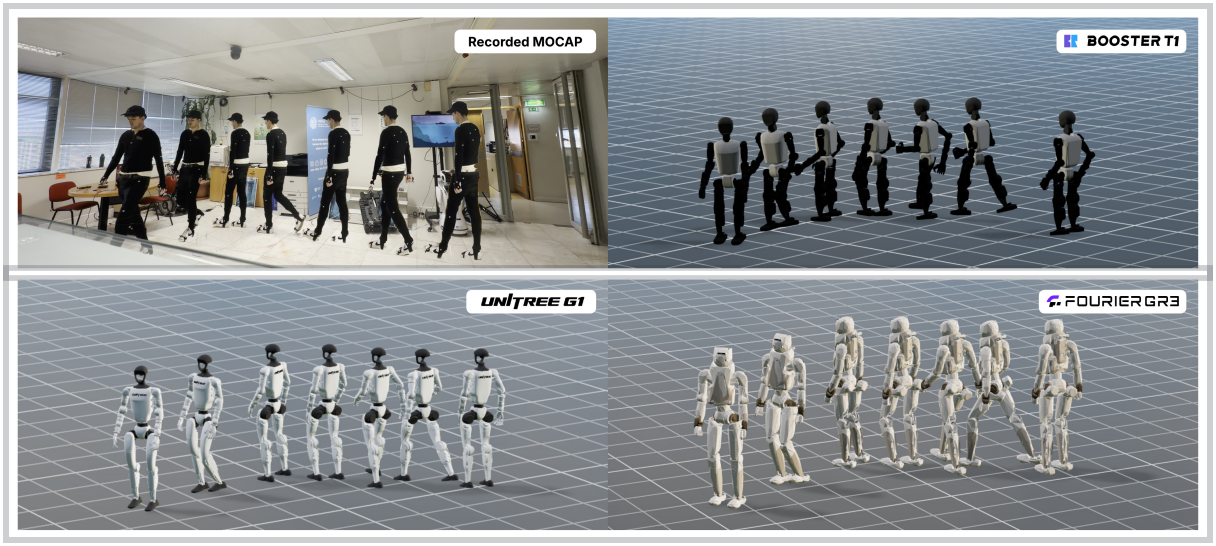


Figure 5.19: Excerpt of the Recorded MOCAP Motions and Their Respective Retargeting (Video).

In every case, the recorded coordination between the stepping legs and the swinging arms is preserved, so that the recorded database yields a diverse set of robot-specific references. Each reference describes a complete walking cycle at a given speed, and together the references span the range of velocity commands used in training. The desired gait is therefore depicted directly at each of these speeds, removing the need to encode each gait characteristic through a separate handcrafted reward.

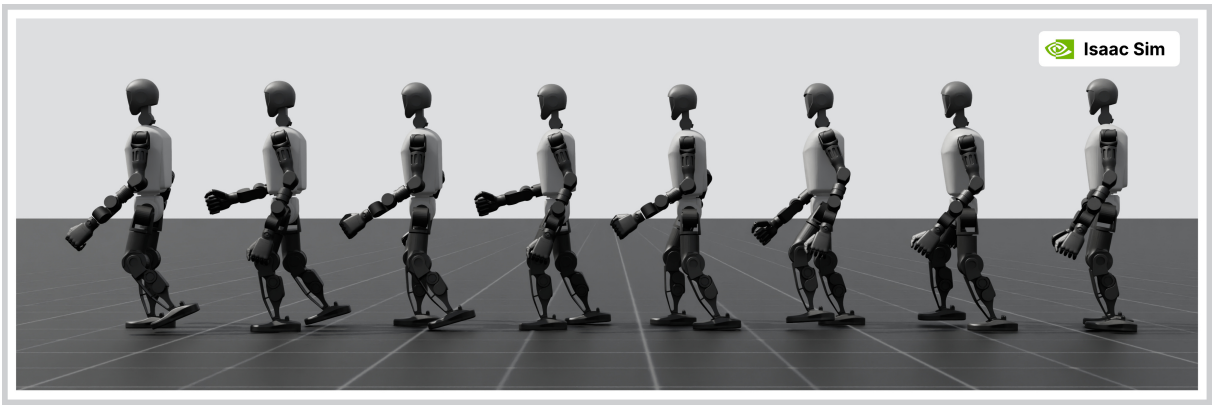


Figure 5.20: Validation Results for the Reference Motion Policy in Isaac Sim (MuJoCo) (Isaac).

The reference library derived for each robot was subsequently incorporated into the developed whole-body training environment, as an additional objective alongside the established locomotion configuration.

The retargeted motions are purely kinematic, so the trained policy does not replay them: it is the physically simulated movement that approximates each reference while still walking, balancing and following its commands. The resulting policies were assessed using the same simulation-validation procedure as the previous objectives.

Figure 5.20 illustrates this stage for the Booster T1 at maximum velocity in Isaac Sim, where the alternating arm motion is now accompanied by visible elbow flexion accurate to the recorded motions, giving the swing a more articulated form than simply moving a nearly fixed arm forwards and backwards. This pronounced swing is, however, specific to fast walking, and is not applied uniformly at every speed. Figure 5.21 shows the same policy in MuJoCo across its command range: at the lowest speed the arms remain close to the trunk, much as in the policy of the previous objective, and both the stride and the arm swing grow as the commanded speed increases.

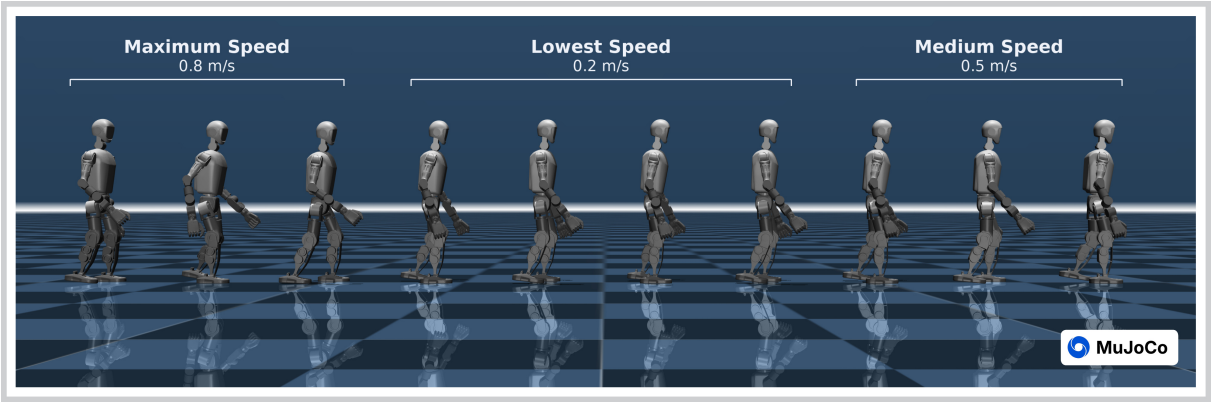


Figure 5.21: Validation Results for the Learned Velocity-Conditioned Behavior in MuJoCo.

Beyond the resulting motion, the references also changed how quickly a stable gait was learned. Figure 5.22 compares the training reward with and without reference guidance for the T1, G1 and K1.

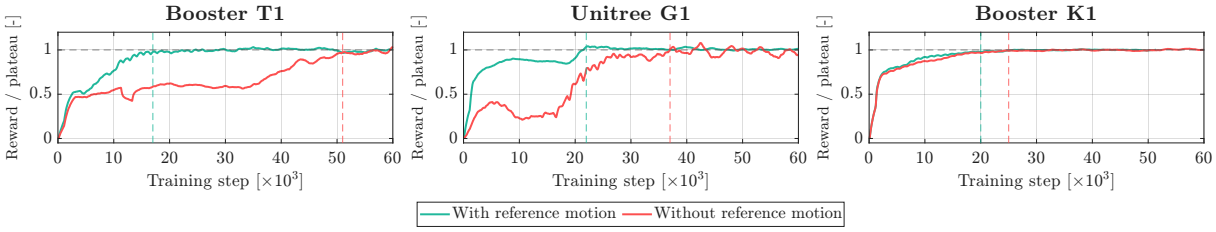


Figure 5.22: Normalized Training Reward of the Walking Policy With (Green) and Without (Red) the Reference Motion, for the Booster T1, Unitree G1 and Booster K1. Each curve is the environment reward smoothed over 2000 steps and rescaled so that 0 is its initial value and 1 its plateau (the median over the second half of training), so the curves compare how fast training stabilizes rather than the absolute reward, which differs between the two reward formulations. Dashed vertical lines mark where each run converges to its plateau.

The effect is largest on the T1, where the reference-guided run stabilizes in roughly 65% fewer training steps. Without references, the reward first stalls well below its final value, a period in which the agent must learn a walking gait from the task and regularization rewards alone; with a reference, that pattern is supplied from the outset and this phase is largely bypassed. The benefit does not transfer equally to the other robots: the G1 shows the same effect in a milder form, while on the K1 the two curves are nearly

indistinguishable. This suggests that the reference helps most where walking is hardest to discover, since a smaller and lighter robot with fewer degrees of freedom, such as the K1, already reaches a stable gait quickly on its own.

The same retargeting and training approach was also applied to the remaining robot models, with the resulting walking policies for the K1, GR3, G1, A3, DR02 and Kangaroo presented in Figure 5.23.

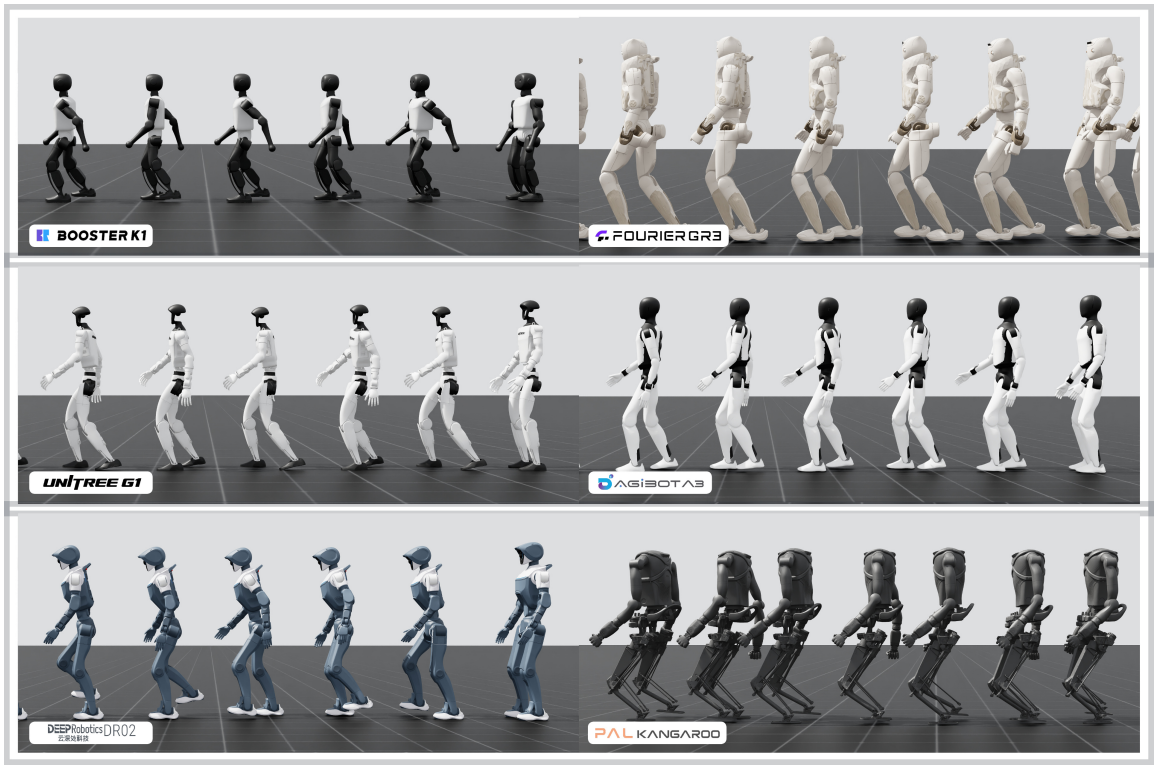


Figure 5.23: Validation Results for the Reference Motion Enhanced Policy for the Other Robots (Videos).

Just as the retargeting produced visibly different references for each robot, the trained policies express the shared human motion differently. What they have in common is the upper body: all six robots adopt a similar alternating arm swing, in contrast with the energy-efficiency policies, where the preferred arm motion differed between embodiments. Here, every robot is attempting to emulate the same human movement.

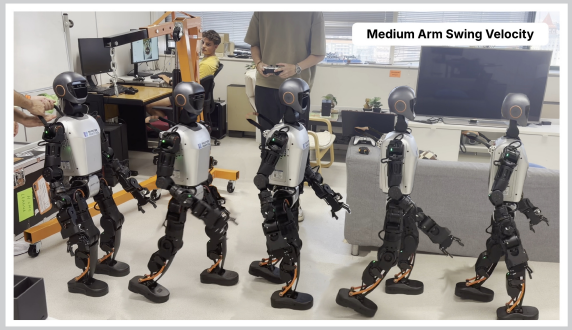
The clearest difference between them lies in the trunk. The G1, GR3 and DR02 have a waist pitch joint, which allows them to reproduce the forward lean of the recorded subject. The G1 and DR02 adopt it, giving their gait a noticeably more natural appearance. The GR3, however, converged to an upright posture despite being guided towards the same lean: on a robot with such a bulky upper body, leaning forward while moving proved less stable than disregarding that part of the reference. The reference therefore acts as guidance that the policy follows only as far as it remains compatible with stable walking. The robots without this joint walk upright, but their gait is still considerably more human-like and symmetrical than before.

The T1 policy was then deployed on the real robot. Figure 5.24 shows three cases: walking at the maximum forward command, decelerating from that command to standstill, and walking at a constant medium forward command.

The policy transfers faithfully onto the real robot. At the maximum command, the upper-body movement is virtually indistinguishable from the one validated in simulation, showing that the real robot is able to swing its heavy arms through this wide range without any motor limitation issue. The improvement over the earlier whole-body deployment is immediately apparent: the movement is more coordinated and controlled, and noticeably more life-like.

(a) Maximum Forward Velocity Commanded.

(b) Deceleration from Maximum to Standstill.



(c) Constant Medium Forward Velocity Commanded.

Figure 5.24: Deployment Results of the Reference Motion Enhanced Policy (Video).

The velocity-conditioned behavior is also accurately preserved. At the medium command the arm swing is clearly reduced, as validated in simulation, and during deceleration the arms return towards their resting configuration as the robot comes to a stop, with only small corrective adjustments and no abrupt movements. This variation with speed gives the locomotion a depth and character of its own: the robot does not simply walk faster or slower, it walks differently.

To support these qualitative assessments, the joint trajectories recorded during deployment were compared across the three locomotion objectives, as shown in Figure 5.25. The legged policy keeps the shoulder and elbow close to their fixed configuration, yet its arm trajectories are the most irregular of the three: holding the arms in place against gravity and the reaction forces of walking forces these joints to correct continuously, which appears as small, uneven oscillations around the nominal value. Freeing the arms removes most of this irregularity. In the whole-body policy, however, the conservative nature of the swing is apparent, mainly in the low peaks of the elbow pitch and, to a lesser extent, of the shoulder pitch. With reference guidance, both joints follow strong, smooth and mutually coordinated waves with distinct timing within the gait cycle, and elbow flexion becomes an articulated component of the swing rather than remaining nearly constant.

The knee pitch shows that this increase in upper-body motion is consistently accompanied by lower

knee flexion across the objectives. The larger reduction occurs between the legged and the whole-body policy, where the robot no longer has to counteract the momentum of its walk with rigidly held arms, an effort that would otherwise be absorbed by the legs. A further reduction is displayed with the reference motion tuned policy, although smaller.

Table 5.3 quantifies this last step. Mean knee flexion decreases from $37.8°$ to $29.3°$ and peak flexion from $64.9°$ to $49.5°$, so the reference-based controller adopts a less crouched leg configuration in accordance to its more expressive upper body. The lateral excursions decrease as well, from $10.7°$ to $8.4°$ in hip roll and from $27.9°$ to $15.8°$ in ankle roll, showing that the greater arm articulation does not amplify movement throughout the robot. The mechanical demand on the legs follows the same direction: mean leg power falls from $20.1\,\mathrm{W}$ to $19.0\,\mathrm{W}$, with the largest change at the ankle, whose power decreases from $13.6\,\mathrm{W}$ to $6.4\,\mathrm{W}$, alongside a smaller reduction in ankle-pitch torque.

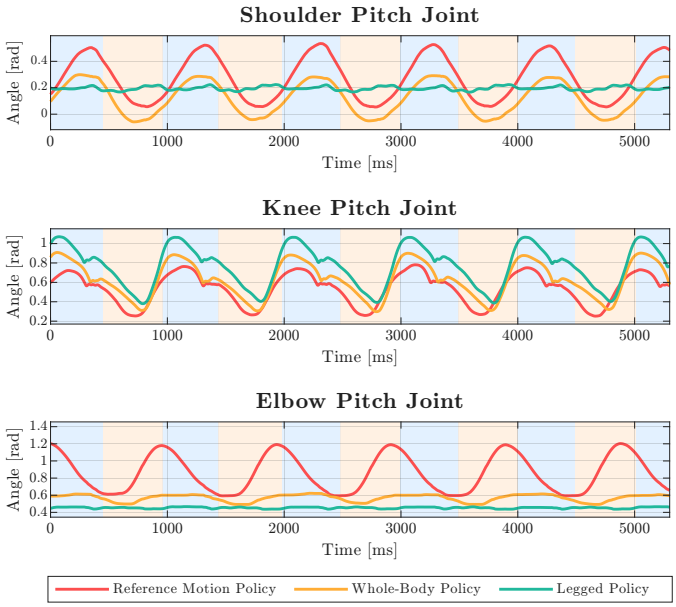


Figure 5.25: Right Shoulder, Knee Pitch and Elbow Pitch Joints of the Three T1 Policies in Real Deployment. Shading marks right (blue) and left (orange) foot forward.

Table 5.3: Leg Load per Stride: Whole-Body vs Reference Motion Policy.

Metric	Whole-Body	Ref. Motion	Change
Leg energy			
Leg power $\overline{\lvert\tau\dot{q}\rvert}$ $[W]$	20.1	19.0	−6%
Leg positive power $[W]$	18.1	17.0	−6%
Ankle load			
Ankle power $[W]$	13.6	6.4	−53%
Ankle share of leg power [%]	39.1	20.0	−49%
Ankle pitch RMS τ $[Nm]$	15.5	14.6	−6%
Lateral sway			
Hip roll range [°]	10.7	8.4	−22%
Ankle roll range [°]	27.9	15.8	−43%
Posture			
Mean knee flexion [°]	37.8	29.3	−22%
Peak knee flexion [°]	64.9	49.5	−24%
L–R wave similarity [%]			
Hip roll	64.3	80.2	+25%
Knee pitch	93.0	96.5	+4%
Ankle pitch	74.4	91.8	+23%
Shoulder pitch	93.8	95.7	+2%
Elbow flexion	51.2	98.1	+92%
Mean, all leg joints	74.7	80.8	+8%
Mean, arm joints	46.8	95.3	+104%

This brings an important consideration. The wider arm movement makes the reference-based policy less energy efficient overall than the whole-body policy, yet it is the one that places the lower load on the legs. As the legs are in constant contact with the ground, they receive the most strenuous forces and are likely to be the joints most prone to wear over time. Where preserving them is the priority, the reference-based policy may therefore be the preferable option, despite its higher total energy cost.

The same table shows how much more symmetric the movement became. The improvement is most pronounced in the upper body, which was also where the whole-body policy was asymmetric: mean left–right waveform similarity across the arm joints increases from 46.8% to 95.3%, with elbow flexion rising from 51.2% to 98.1%, while shoulder pitch, already at 93.8%, improves modestly to 95.7%. The gain is therefore not simply a larger swing, but a much closer correspondence between the two arms, especially in the elbow behavior that the conventional rewards specified only weakly.

The legs started from a higher baseline, since they are constrained by ground contact: any strong

asymmetry between them would compromise stability, so even without symmetry being explicitly rewarded, the whole-body policy could never drift towards a highly asymmetric leg gait. Their similarity nevertheless increases from 74.7% to 80.8% on average, most visibly in hip roll, from 64.3% to 80.2%, and ankle pitch, from 74.4% to 91.8%, with knee pitch rising from 93.0% to 96.5%. The leg movement itself thus became more natural, capturing nuances of the human gait beyond the arm swing.

Taken together, this objective adds a more explicit description of the desired gait to the locomotion framework established in the previous stages. Its contribution is not simply that the robot moves its arms further, but that the movement becomes more structured across joints and between sides, with velocity-dependent upper-body expression and a lower load on the legs, while remaining deployable under the tested walking and stopping commands. This establishes the reference-enhanced walking behavior used as the foundation for the subsequent terrain-traversal objective.

5.4 Height Traversal

5.4.1 Ascent Results

The final objective extends the reference-enhanced walking behavior to stairs. Figure 5.26 shows the resulting ascent on the T1 in Isaac Sim. The robot approaches with its regular walking gait, and at each step lifts the lead leg with pronounced hip and knee flexion to place the foot on the next tread, before raising its body and bringing the trailing leg up to meet it.

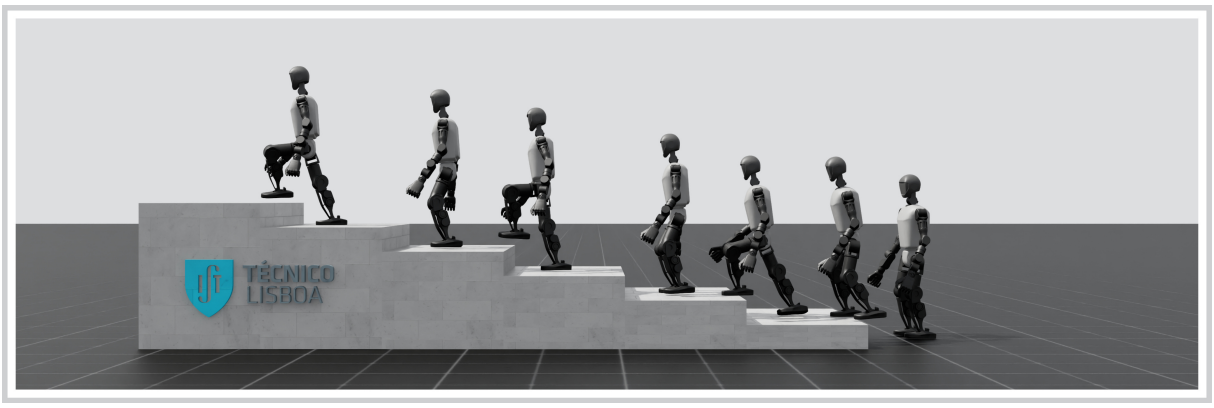


Figure 5.26: Stair Ascent of the Booster T1 in Isaac Sim of 18 cm Steps (Video).

This is a much more strenuous motion than the previously developed walking, where now both the joint positions and torques get utilized at a considerably larger magnitude in order to perform the required positioning and force on these higher steps. The same approach was applied to the remaining robots, which makes it possible to compare how different embodiments solve the same staircase. Figure 5.27 shows the leg pitch joints of six robots climbing an identical flight. All of them converge to the same basic strategy: the lead leg flexes strongly at the hip and knee at the start of each step, while the trailing leg follows later in the cycle with a smaller excursion. What differs is how much effort this requires. For the smaller T1 and K1, a $15\,\mathrm{cm}$ step is large in proportion to their legs, and their lead hip reaches the highest flexion of the group, whereas the taller A3 and GR3 clear the same step with roughly half of

that excursion. The same ordering appears in the time taken per step: most robots climb at a similar pace, but the K1 needs more than twice as long, taking additional footsteps on each tread to reposition itself before the next step. The narrow deviation bands show that each robot repeats its own pattern consistently from one step to the next.

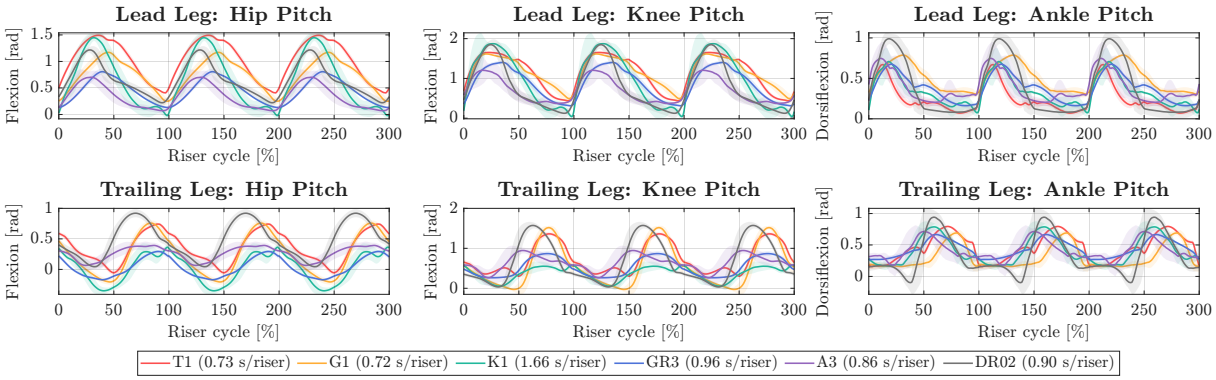


Figure 5.27: Leg Pitch Joints of the Six Humanoid Robots Climbing the Same Flight of Twelve $15\,\mathrm{cm}$ Steps with $0.4\,\mathrm{m}$ Treads at a Commanded $0.5\,\mathrm{m\,s^{-1}}$. A step cycle runs from the lift-off of the foot that first reaches a new step to the next lead leg lift-off, so each cycle climbs exactly one step regardless of how many footsteps the robot takes on the tread. Lines show the mean and shaded bands ± 1 standard deviation across step cycles within a single climb.

To establish how far this capability extends, the trained policies were evaluated on full flights of increasing step height, in both simulators and for three tread depths, as shown in Figure 5.28 for the Booster T1, where success is granted only when the robot climbs the full 12 step staircase at the given height and depth without any foot misplacement or instability. It was noted that the ascent is completed in virtually every episode up to steps of $25\,\mathrm{cm}$ in height, after which the success rate collapses within two to three centimeters. This limit is nearly the same across tread depths and across MuJoCo and Isaac Sim, which indicates that it reflects what the robot can physically reach rather than a particularity of one simulator or staircase.

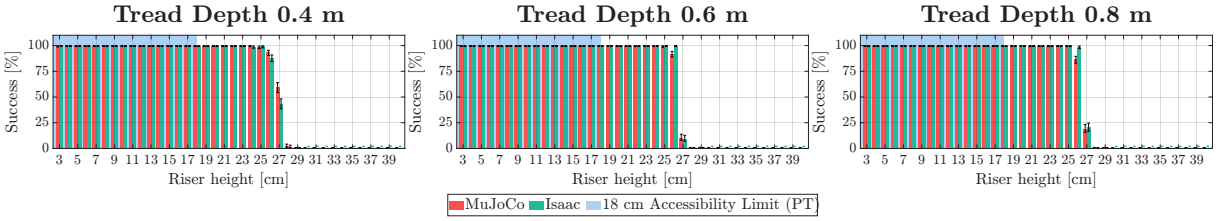


Figure 5.28: T1 Stair Ascent Success in MuJoCo and Isaac Sim across Step Heights and Tread Depths. Each condition contains 400 randomized-start episodes on a uniform 12-step staircase, with a forward command of $0.5\,\mathrm{m/s}$ and an $80\,\mathrm{s}$ simulation-time limit. Error bars indicate pointwise 95% Wilson confidence intervals.

For the remaining robots, Figure A.16 in the Appendix supports that conclusion, as the ones that exhibit longer legs and higher ease of clearance are robust to larger intervals of step heights. The K1 starts struggling from the 21 cm height onwards, worse than the T1, as expected of its smaller stature. However it maintains a very high success rate until that range, being able to ascend entire staircases whose steps are larger than 23% of its height in a controlled manner. The GR3, DR02 and A3 exhibit a much more gradual decay of their ascent capabilities. Their success rates stay above 90% up to roughly $25\,\mathrm{cm}$ (A3), $26\,\mathrm{cm}$ (GR3) and $28\,\mathrm{cm}$ (DR02), and then take a further 6 to 10 cm of step height to reach

the robot's capability limits, whereas the T1 goes through the same drop within about $2\,\mathrm{cm}$, as shown in Table 5.4. The Kangaroo's intricate parallel linkage causes a similar leg reach to the T1 and K1, which manifests itself in similar success rates, despite its comparable size to the larger robots of the study. However, all of them show near perfect success rates inside the 18 cm regulation limit.

Table 5.4: High-Step Completion Rates and Measured Behavior of the Trained Ascent Policies.

	Step Height Limits			*Arm motion*		*Policy stepping behavior*				
Robot	**24 cm** [%]	**28 cm** [%]	**32 cm** [%]	**Arm H** [10^{-3}]	**Opposition** [%]	**Tread time** [s]	**Lead lift** [% L]	**Trail lift** [% L]	**Swing RMS** [% L]	**Margin L/T** [cm]
T1	98	0	0	10.2	60.4	1.17	39.8	45.7	4.2	29.6 / 26.8
K1	4	0	0	10.3	32.9	2.14	54.3	51.6	15.9	35.5 / 29.4
G1	99.5	47	0	15.0	59.7	1.24	33.3	39.7	5.7	26.4 / 30.4
GR3	100	92	0	5.5	11.3	1.18	26.4	26.1	7.4	23.6 / 6.8
A3	100	100	41	6.0	27.1	1.42	28.0	29.6	5.7	30.3 / 28.6
DR02	100	96	24	16.9	47.7	1.02	43.7	61.5	10.9	20.5 / 7.2
Kangaroo	67	20	0	15.9	25.0	1.58	38.1	34.3	6.4	29.8 / 25.7

Notes. L is the robot's leg length and M its total mass. **Arm H**: RMS angular momentum of the arms about the whole-body center of mass over the ascent, normalized by $ML\sqrt{gL}$; larger values mean the arms carry more rotational motion relative to the robot's size. **Opposition**: share of the ascent during which the angular momentum of the arms points against that of the rest of the body; values above 50% indicate arms that mostly counter-rotate, and values below 50% are arms that mostly rotate with the body. **Tread time**: time between the first foot landings on consecutive treads, i.e. the time taken per step. **Lead lift** and **Trail lift**: peak height of the sole above the tread it leaves during a swing, for the first foot to reach a new tread and for the foot that follows it. **Swing RMS**: RMS deviation of each swing trajectory from the mean swing of the same foot within an episode; lower values mean a more repeatable step. **Margin L/T**: horizontal gap between the sole and the next step at touchdown, for the lead and trail foot; small values mean the foot lands close to the step face.

Relative to leg length, the K1 and DR02 lift their feet the highest, and the T1 has the most repeatable swing of the group, while the K1 has the least. The arms also take part in the ascent as a way to adjust its center of mass. These behaviors are learned by the policies on their own accord, as no established rewards force these adaptations, and appear to have different degrees for each robot: on the T1 and G1, the arm momentum opposes that of the rest of the body for around $60\,\%$ of the active samples, whereas on the GR3 it rarely does (Figure 5.29).

Figure 5.29: Ascent Validation on Isaac Sim across all the Studied Robots (Video).

5.4.2 Descent Results

Including descent in the training did not degrade the ascent robustness, resulting in a single policy that performs both interchangeably, distinguishing between them through the heightmap observations. As mentioned in the methodology, step descent requires different placements and timings, with a focus on not speeding through the steps, by being able to halt its momentum at every drop. Whereas ascent requires performing high vertical steps, the trained controlled movement uses long horizontal strides to

firmly plant itself. This is exacerbated on the T1 due to its short leg reach, so it utilizes its entire hip pitch range on the swing. Figure 5.30 shows the validation in Isaac Sim of the trained policy, where this controlled stepping and momentum regulating can be observed, especially considering that for these heights the robot's only form of stability is on a single foot, while completely spread horizontally. This is why the arms perform grounding maneuvers, to even out the weight distribution and stop any forward momentum to the next steps uncontrollably.

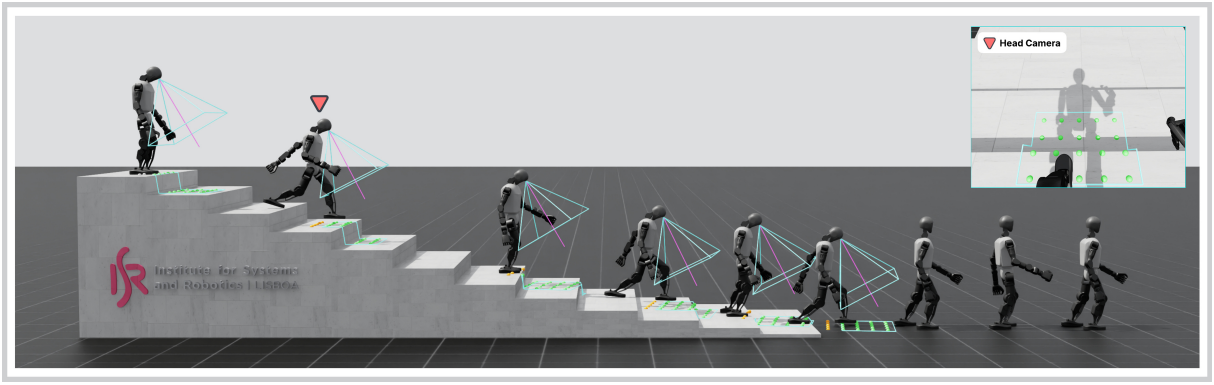


Figure 5.30: Stair Descent of the Booster T1 Guided by its Head Camera (Video).

Furthermore, the Figure also shows in detail how the sensor architecture works in real time: the robot turns its head down, towards an angle at which the camera can capture the highest portion of heightmap points as it can, represented as the green dots. The row of yellow dots cannot be directly seen, so they are estimated from stored memory of the perceived heightmap points. Occlusions with its legs or arms when it performs the long horizontal swings can be denoted, which are filtered out to avoid heightmap tampering. It then exhibits the transition blend towards the reference tuned locomotion policy, at the moment when the heightmap ceases to detect more steps in front of it, measured in Figure 5.31.

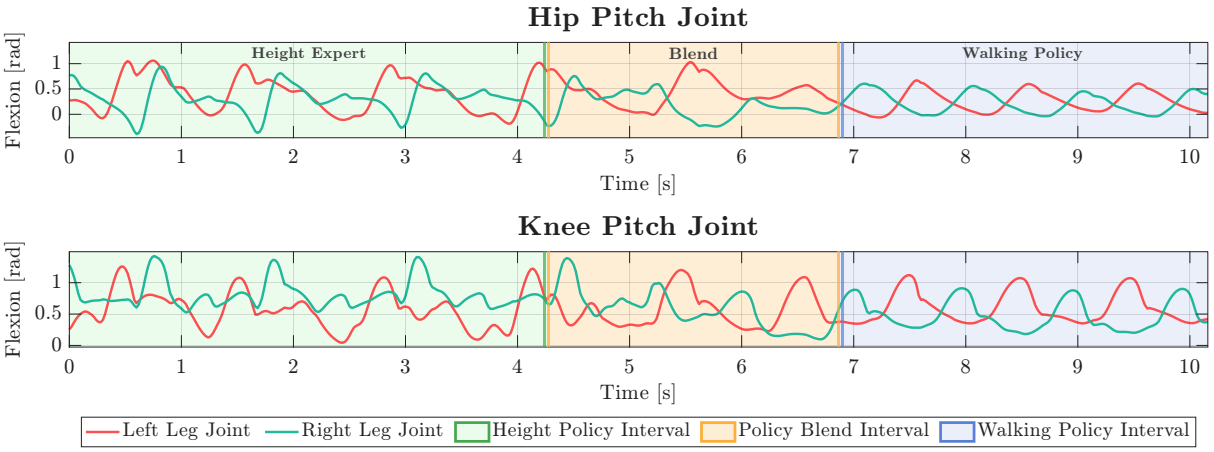


Figure 5.31: T1 Hip and Knee Pitch across the Hand-Off from Stair Descent to Walking. Shading marks the policy in control: height expert, blend, and walking policy. The uneven height traversal gait gives way to similar, alternating walking cycles of about $1.0\,\mathrm{s}$.

Control is moderately handed back to the walking policy through a blend lasting approximately 2.8 seconds, and within a further second and a half the joints settle into the regular, alternating cycles of the reference-enhanced gait, at about one second per cycle. The transition takes place without the robot stopping or moving erratically.

Trialed success rates in Figure A.17 in the Appendix show the same level of robustness as recorded for the ascent, except at the shortest trained step depth, 0.4 m. This is because this depth is barely longer than the bulky feet of the robot, which makes placement have to be extremely accurate, and in the 14-18 cm range, the horizontal lead leg swing can no longer touch down safely before the trailing leg moves, which generates momentum that is hard to stop completely with so little room between steps. This effect accumulates on each step until it misses the placement on a step, so it fails due to an incorrect touchdown.

5.4.3 Alternating Stride Results

Figure 5.32 shows the extended ascent behavior, obtained on the DR02. This robot was chosen due to having the highest leg length and reach, which is essential for this motion to be achievable: because it requires having the robot step over two steps per leg instead of one, a robot like the T1 cannot achieve the necessary lunge combined with the level of stability and precision needed to perfectly touchdown in a controlled motion. Its much shorter and bulky reach renders the margin of error to be impossible to learn and perform repeatedly.

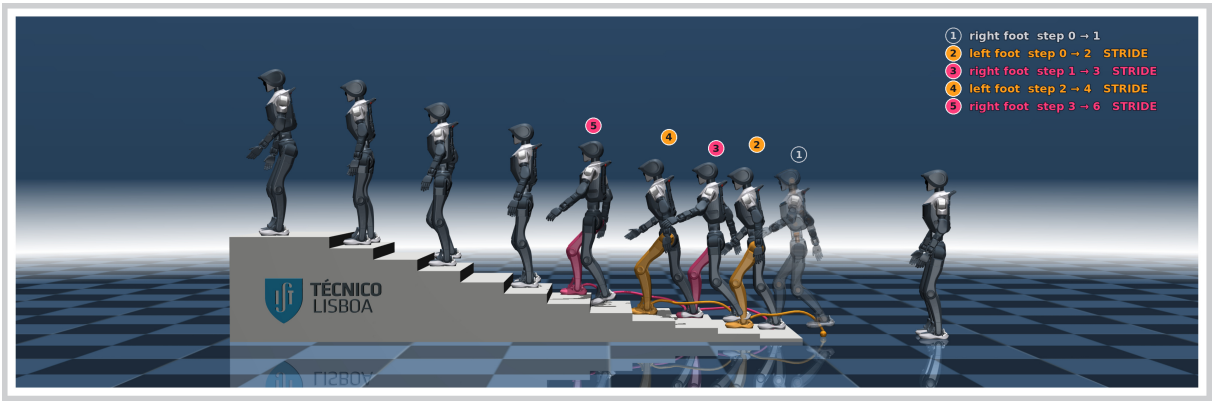


Figure 5.32: Striding Ascent of the DR02 in MuJoCo (Video).

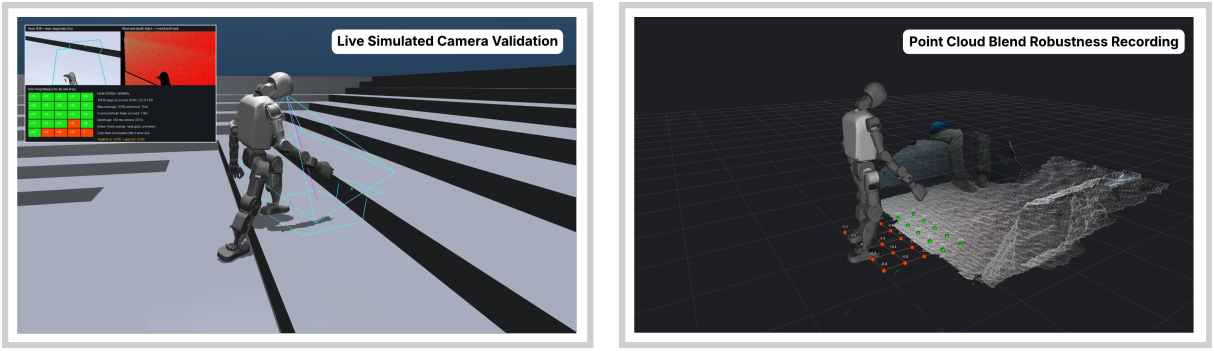
Thus, the robot performs the prolonged step when in the 0.40-0.50 m step depth range and a specified height interval, but even in these movements, it utilizes its maximum leg reach, and can marginally maintain the trailing foot planted when taking the lead swing. When it encounters steps it cannot clear this way, it falls back to the original ascent motion.

5.4.4 Live Camera to Heightmap Robustness Trials

The ascent and descent results of the previous sections were obtained with the heightmap imperfections that domain randomization introduces to approximate those of the real robot (Section 4.8), so these imperfections did not compromise the climbing behavior. When evaluated on the same terrains with ground truth heightmap measurements, the policy's ascent progress was increased by only 8 % and the share of episodes completed without a fall by 7 %, while its descent progress was not increased,

so the randomization had little cost on performance. Remaining gaps to the real robot include the perception pipeline itself, and the deployment script that runs it, which were evaluated in the three stages described there (Figure 5.33).

In validation, with the policy observing only the heightmap reconstructed from the simulated head camera, with its depth noise, dropouts and latency, the robot climbed the stairs with the same controlled stepping, as shown by the lead foot touching down on a 16 cm step (Figure 5.33a).



(a) Simulated Head Camera in MuJoCo Run.

(b) Real Point Clouds Replayed in Simulation.

(c) Deployment Script Ascent and Descent in Webots through the SDK.

Figure 5.33: Validation Tests of the Camera-Based Stair Pipeline. Cyan marks the camera's field of view. Heightmap samples are green when observed in the latest depth frame and orange when read from the map's memory.

In Webots, in one continuous run through the SDK, the robot climbed four uniform steps, crossed a landing and descended four more, with the heightmap reconstructed from the simulated camera throughout (Figure 5.33c). The limbs that induced occlusion were removed from the heightmap during the long swings, the remembered rows covered the area under the feet, and the blend handed control back to the walking policy once no further steps were detected, confirming that the added observations, the climbing context and the policy switching behave in the deployment script as they did in validation.

Finally, on the point clouds recorded by the real head camera, replayed along the robot's route (Figure 5.33b), only the two far rows were in view, and the nearer rows were filled from memory. Over the 58 s recording, the depth feed delivered all 1,737 frames at 30 Hz without a single drop, its longest gap of 46 ms remaining well below the 60–300 ms drops randomized in training, and every far-row cell received depth returns in every frame, with only 1.2 % of the depth pixels around them missing on average. On this mostly flat floor, the blend held the walking policy for all 2,880 control steps without a single false traversable height detection.

6

Conclusions

This thesis set out to achieve natural and robust locomotion for the Booster T1, a humanoid whose long, heavy arms and dexterous hands make both harder to obtain, and whose original controller answers this with short, stompy steps and permanently retracted arms. The objectives were developed through a common structure of training, validation in multiple simulators and real robot deployment, and every approach was also trained on six further humanoids, which allowed each behavior to be compared directly against the previous one and across different statures and morphologies.

6.1 Achievements

The first objective established a legged locomotion policy that transfers directly onto the real robot, with joint trajectories closely matching those observed in MuJoCo, and with a wider, more fluid range of movement than the original Booster controller. Lowering the arms, even while still fixed, already reduced the load the legs have to take.

Extending the policy to the whole body confirmed that the arms are an asset rather than a burden for this morphology. With the arms swinging in anti-phase with the legs, the deployed policy reduced mean mechanical power by 41% and torque-rate RMS by 29.5% relative to the original Booster controller, and the arm trajectories transferred to the real robot with correlations of 0.95–0.99. The energy efficiency formulation showed that, when the cost of transport is the primary objective, the policy converges to

the same restrained arm swing, which indicates that this swing is the economical solution for the T1. Across the other robots, the economical swing follows each morphology rather than a universal pattern. Those with more human-like proportions swing their arms visibly more and stay upright, whereas the T1, with long and heavy arms, rotates its shoulder little yet displaces about as much arm mass as the K1. Efficiency alone, therefore, does not lead the T1 to a human-like gait.

The reference motion objective supplied what the reward function could not. Walking cycles extracted from an established motion library and from our own recordings, organized by velocity, produced a gait that is expressive at high speeds and reserved as the robot slows down, with strong, coordinated arm motion, a less crouched leg posture and lower ankle effort. On the T1, the reference-guided training also stabilized in roughly 65% fewer steps.

Finally, the perceptive climbing policy decomposes stair traversal into the sub-movements a person performs, rather than stumbling into the step and recovering. The T1 ascends full flights reliably up to steps of 25 cm in validation, well beyond the 18 cm regulated range, with a limit that is consistent across tread depths and simulators and therefore reflects the robot's physical reach. Descent reaches the same robustness except on the shallowest treads, where the robot's feet leave almost no margin for placement. Across the other robots, the reachable heights follow leg length, and the DR02 additionally learned an alternating stride over two treads where geometry allows. Blending the climbing policy with the reference-motion walker preserved both behaviors, with smooth hand-offs and without either expert degenerating.

6.2 Limitations

The stair traversal behaviors were validated in simulation, including the live conversion of the camera point cloud into the heightmap and the deployment script in Webots, but not yet on the real robot. The real camera adds potential noise, dropped frames and latency, and the stair motions use considerably more range and torque of the robot's joints, so they demand a far more exact transfer than walking does. Additionally, the descent behavior on treads that are barely longer than the feet remains fragile.

6.3 Future Work

The most immediate step is the real deployment of the stair traversal and the policy blend system, with numerous robustness trials needed to support it. Beyond that, several directions follow from the developed work:

- **Directed and cautious stepping.** Making omnidirectional switching instantaneous, with the foot rotating under yaw commands so that it can be directed to where it should land. This requires a much slower gait, but enables cautious stepping over floor irregularities, and could be made autonomous by mapping the floor and selecting footholds.
- **Step-level control on stairs.** Giving the operator complete control over individual steps: one

command per step, the ability to stop and remain stable between two treads, and a separate command for the trailing foot to meet the lead foot.

- **Perception at a distance.** Building a map of the staircase ahead, for instance an OctoMap, so that the robot can estimate and approach the stairs without having to look down, or looking down only slightly as it arrives.
- **Approach and adjustment policy.** Adding to the blend a policy dedicated to small, faint adjustment movements before a step, concerned only with the commanded direction and not with the commanded velocity.
- **Manipulation.** Following the same approach for manipulation: mapping the target surface, training a set of grasps on top of the stabilized locomotion policy, and adding a crouching motion to reach lower surfaces, all whilst ensuring that the robot remains stable and balanced throughout.

Bibliography

[1] L. Bao, J. Humphreys, T. Peng, and C. Zhou. Deep reinforcement learning for robotic bipedal locomotion: a brief survey. *Artificial Intelligence Review*, 2025.

[2] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn. Humanplus: Humanoid shadowing and imitation from humans. *arXiv preprint arXiv:2406.10454*, 2024.

[3] H. Zhang, L. Zhang, Z. Chen, L. Chen, Y. Wang, and R. Xiong. Natural humanoid robot locomotion with generative motion prior. *arXiv preprint arXiv:2503.09015*, 2025.

[4] B. Robotics. *T1 Instruction Manual*. https://booster.feishu.cn/wiki/XAS3wv4lwiSiXXkDbMrceE6UnHc, 2025.

[5] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Icml*, volume 99, pages 278–287. Citeseer, 1999.

[6] R. S. Sutton, A. G. Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[7] Y. Duan, X. Chen, R. Houthooft, J. Schulman, and P. Abbeel. Benchmarking deep reinforcement learning for continuous control. In *International conference on machine learning*, pages 1329–1338. PMLR, 2016.

[8] X. B. Peng, P. Abbeel, S. Levine, and M. Van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions On Graphics (TOG)*, 37(4): 1–14, 2018.

[9] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[11] A. P. Kalidas, C. J. Joshua, A. Q. Md, S. Basheer, S. Mohan, and S. Sakri. Deep reinforcement learning for vision-based navigation of uavs in avoiding stationary and mobile obstacles. *Drones*, 7 (4):245, 2023.

[12] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[13] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.

[14] S. Liu, Z. Yang, Z. Zhang, R. Jiang, T. Ren, Y. Jiang, S. Chen, and X. Zhang. Application of deep reinforcement learning in reconfiguration control of aircraft anti-skid braking system. *Aerospace*, 9 (10):555, 2022.

[15] K. Ter, O.-O. Ajayi, and D. Udekwe. Taxonomy and trends in reinforcement learning for robotics and control systems: A structured review. *arXiv preprint arXiv:2510.21758*, 2025.

[16] G. Gabrielli and T. von Kármán. What price speed? specific power required for propulsion of vehicles. *Mechanical Engineering*, 72:775–781, 1950.

[17] V. A. Tucker. The energetic cost of moving about. *American Scientist*, 63(4):413–419, 1975.

[18] S. Collins, A. Ruina, R. Tedrake, and M. Wisse. Efficient bipedal robots based on passive-dynamic walkers. *Science*, 307(5712):1082–1085, 2005.

[19] P. A. Bhounsule, J. Cortell, A. Grewal, B. Hendriksen, J. G. D. Karssen, C. Paul, and A. Ruina. Low-bandwidth reflex-based control for lower power walking: 65 km on a single battery charge. *The International Journal of Robotics Research*, 33(10):1305–1321, 2014.

[20] Z. Fu, A. Kumar, J. Malik, and D. Pathak. Minimizing energy consumption leads to the emergence of gaits in legged robots. In *Conference on Robot Learning (CoRL)*, 2021.

[21] R. M. Alexander and A. S. Jayes. A dynamic similarity hypothesis for the gaits of quadrupedal mammals. *Journal of Zoology*, 201(1):135–152, 1983.

[22] R. M. Alexander. Optimization and gaits in the locomotion of vertebrates. *Physiological Reviews*, 69(4):1199–1227, 1989.

[23] R. Kram, A. Domingo, and D. P. Ferris. Effect of reduced gravity on the preferred walk-run transition speed. *Journal of Experimental Biology*, 200(4):821–826, 1997.

[24] H. Elftman. The function of the arms in walking. *Human Biology*, 11(4):529–535, 1939.

[25] H. Herr and M. Popovic. Angular momentum in human walking. *Journal of Experimental Biology*, 211(4):467–481, 2008.

[26] S. H. Collins, P. G. Adamczyk, and A. D. Kuo. Dynamic arm swinging in human walking. *Proceedings of the Royal Society B*, 276(1673):3679–3688, 2009.

[27] M. Gleicher. Retargetting motion to new characters. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 33–42, 1998.

[28] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi. Learning human-to-humanoid real-time whole-body teleoperation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[29] Z. Luo, J. Cao, A. Winkler, K. Kitani, and W. Xu. Perpetual humanoid control for real-time simulated avatars. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[30] E. S. L. Ho, T. Komura, and C.-L. Tai. Spatial relationship preserving character motion adaptation. *ACM Transactions on Graphics*, 29(4), 2010.

[31] P. Badoni. Robotics controller: a literature survey. *Int J Eng Res Technol (IJERT)*, 4(10):502–508, 2015.

[32] S. W. Sung and I.-B. Lee. Limitations and countermeasures of pid controllers. *Industrial & engineering chemistry research*, 35(8):2596–2610, 1996.

[33] E. H. Binugroho, D. Pratama, A. Z. R. Syahputra, and D. Pramadihanto. Control for balancing line follower robot using discrete cascaded pid algorithm on adroit v1 education robot. In *2015 International Electronics Symposium (IES)*, pages 245–250. IEEE, 2015.

[34] R. Tedrake. *Underactuated Robotics*. 2023. URL `https://underactuated.csail.mit.edu`.

[35] A. Carron, E. Arcari, M. Wermelinger, L. Hewing, M. Hutter, and M. N. Zeilinger. Data-driven model predictive control for trajectory tracking with a robotic arm. *IEEE Robotics and Automation Letters*, 4(4):3758–3765, 2019.

[36] G. Klančar and I. Škrjanc. Tracking-error model-based predictive control for mobile robots in real time. *Robotics and autonomous systems*, 55(6):460–469, 2007.

[37] M. Spahn, B. Brito, and J. Alonso-Mora. Coupled mobile manipulation via trajectory optimization with free space decomposition. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12759–12765. IEEE, 2021.

[38] A. Plaat, W. Kosters, and M. Preuss. Deep model-based reinforcement learning for high-dimensional problems, a survey. *arXiv preprint arXiv:2008.05598*, 2020.

[39] M. Y. Galliker, N. Csomay-Shanklin, R. Grandia, A. J. Taylor, F. Farshidian, M. Hutter, and A. D. Ames. Planar bipedal locomotion with nonlinear model predictive control: Online gait generation using whole-body dynamics. In *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, pages 622–629. IEEE, 2022.

[40] Q. Chen, J. Li, S. Cheng, N. Hovakimyan, and Q. Nguyen. Autotuning bipedal locomotion mpc with grfm-net for efficient sim-to-real transfer. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 21296–21303. IEEE, 2025.

[41] J. Li and Q. Nguyen. Dynamic walking of bipedal robots on uneven stepping stones via adaptive-frequency mpc. *IEEE Control Systems Letters*, 7:1279–1284, 2023.

[42] Y.-M. Chen. *Toward High-Performance Simple Models of Legged Locomotion*. PhD thesis, University of Pennsylvania, 2023.

[43] C. Khazoom, S. Hong, M. Chignoli, E. Stanger-Jones, and S. Kim. Tailoring solution accuracy for fast whole-body model predictive control of legged robots. *IEEE Robotics and Automation Letters*, 2024.

[44] M. Krause, J. Englsberger, P.-B. Wieber, and C. Ott. Stabilization of the capture point dynamics for bipedal walking based on model predictive control. *IFAC Proceedings Volumes*, 45(22):165–171, 2012.

[45] F. M. Smaldone, N. Scianca, L. Lanari, and G. Oriolo. From walking to running: 3d humanoid gait generation via mpc. *Frontiers in Robotics and AI*, 9:876613, 2022.

[46] G. García, R. Griffin, and J. Pratt. Mpc-based locomotion control of bipedal robots with line-feet contact using centroidal dynamics. In *2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids)*, pages 276–282, 2021. doi: 10.1109/HUMANOIDS47582.2021.9555775.

[47] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel. Asymmetric actor critic for image-based robot learning. *arXiv preprint arXiv:1710.06542*, 2017.

[48] T. Peng, L. Bao, and C. Zhou. Gait-conditioned reinforcement learning with multi-phase curriculum for humanoid locomotion. *arXiv preprint arXiv:2505.20619*, 2025.

[49] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath. Real-world humanoid locomotion with reinforcement learning. *Science Robotics*, 9(89):eadi9579, 2024.

[50] B. Tidd, N. Hudson, and A. Cosgun. Guided curriculum learning for walking over complex terrain. *arXiv preprint arXiv:2010.03848*, 2020.

[51] Z. Gu, J. Li, W. Shen, W. Yu, Z. Xie, S. McCrory, X. Cheng, A. Shamsah, R. Griffin, C. K. Liu, et al. Humanoid locomotion and manipulation: Current progress and challenges in control, planning, and learning. *arXiv preprint arXiv:2501.02116*, 2025.

[52] G. Matheron, N. Perrin, and O. Sigaud. The problem with ddpg: understanding failures in deterministic environments with sparse rewards. *arXiv preprint arXiv:1911.11679*, 2019.

[53] O. Aydogmus and M. Yilmaz. Comparative analysis of reinforcement learning algorithms for bipedal robot locomotion. *IEEE Access*, 12:7490–7499, 2023.

[54] N. Heess, D. Tb, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. Eslami, et al. Emergence of locomotion behaviours in rich environments. *arXiv preprint arXiv:1707.02286*, 2017.

[55] Y. Seo, C. Sferrazza, H. Geng, M. Nauman, Z.-H. Yin, and P. Abbeel. Fasttd3: Simple, fast, and capable reinforcement learning for humanoid control. *arXiv preprint arXiv:2505.22642*, 2025.

[56] F. Torabi, G. Warnell, and P. Stone. Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*, 2018.

[57] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics (ToG)*, 40(4):1–20, 2021.

[58] F. G. Harvey, M. Yurick, D. Nowrouzezahrai, and C. Pal. Robust motion in-betweening. *ACM Transactions on Graphics*, 39(4), 2020.

[59] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black. AMASS: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5442–5451, 2019.

[60] J. P. Araujo et al. Retargeting matters: General motion retargeting for humanoid motion tracking. *arXiv preprint arXiv:2510.02252*, 2025.

[61] L. Yang et al. OmniRetarget: Interaction-preserving data generation for humanoid whole-body loco-manipulation and scene interaction. *arXiv preprint arXiv:2509.26633*, 2025.

[62] I. Radosavovic, S. Kamat, T. Darrell, and J. Malik. Learning humanoid locomotion over challenging terrain. *arXiv preprint arXiv:2410.03654*, 2024.

[63] X. Gu, Y.-J. Wang, X. Zhu, C. Shi, Y. Guo, Y. Liu, and J. Chen. Advancing humanoid locomotion: Mastering challenging terrains with denoising world model learning. In *Robotics: Science and Systems (RSS)*, 2024.

[64] J. Zhang, W. Xu, S. Ye, Y. He, W. Qi, Q. Sun, N. Ding, and L. Zhou. Explicit stair geometry conditioning for robust humanoid locomotion. *arXiv preprint arXiv:2605.09944*, 2026.

[65] A. Agarwal, A. Kumar, J. Malik, and D. Pathak. Legged locomotion in challenging terrains using egocentric vision. In *Conference on Robot Learning (CoRL)*, 2023.

[66] Z. Zhuang, S. Yao, and H. Zhao. Humanoid parkour learning. In *Conference on Robot Learning (CoRL)*, 2024.

[67] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2022.

[68] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.

[69] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter. Elevation mapping for locomotion and navigation using GPU. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[70] H. Wang, Z. Wang, J. Ren, Q. Ben, T. Huang, W. Zhang, and J. Pang. BeamDojo: Learning agile humanoid locomotion on sparse footholds. In *Robotics: Science and Systems (RSS)*, 2025.

[71] D. Wang, X. Wang, X. Liu, J. Shi, Y. Zhao, C. Bai, and X. Li. MoRE: Mixture of residual experts for humanoid lifelike gaits learning on complex terrains. *arXiv preprint arXiv:2506.08840*, 2025.

[72] J. Long, J. Ren, M. Shi, Z. Wang, T. Huang, P. Luo, and J. Pang. Learning humanoid locomotion with perceptive internal model. *arXiv preprint arXiv:2411.14386*, 2024.

[73] Y. Wang, P. Chen, X. Han, F. Wu, and M. Zhao. Booster gym: An end-to-end reinforcement learning framework for humanoid robot locomotion. *arXiv preprint arXiv:2506.15132*, 2025.

[74] D. A. Winter. *Biomechanics and Motor Control of Human Movement*. John Wiley & Sons, Hoboken, NJ, 4th edition, 2009. doi: 10.1002/9780470549148.

A

Appendix

This appendix gathers the material that supports Chapters 4 and 5: the training configuration and training curves of the policies, the joint limits of the Booster T1, the joint velocities and torques of the deployed policies, the energy efficiency policies across the additional robot morphologies, the comparison of real deployments with simulation, and the supplementary results of the height traversal policies.

A.1 Training Configuration of the Policies

Table A.1 lists the network, training and domain randomization parameters of the legged locomotion (Section 5.1) and whole-body objectives. The remaining objectives keep the same network, learning rates, number of environments, batch size, discount factor and domain randomization, and differ only in a smaller buffer of 4096 and their training length: 4 000 000 iterations for the energy objective, whose x and y command limits are also scaled by the Froude factor to $[-0.897, 0.897]$ and $[-0.717, 0.717]$, and 2 000 000 for the reference motion and 8 000 000 for the height traversal objective, the latter adding the stair randomization of its own section.

Table A.1: Network, Training and Domain Randomization Parameters of the Legged Locomotion and Whole-Body Policies.

Network Parameters	**Value**
Actor Hidden Layer Size	512
Critic Hidden Layer Size	1024
Actor Learning Rate	3×10^{-4}
Critic Learning Rate	3×10^{-4}
Training Parameters	**Value**
Number of Environments	1024
Number of Iterations	1 000 000
Buffer Size	8192
Batch Size	32768
Number of Evaluation Environments	1024
Discount Factor (γ)	0.97
Velocity Command Limit x and ω	$[-1.0, 1.0]$
Velocity Command Limit y	$[-0.8, 0.8]$
Domain Randomization	**Distribution**
Floor Friction	$\mathbf{u} \sim \mathcal{U}(0.2, 0.6)$
Link Mass	$\mathbf{u} \sim \mathcal{U}(0.98, 1.02)$
Torso Mass	$\mathbf{u} \sim \mathcal{U}(-1.0, 1.0)$
Initial Joint Position Jitter	$\mathbf{u} \sim \mathcal{U}(-0.05, 0.05)$
Joint Stiffness	$\mathbf{u} \sim \mathcal{U}(0.7, 1.3)$
Joint Damping	$\mathbf{u} \sim \mathcal{U}(0.7, 1.3)$
Ankle Joint Damping	$\mathbf{u} \sim \mathcal{U}(0.5, 2.0)$
Joint Friction Loss	$\mathbf{u} \sim \mathcal{U}(0.9, 1.1)$
Joint Armature	$\mathbf{u} \sim \mathcal{U}(1.0, 1.05)$

A.2 Training Curves of the Retracted Arms Configuration

Figures A.1 and A.2 complement the training results of Figure 5.1. The average reward rises quickly over the first 20,000 iterations, shown in the inset, and settles at its plateau of about 0.047 by around 80,000 iterations, with the evaluation return leveling at about 86 over the same period.

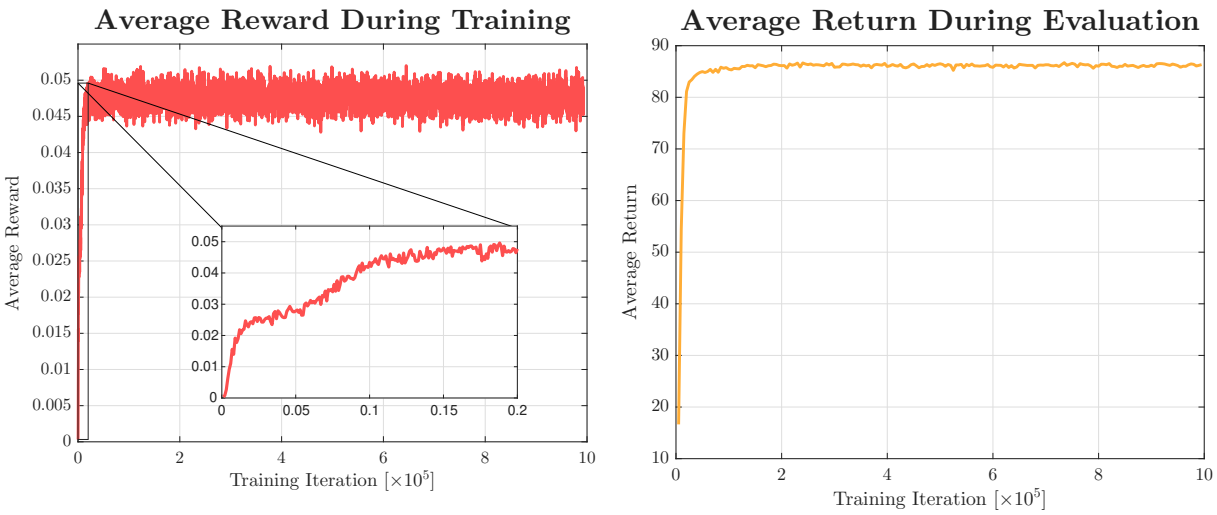


Figure A.1: Average Reward and Return During Training of the Retracted Arms Configuration.

Both losses keep decaying after the reward plateau, the actor loss from about -2.4 to -2.5 and the critic loss, more gradually, from approximately 3.95 to 3.85, which reflects the gradual removal of the residual behaviors described in Section 5.1, such as micro-movements when standing still.

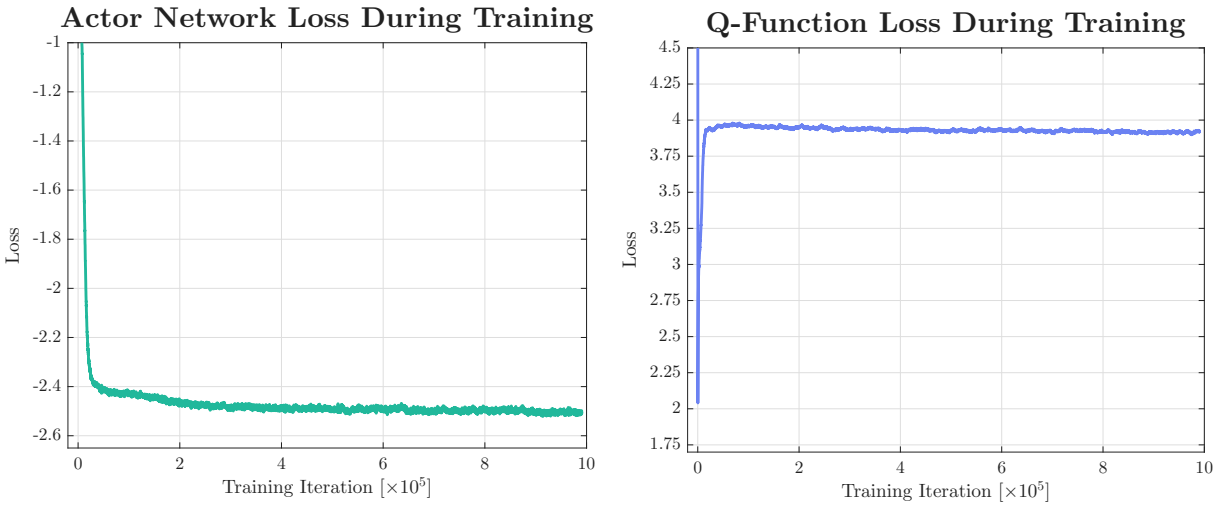


Figure A.2: Actor and Critic Losses During Training of the Retracted Arms Configuration.

A.3 Training Curves of the Rested Arms Configuration

Figures A.3 and A.4 show the same quantities for the rested arms configuration of Figure 5.2. The reward and return follow a very similar progression to the retracted arms, with the rested arms rising slightly faster in reward over the first 10,000 iterations, before both settle at the same level by around 80,000 iterations.

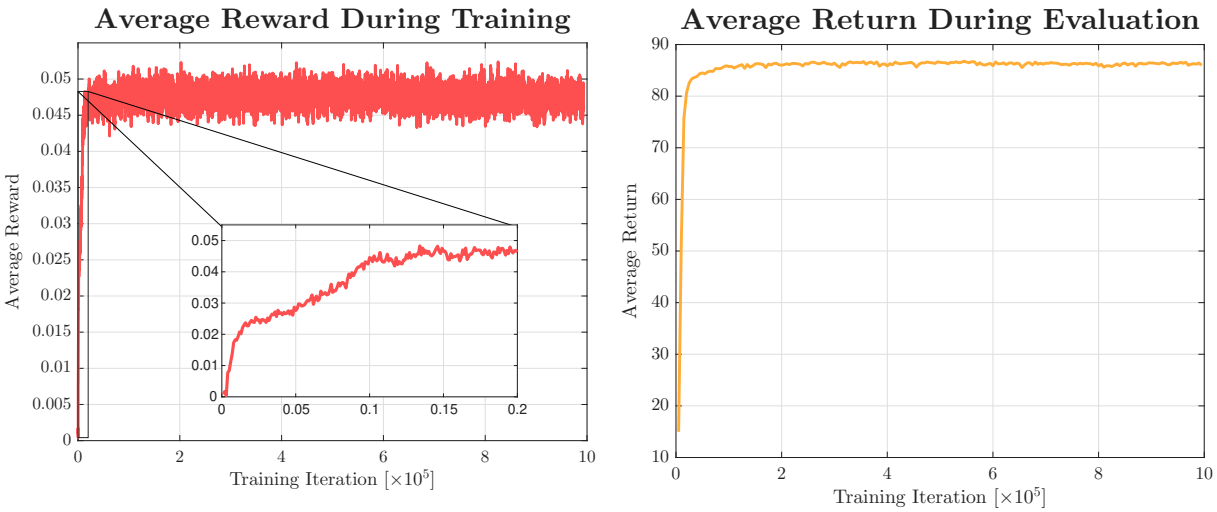


Figure A.3: Average Reward and Return During Training of the Rested Arms Configuration.

The losses also match the retracted arms configuration, both decaying slowly after the plateau, the actor loss to about -2.5 and the critic loss to about 3.85.

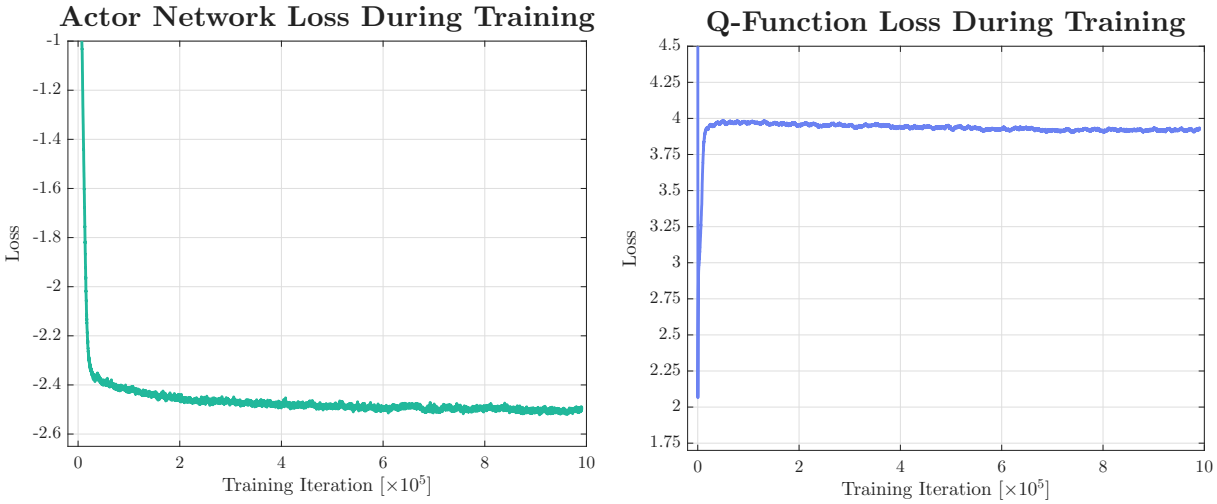


Figure A.4: Actor and Critic Losses During Training of the Rested Arms Configuration.

A.4 Joint Limits of the Booster T1

Table A.2 lists the joint angle limits of the Booster T1, as specified by the manufacturer. They are the reference for the joint ranges reported in Section 5.1.3 and for the admissible arm intervals of Section 4.3.

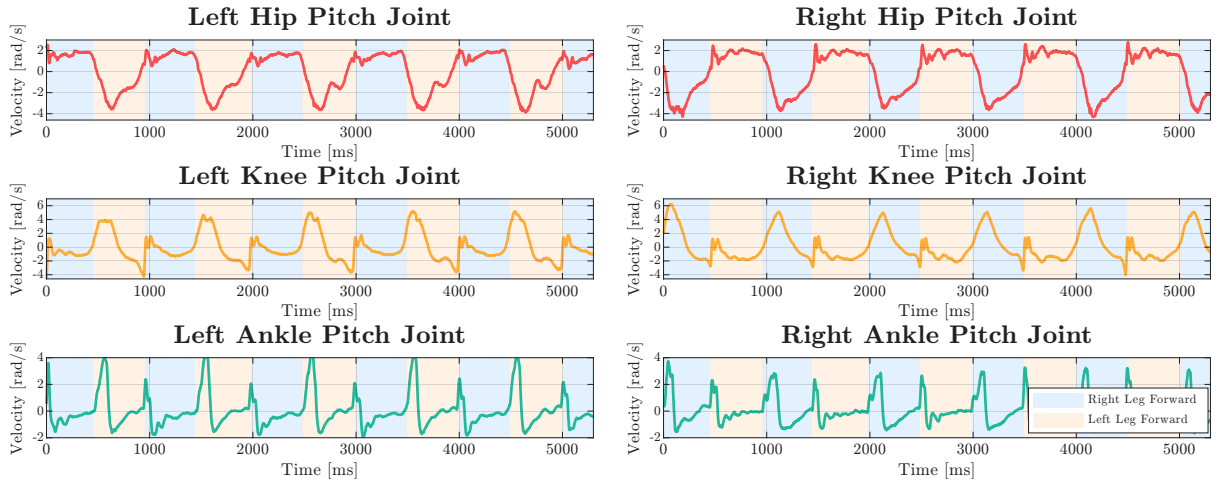
Table A.2: Joint Angle Limits in Radians for the Upper and Lower Body, as Detailed in [4].

Joint	**Minimum Angle Limit** [rad]	**Maximum Angle Limit** [rad]
Head Yaw	-1.01	1.01
Head Pitch	0.31	0.82
Left Shoulder Pitch	-3.28	1.19
Left Shoulder Roll	-1.64	1.54
Left Elbow Pitch	-2.23	2.23
Left Elbow Yaw	-2.09	0.03
Right Shoulder Pitch	-3.28	1.19
Right Shoulder Roll	-1.54	1.64
Right Elbow Pitch	-2.23	2.23
Right Elbow Yaw	-0.03	2.09
Waist Yaw	-1.01	1.01
Left Hip Pitch	-2.06	2.06
Left Hip Roll	-0.37	1.54
Left Hip Yaw	-1.01	1.01
Left Knee Pitch	0.00	2.15
Left Ankle Pitch	-0.85	0.40
Left Ankle Roll	-0.42	0.78
Right Hip Pitch	-2.06	2.06
Right Hip Roll	-1.54	0.37
Right Hip Yaw	-1.01	1.01
Right Knee Pitch	0.00	2.15
Right Ankle Pitch	-0.85	0.40
Right Ankle Roll	-0.42	0.78

The leg limits are in line with other humanoids of its size. The ankle pitch spans 1.25 rad, against 1.2–1.6 rad for the G1, K1, GR3 and A3, and the knee flexes up to 2.15 rad, the lowest of the group but close to the K1 (2.23 rad) and GR3 (2.36 rad). The hip pitch is the most restricted, spanning 4.12 rad against 5.2–5.5 rad for the others, though both cover far more than walking requires. The largest difference is in the arms: the shoulder pitch extends backwards by only 1.19 rad, a limit shared with the K1 from the same manufacturer, whereas the G1, GR3 and A3 allow 2.7–3.0 rad. The arm swing of the T1 is therefore bounded behind the body, closer to the human shoulder, whose extension stops at about 1 rad.

A.5 Joint Velocities of the Walking Policies in Deployment

The joint position analysis of Section 5.1.3 is complemented here by the corresponding angular velocities over the same forward walk cycle. Figure A.5 shows the velocities recorded on the real robot, where the hip pitch reaches its peak of 3.6–3.7 rad/s about a third of the way into the swing, and Figure A.6 compares them with the reference motion and whole-body policies.

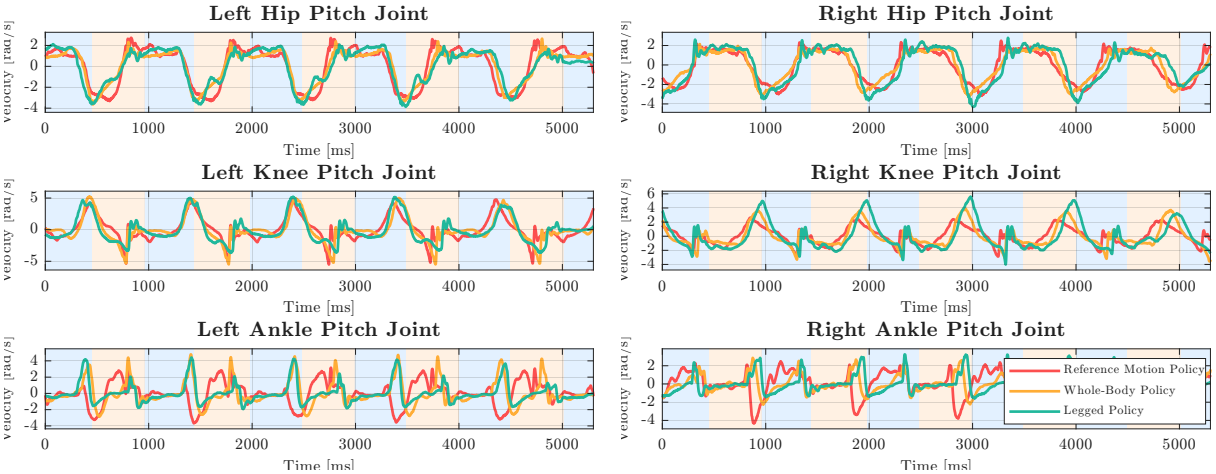


(a) Left Leg Joint Angular Velocities in a Forward Walk Cycle.

(b) Right Leg Joint Angular Velocities in a Forward Walk Cycle.

Figure A.5: Joint Angular Velocities of the Legged Policy Deployed on the Real Robot.

Figure A.6 overlays the joint velocities of the three walking policies on the real robot, aligned by stride. All three reach similar peaks at the same points of the gait cycle, about 3–4 rad/s at the hip and 5–5.6 rad/s at the knee. They differ at the ankle, where the reference motion policy reaches -3.7 to -4.3 rad/s against at most -2.9 rad/s for the other two, matching the deeper ankle motion seen in its joint angles.



(a) Left Leg Joint Angular Velocities in a Forward Walk Cycle.

(b) Right Leg Joint Angular Velocities in a Forward Walk Cycle.

Figure A.6: Joint Angular Velocities of the Reference Motion, Whole-Body and Legged Policies on the Real Robot.

A.6 Energy Efficiency Policies Across Robot Morphologies

Figures A.7 to A.11 show the energy efficiency policies of the five robots walking forward at their Froude-matched speeds (Videos), illustrating the postures and arm swings compared in Table 5.2. The T1 walks in a flexed stance, with its long arms hanging close to the knees and a shoulder swing of only 16.4°, which still displaces about as much arm mass as the robots with more human-like proportions.

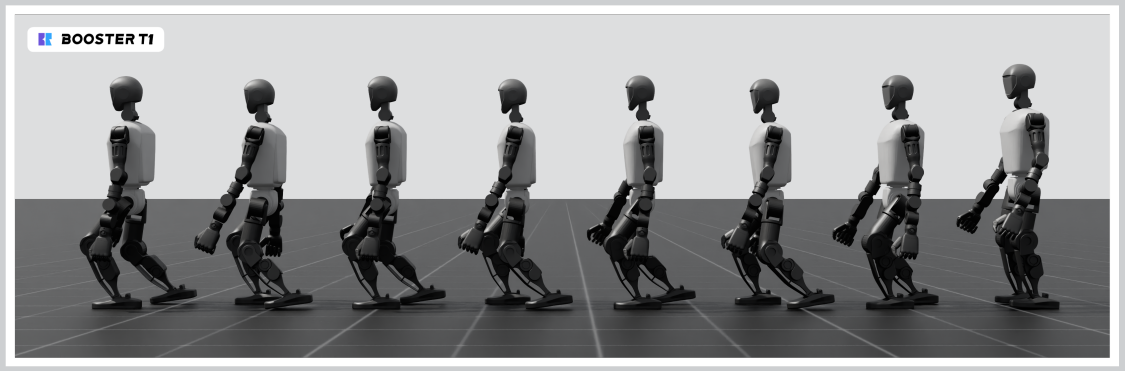


Figure A.7: Energy Efficiency Policy of the Booster T1.

The K1, from the same manufacturer but with arms closer to the human ratio, takes the opposite approach, keeping an upright posture and the largest arm swing of the group, with 41.0° at the shoulder.

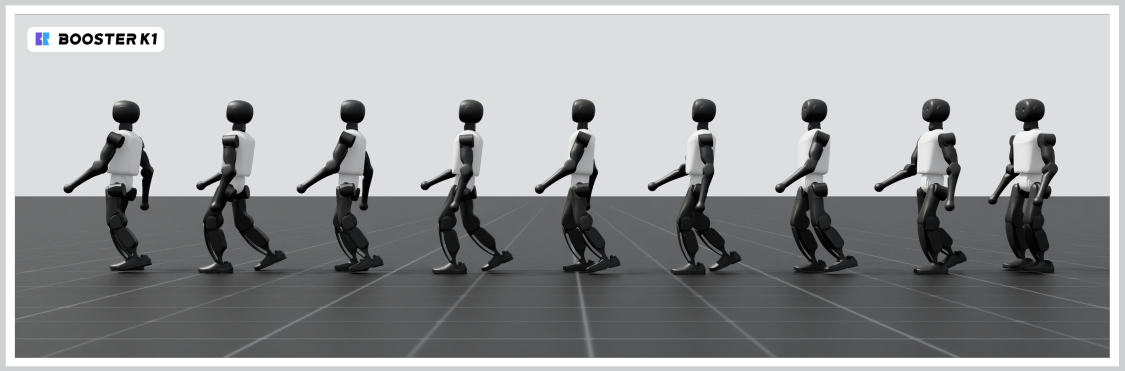


Figure A.8: Energy Efficiency Policy of the Booster K1.

The GR3, the tallest and heaviest of the robots, also walks with flexed knees, and its short, bulky arms barely move, with the smallest shoulder excursion (11.0°) and arm displacement of all five.

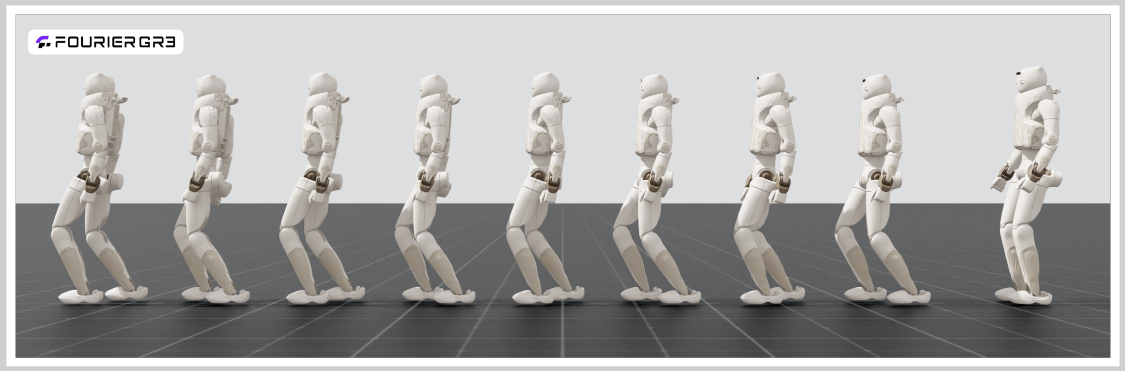


Figure A.9: Energy Efficiency Policy of the Fourier GR3.

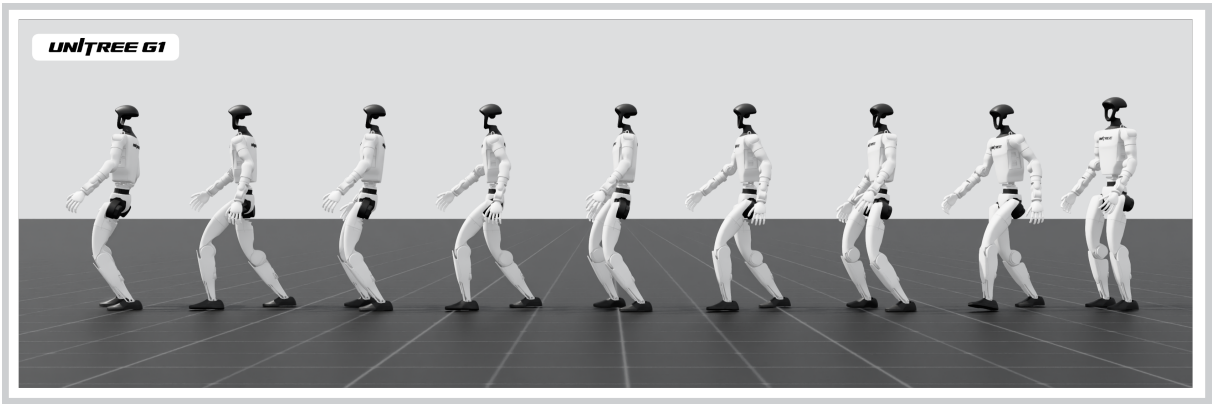


Figure A.10: Energy Efficiency Policy of the Unitree G1.

The G1 and A3, which match the human arm-to-leg ratio almost exactly, both walk upright with a clearly visible swing, of 24.0° and 21.3° at the shoulder, falling between the two extremes.

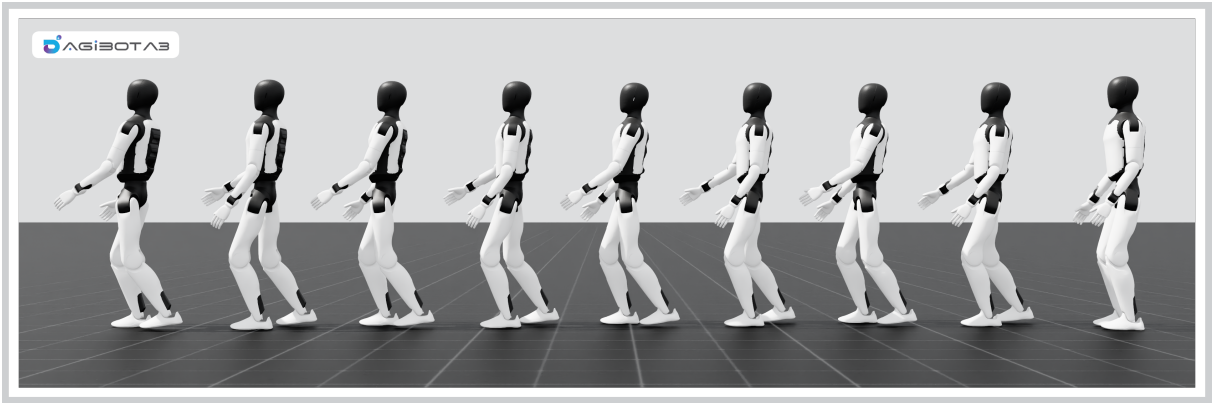
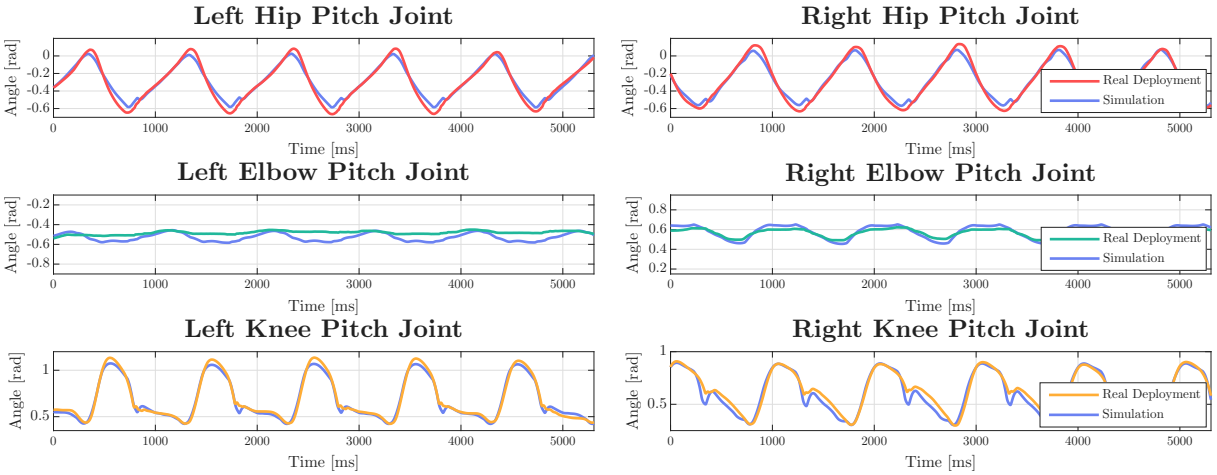


Figure A.11: Energy Efficiency Policy of the AgiBot A3.

A.7 Real Deployment *vs* Simulation Run for the Other Two Walking Objectives

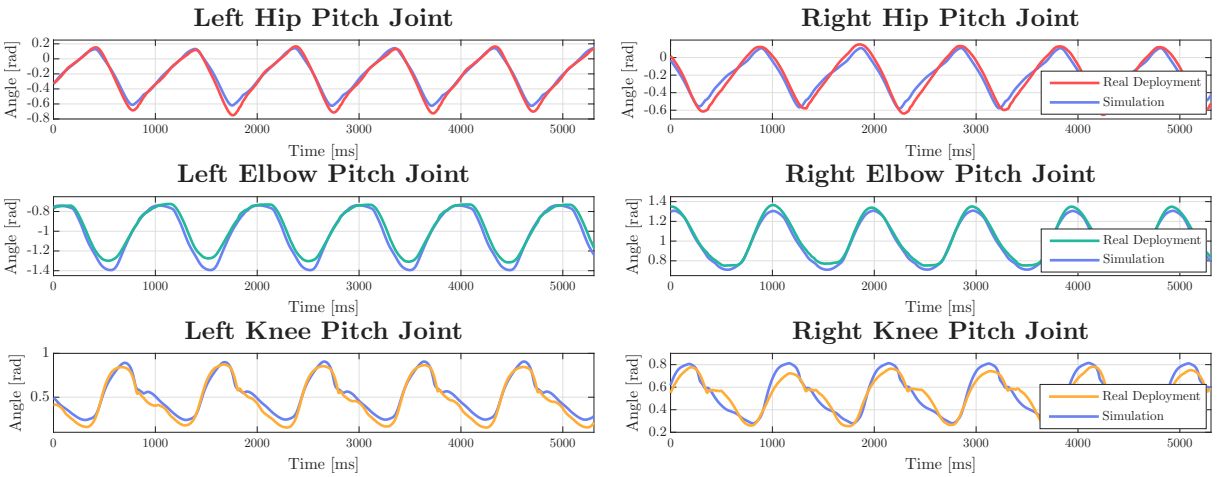
Figures A.12 and A.13 compare the real deployments of the whole-body and reference motion policies with simulation runs under the same command and gait frequency. Over the hip and knee, both deviate from simulation by only 6% and 8% of the range of motion ($r = 0.99$ and 0.98), on par with the legged policy (8%, $r = 0.97$).



(a) Left Side Joint Angles in a Forward Walk Cycle. (b) Right Side Joint Angles in a Forward Walk Cycle.

Figure A.12: Real Deployment *vs* Simulation of the Whole-Body Policy.

The arms show the clearest difference between the two policies. The reference motion policy swings its elbow through about $35°$ per stride, and the robot follows the simulated swing with $r = 0.99$ and $r = 1.00$ on the left and right ($3.3°$ and $1.9°$ RMSE), while the whole-body policy holds its elbows nearly still in both. The reference motion gait is also more symmetrical between sides. Comparing each left joint with the right one half a stride later, its arms swing as mirror images ($r = 0.99$, against 0.51 for the whole-body policy) and its knee trajectories are closer in shape ($r = 0.96$ against 0.93).

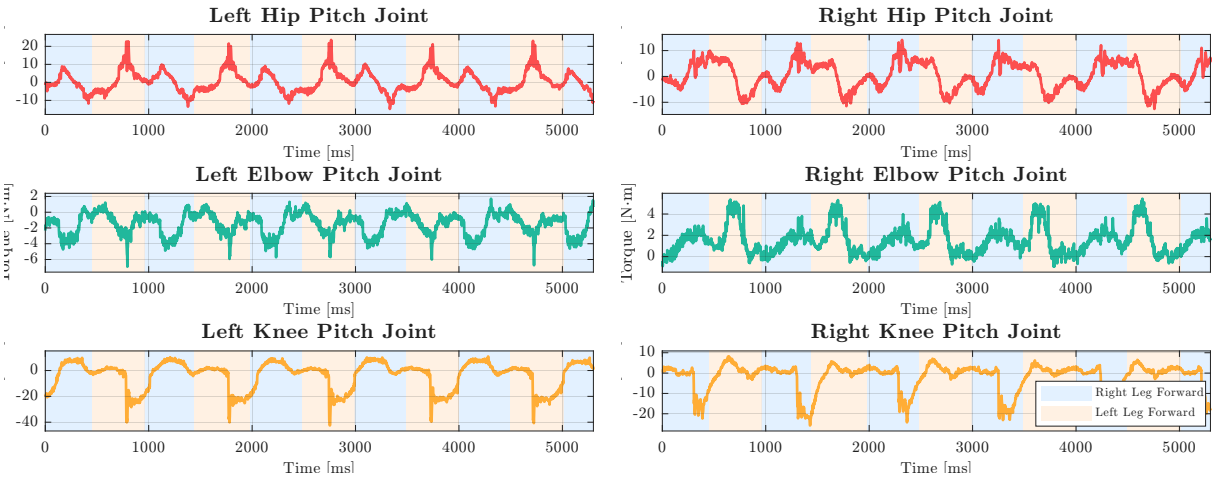


(a) Left Side Joint Angles in a Forward Walk Cycle. (b) Right Side Joint Angles in a Forward Walk Cycle.

Figure A.13: Real Deployment *vs* Simulation of the Reference Motion Policy.

A.8 Joint Torques of the Walking Policies in Deployment

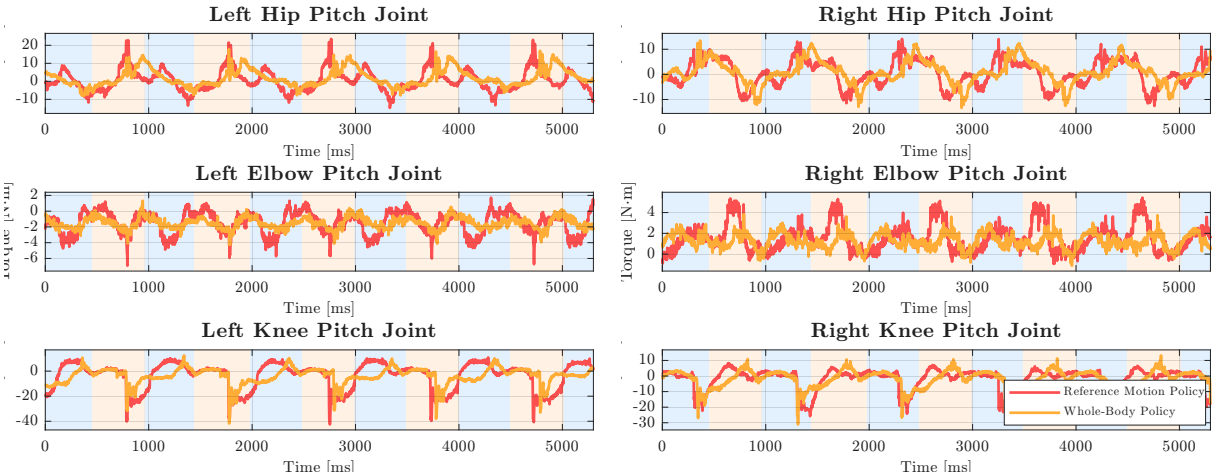
The joint angles of Appendix A.7 are complemented here by the corresponding joint torques over the same forward walk. Figure A.14 shows the torques of the reference motion policy on the real robot, where the knee carries the largest load, peaking at -42 N·m on the left and -26 N·m on the right, while the hip stays within 24 N·m and the elbow, driving the arm swing, within 7 N·m.



(a) Left Side Joint Torques in a Forward Walk Cycle. (b) Right Side Joint Torques in a Forward Walk Cycle.

Figure A.14: Joint Torques of the Reference Motion Policy Deployed on the Real Robot.

Figure A.15 compares them with the whole-body policy. Both load the knee similarly, peaking at about -42 N·m on the left leg, while the reference motion policy uses more torque at the hip (up to 24 N·m against 18) and at the elbow (up to 7 N·m against 4), where it drives its larger arm swing.



(a) Left Side Joint Torques in a Forward Walk Cycle. (b) Right Side Joint Torques in a Forward Walk Cycle.

Figure A.15: Joint Torques of the Reference Motion and Whole-Body Policies on the Real Robot.

A.9 Supplementary Results of the Height Traversal Policies

Figures A.16 and A.17 report the full-flight success rates referred to in Section 5.4: ascent on the additional robots, and descent on the Booster T1, across the trained step heights and tread depths.

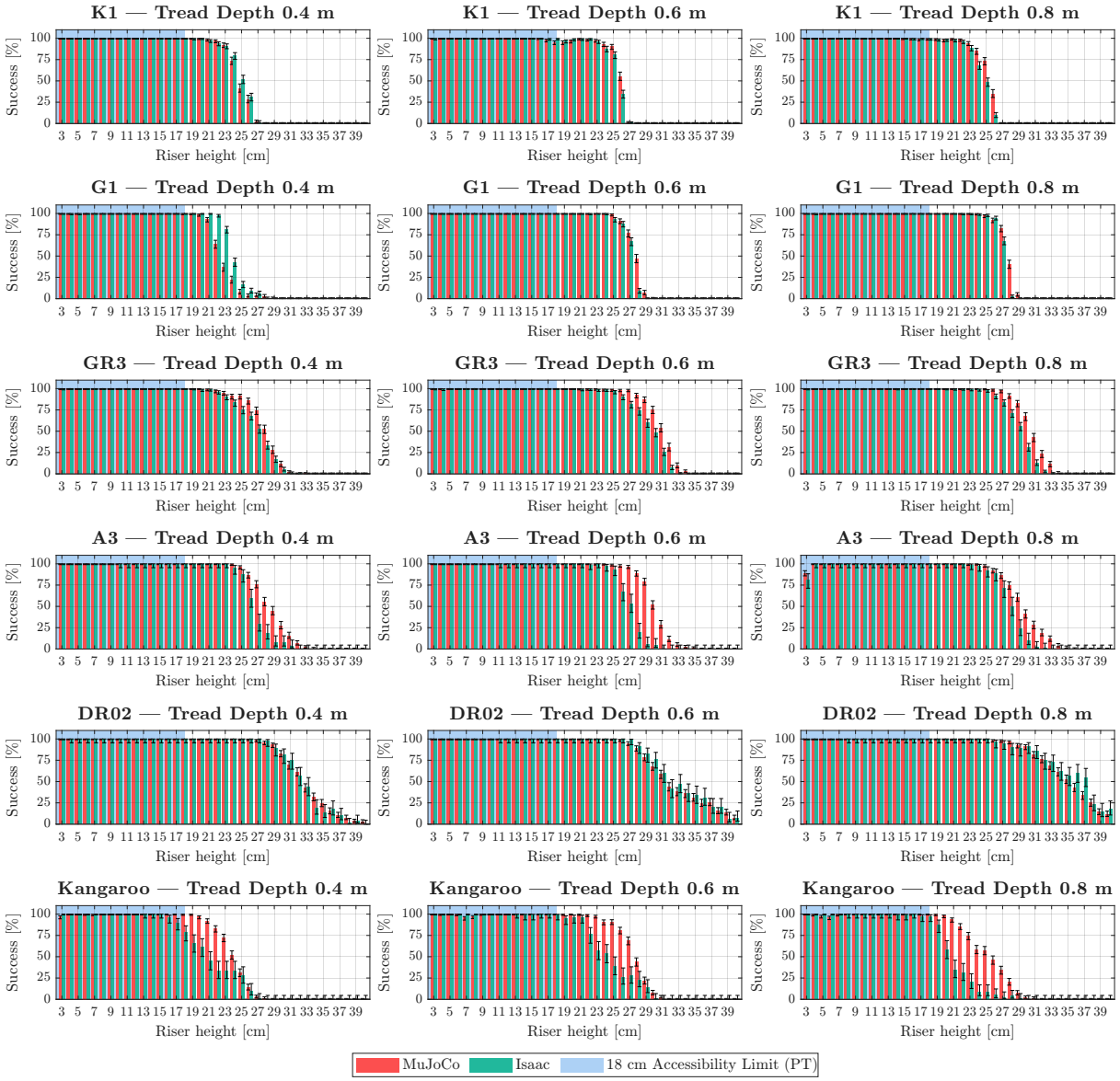


Figure A.16: Full-Flight Ascent Success on the Additional Robots.

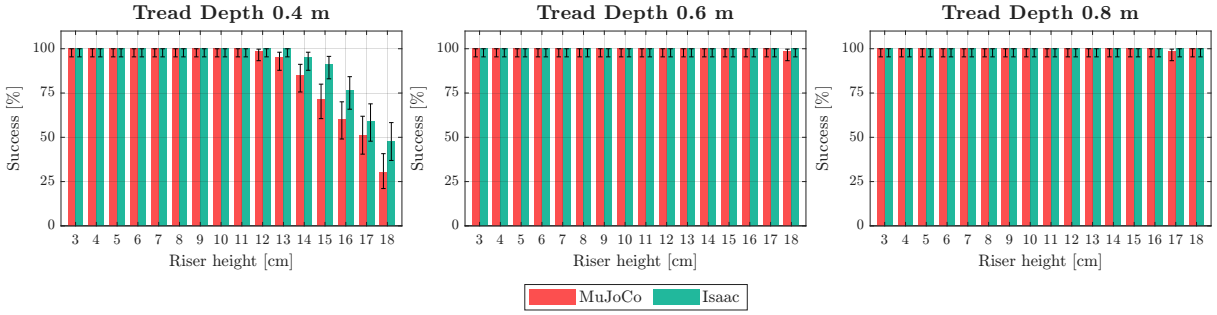


Figure A.17: Full-Flight Descent Success on the Booster T1.